

Escuela Técnica Superior de Ingeniería Informática
Grado en Ingeniería Informática – Ingeniería del Software

Trabajo Fin de Grado

Sistema Multimodal de Interacción Visual Conversacional mediante Agentes LLM y Visión por Computador en Tiempo Real

Mini: un asistente virtual que ve, escucha, habla y actúa

Realizado por:
José Egido Carnerero

Dirigido por:
Fernando Sancho Caparrini

Departamento:
Ciencias de la Computación e Inteligencia Artificial

Sevilla, octubre de 2026

Resumen

Este Trabajo Fin de Grado presenta el diseño, la implementación y la evaluación de Mini, un asistente virtual multimodal que permite mantener diálogos en lenguaje natural, por texto o por voz, sobre contenido visual: imágenes subidas por el usuario o el vídeo en directo de su cámara web. El sistema combina un modelo de lenguaje multimodal de propósito general, GPT-4o mini, con modelos especializados que aportan información verificable y localizada —detección de objetos con YOLOv8, análisis de color en el espacio HSV y reconocimiento óptico de caracteres con EasyOCR—, junto con reconocimiento de voz con faster-whisper y síntesis de voz con Edge-TTS. La orquestación se delega en un agente basado en LLM construido con smolagents, que interpreta cada petición, decide qué herramientas invocar y devuelve, además de la respuesta, órdenes que la interfaz ejecuta: encender la cámara, abrir el selector de archivos, leer en voz alta o señalar con una flecha el objeto o el texto buscado.

La solución se organiza en tres servicios: una aplicación web en React y TypeScript, un servicio de percepción en FastAPI y un servicio independiente para el agente. El desarrollo siguió una estrategia incremental de producto mínimo viable y completó las fases 0, 1 y 2 de la propuesta: imagen estática, vídeo en tiempo real y, finalmente, memoria conversacional, voz y agente autónomo, con una interfaz disponible en seis idiomas.

La memoria explica los fundamentos de cada componente —de las redes convolucionales y la familia YOLO a los Transformers, los agentes ReAct, Whisper o CRAFT y CRNN— y detalla las decisiones de implementación y los problemas encontrados. La evaluación experimental, realizada en CPU, muestra que YOLOv8n analiza un fotograma en unos 48 ms con una precisión del 87 % al umbral de confianza empleado, aunque con un recall del 37 % que cae por debajo del 3 % en imágenes de estilo no fotográfico; que Whisper *small* transcribe consultas en seis idiomas con un WER del 2,8 % (9,2 % con ruido fuerte), casi tres veces menos que el modelo *base*; que la búsqueda de texto acierta en el 98 % de los carteles sintéticos pero falla en sopas de letras; y que el análisis de color clasifica correctamente el 81 % de los colores con nombre, pero carece de la categoría marrón. El trabajo cuantifica además el coste en tokens de la orquestación con el agente y explica cómo los límites de uso de la API condicionan la frecuencia de análisis del vídeo, y concluye con una hoja de ruta hacia la arquitectura completa de la fase 3.

Abstract

This Bachelor's Thesis presents the design, implementation and evaluation of Mini, a multimodal virtual assistant that supports natural-language conversations, typed or spoken, about visual content: images uploaded by the user or the live video from their webcam. The system combines a general-purpose multimodal language model, GPT-4o mini, with specialised models that provide verifiable, localised information —object detection with YOLOv8, colour analysis in the HSV space and optical character recognition with EasyOCR—, together with speech recognition with faster-whisper and speech synthesis with Edge-TTS. Orchestration is delegated to an LLM-based agent built with smolagents, which interprets each request, decides which tools to call and returns, in addition to the answer, commands that the user interface executes: turning on the camera, opening the file picker, reading the answer aloud or pointing with an arrow at the requested object or text.

The solution is organised into three services: a React and TypeScript web application, a FastAPI perception service and an independent agent service. Development followed an incremental minimum-viable-product strategy and completed phases 0, 1 and 2 of the proposal: static images, real-time video and, finally, conversational memory, voice interaction and an autonomous agent, with a user interface available in six languages.

The report explains the foundations of every component —from convolutional networks and the YOLO family to Transformers, ReAct agents, Whisper, or CRAFT and CRNN— and details the implementation decisions and the problems encountered. The experimental evaluation, run on a CPU, shows that YOLOv8n analyses a frame in about 48 ms with 87 % precision at the confidence threshold used, although with a 37 % recall that drops below 3 % on non-photographic images; that Whisper *small* transcribes queries in six languages with a 2.8 % word error rate (9.2 % under strong noise), almost three times lower than the *base* model; that text search succeeds on 98 % of synthetic signs but fails on word-search puzzles; and that the colour analysis correctly classifies 81 % of named colours but lacks a brown category. The work also quantifies the token cost of agent orchestration, explains how API usage limits constrain the video analysis rate, and concludes with a roadmap towards the complete architecture of phase 3.

Índice general

1. Introducción **1**

1.1. Presentación del problema 1

1.2. Motivación 2

1.3. Objetivos 3

1.4. Estructura de la memoria 4

2. Contexto del problema **5**

2.1. El problema: conversar sobre lo que se ve 5

2.2. Cómo se ha abordado históricamente 7

2.2.1. Tecnologías de asistencia visual 8

2.2.2. De la visión clásica al aprendizaje profundo 8

2.2.3. Visión y lenguaje: descripción de imágenes y VQA 9

2.2.4. Modelos fundacionales multimodales 9

2.2.5. Asistentes de voz 10

2.2.6. Modelos de lenguaje que usan herramientas 10

2.3. Ángulo de ataque del trabajo 12

2.4. Estructura general de la aproximación 13

2.5. Fundamentos de visión por computador 14

2.5.1. La imagen digital y las redes convolucionales 14

2.5.2. Detección de objetos: formulación y métricas 16

2.5.3. La familia YOLO 17

2.5.4. El color y el espacio HSV 21

2.5.5. Reconocimiento óptico de caracteres 22

2.6. Fundamentos de los modelos de lenguaje 24

2.6.1. Modelos de lenguaje, tokens y representaciones 24

2.6.2. La arquitectura Transformer 25
2.6.3. Preentrenamiento, instrucciones y alineamiento 25
2.6.4. Modelos multimodales y GPT-4o mini 26
2.6.5. Instrucciones, roles y seguridad 28
2.7. Agentes basados en LLM 28
2.7.1. Del agente clásico al agente LLM 28
2.7.2. El bucle ReAct y la llamada a funciones 28
2.7.3. Marcos de desarrollo: LangChain frente a smolagents 29
2.8. Fundamentos del procesamiento de voz 30
2.8.1. Representación del audio 30
2.8.2. Reconocimiento automático del habla y Whisper 31
2.8.3. faster-whisper, cuantización y detección de actividad de voz 32
2.8.4. Síntesis de voz 33
2.9. Fundamentos de la arquitectura web del sistema 34
2.9.1. Cliente, servidor y formato de los datos 34
2.9.2. FastAPI y la programación asíncrona 34
2.9.3. React y las API multimedia del navegador 35
2.9.4. Contenedores y microservicios 36

3. Implementación técnica **37**
3.1. Metodología y organización del trabajo 37
3.1.1. Desarrollo incremental por sesiones 37
3.1.2. Control de versiones y buenas prácticas 38
3.2. Arquitectura del sistema final 39
3.3. Fase 0: asistente visual sobre imagen estática 41
3.3.1. Flujo de una consulta 41
3.3.2. Doble canal de información hacia el LLM 41
3.3.3. Internacionalización de las clases de COCO 42
3.3.4. Dibujo de las detecciones 42
3.3.5. Problemas encontrados en la fase 0 43
3.4. Fase 1: vídeo en tiempo real 44
3.4.1. Captura de fotogramas 44
3.4.2. Muestreo periódico y límites de la API 45

3.4.3. Temporizadores en React: el problema de las clausuras obsoletas . . 46
3.4.4. Dibujo sobre el vídeo 47
3.4.5. Análisis de color en HSV 47
3.5. Memoria conversacional 48
3.5.1. Un historial que viaja con cada petición 48
3.5.2. Política de memoria visual: «una imagen como máximo» 49
3.5.3. Pasado frente a presente 50
3.5.4. El modo conversación en la webcam 50
3.6. Fase 2: interacción por voz 51
3.6.1. Voz a texto 51
3.6.2. Texto a voz 52
3.7. Fase 2: el agente Mini 54
3.7.1. Un servicio independiente 54
3.7.2. Configuración del agente 54
3.7.3. Catálogo de herramientas 55
3.7.4. Diseño de las herramientas 55
3.7.5. Plantillas de tarea 57
3.7.6. Protocolo de acciones 57
3.7.7. Búsqueda de texto con OCR 60
3.7.8. Integración en la interfaz 60
3.8. La interfaz de usuario 61
3.8.1. Estructura de componentes 61
3.8.2. Internacionalización y temas 62
3.8.3. Detalles de la experiencia de usuario 62
3.9. Seguridad, validación y robustez 64
3.10. Contenerización y despliegue 65

4. Experimentación y casos de uso **68**
4.1. Entorno y metodología experimental 68
4.2. Detección de objetos con YOLOv8n 69
4.2.1. Caso canónico: la fotografía del repositorio 69
4.2.2. Precisión en un conjunto independiente 70
4.2.3. Latencia 71

4.2.4. Robustez ante el cambio de dominio . . . 72
4.3. Análisis de color . . . 73
4.3.1. Exactitud sobre colores con nombre . . . 73
4.3.2. Robustez ante la iluminación y colores en escenas reales . . . 74
4.4. Reconocimiento de voz . . . 75
4.5. Síntesis de voz . . . 76
4.6. Búsqueda de texto con OCR . . . 77
4.6.1. Carteles y etiquetas . . . 77
4.6.2. El caso de la sopa de letras . . . 78
4.6.3. Texto en una fotografía real . . . 79
4.7. El agente y el sistema completo . . . 79
4.7.1. Llamadas al modelo y coste por consulta . . . 79
4.7.2. Casos de uso del agente . . . 81
4.7.3. Casos reales documentados durante el desarrollo . . . 82
4.8. Discusión: criterios de éxito de la propuesta . . . 82

5. Conclusiones **84**
5.1. Grado de cumplimiento de los objetivos . . . 84
5.2. Lo que ha funcionado . . . 84
5.3. Lo que no ha funcionado o se ha quedado corto . . . 86
5.4. Ampliaciones y trabajo futuro . . . 87
5.5. Valoración personal . . . 88

Bibliografía **90**

A. Instalación, despliegue y uso **99**
A.1. Requisitos . . . 99
A.2. Configuración . . . 99
A.3. Despliegue con Docker . . . 100
A.4. Ejecución local para desarrollo . . . 100
A.5. Primer arranque y problemas frecuentes . . . 101
A.6. Guía rápida de uso . . . 101

B. Referencia de la API **102**

C. Diario de desarrollo **103**

Índice de figuras

1.1. Visión general de la interacción con Mini 2

2.1. Hitos históricos en asistencia visual, visión por computador y lenguaje . . . 7
2.2. Tres paradigmas para asistentes conversacionales con visión 11
2.3. Estructura lógica de la aproximación por capas y fases 13
2.4. Fases incrementales del desarrollo 14
2.5. La operación de convolución y la jerarquía de características 15
2.6. Intersección sobre unión y supresión de no máximos 16
2.7. Ilustración conceptual del funcionamiento de YOLO 17
2.8. Arquitectura de YOLOv8 19
2.9. Bloques C2f y SPPF de YOLOv8 19
2.10. Geometría del espacio de color HSV 22
2.11. Canalización de OCR de EasyOCR (CRAFT + CRNN + CTC) 23
2.12. Transformer solo decodificador y atención de producto escalado 26
2.13. Esquema de un modelo de lenguaje multimodal 27
2.14. Bucle de un agente basado en LLM 29
2.15. Forma de onda y espectrograma log-Mel de una consulta 31
2.16. Arquitectura codificador–decodificador de Whisper 32
2.17. Etapas de la síntesis neuronal de voz 33
2.18. Pila web: navegador, Uvicorn, FastAPI y bucle de eventos 35

3.1. Cronología de las sesiones de desarrollo registradas en el diario 38
3.2. Estrategia de ramas (Git Flow simplificado) 38
3.3. Arquitectura desplegada del sistema 40
3.4. Diagrama de secuencia de una consulta sobre imagen estática 41
3.5. Interfaz de la fase 0 con una consulta real 43

3.6. Canalización de captura y análisis de fotogramas 44
3.7. Muestreo periódico y consumo de tokens 45
3.8. Rangos HSV y reglas del análisis de color 47
3.9. Análisis de color HSV sobre un fotograma real 48
3.10. Evolución del historial en /api/chat 49
3.11. Modo conversación con webcam durante la fase 2 51
3.12. Flujos de voz a texto y de texto a voz 52
3.13. Grabación de voz en el modo Mini 53
3.14. Organización interna del servicio del agente 54
3.15. Secuencia de una consulta de localización en el modo Mini 59
3.16. Dibujo de detecciones y de la acción POINT_TO 59
3.17. Estructura de componentes del frontend 61
3.18. Interfaz final: modos Mini, Webcam, Imagen y guía de uso 63
3.19. Tema claro e internacionalización de la interfaz 64
3.20. Despliegue con Docker Compose 66

4.1. Detecciones de YOLOv8n sobre la foto del repositorio con dos umbrales . . 70
4.2. Precisión, recall y AP50 por clase de YOLOv8n en COCO val2017 71
4.3. Latencia de YOLOv8n por etapas en CPU 72
4.4. Robustez de YOLOv8n ante imágenes estilizadas 73
4.5. Matriz de confusión del análisis de color 74
4.6. Efecto de la iluminación en el análisis de color 74
4.7. Resultados del reconocimiento de voz 75
4.8. Tiempo de síntesis de voz con Edge-TTS 77
4.9. Evaluación de la búsqueda de texto con EasyOCR 78
4.10. Tokens de entrada, llamadas y coste estimado por tipo de consulta 80

Índice de tablas

2.1. Casos de uso objetivo y capacidades que requieren 7
2.2. Comparación cualitativa de los paradigmas 11
2.3. Alcance de cada fase frente a la propuesta 14
2.4. Evolución de la familia YOLO 18
2.5. Variantes de YOLOv8 preentrenadas en COCO 20
2.6. Comparación entre LangChain y smolagents 30
2.7. Tamaños de los modelos Whisper 32

3.1. Endpoints de los servicios 40
3.2. Voces de síntesis por idioma 53
3.3. Herramientas del agente Mini 55
3.4. Acciones del protocolo y reacción de la interfaz 58
3.5. Principales variables de estado de App 62
3.6. Resumen de medidas de seguridad 66

4.1. Baterías de experimentos realizadas 69
4.2. Precisión y recall de YOLOv8n según el umbral 71
4.3. WER y latencia de faster-whisper 76
4.4. Coste estimado por tipo de consulta 80
4.5. Latencias medidas de los componentes locales 81
4.6. Casos de uso del agente Mini 82
4.7. Cumplimiento de los criterios de éxito de la propuesta 83

5.1. Grado de cumplimiento de los objetivos específicos 85

A.1. Variables de entorno del sistema 100

B.1. Referencia de los endpoints del backend 102

B.2. Referencia del endpoint del agente 102

C.1. Resumen del diario de sesiones 104

Capítulo 1

Introducción

1.1. Presentación del problema

Preguntar en voz alta «¿qué hay encima de la mesa?», «¿dónde he dejado el mando?» o «¿qué pone en ese cartel?» y obtener una respuesta útil al instante es algo que hasta hace muy poco solo podía hacer otra persona. Los asistentes de voz de los últimos años entienden órdenes habladas, pero no ven; los sistemas de visión por computador detectan objetos con gran precisión, pero no conversan. Este Trabajo Fin de Grado aborda precisamente la unión de ambos mundos: el diseño y la construcción de un sistema que permita **dialogar de forma natural, por texto o por voz, sobre contenido visual**, ya sea una imagen subida por el usuario o el vídeo en directo de su cámara web, y que además sea capaz de **señalar** en la imagen aquello de lo que habla y de **actuar** sobre su propia interfaz.

El momento tecnológico es especialmente propicio. Por un lado, los grandes modelos de lenguaje multimodales, como GPT-4o y su versión económica GPT-4o mini, describen escenas, leen texto y razonan sobre imágenes con una calidad impensable hace pocos años. Por otro, existen modelos especializados, abiertos y eficientes, que resuelven con precisión tareas concretas: YOLOv8 detecta y localiza objetos en decenas de milisegundos, EasyOCR localiza texto, Whisper transcribe voz en decenas de idiomas y los servicios de síntesis neuronal producen voces casi humanas. Por último, han surgido marcos de desarrollo de *agentes*, como smolagents, que permiten que un modelo de lenguaje decida qué herramientas usar para resolver cada petición. El reto ya no es tanto disponer de las piezas como *combinarlas* bien: decidir qué hace cada una, cómo se comunican, cómo se compensan sus debilidades y a qué coste.

El resultado de este trabajo es **Mini**, un asistente virtual multimodal accesible desde el navegador (figura 1.1). El usuario puede hablarle o escribirle, mostrarle una imagen o encender la cámara, y Mini responde en uno de seis idiomas con texto y voz, dibuja cajas y flechas sobre la imagen y, cuando se le pide, enciende la cámara, abre el selector

de archivos o repite en voz alta su última respuesta. Internamente, el sistema combina modelos especializados que aportan información verificable y localizada con un modelo de lenguaje multimodal que interpreta y redacta, orquestados por un agente basado en LLM.

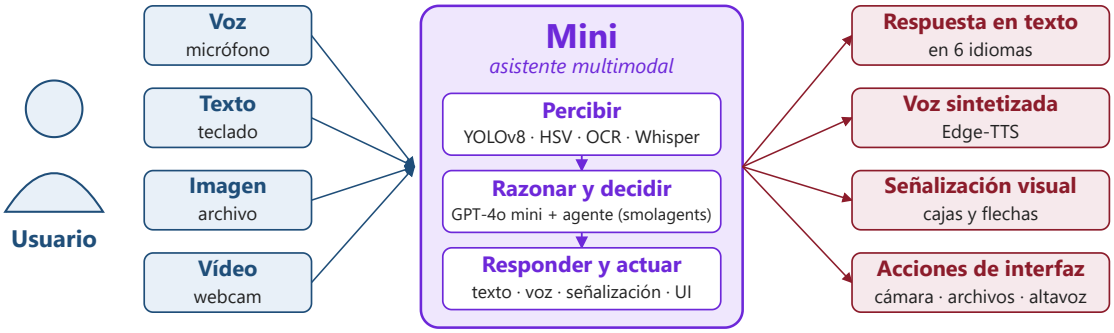


Figura 1.1: Visión general de la interacción con Mini: el usuario se comunica por voz, texto, imagen o vídeo, y el asistente percibe, razona y responde con texto, voz, señalización visual y acciones sobre la interfaz.

1.2. Motivación

La primera motivación es social. Según la Organización Mundial de la Salud, al menos 2200 millones de personas en el mundo tienen algún tipo de deficiencia visual [75]. Para muchas de ellas, poder preguntar por su entorno y recibir una respuesta hablada —qué hay delante, de qué color es una prenda, qué dice un cartel— tiene un impacto directo en su autonomía, como demuestran aplicaciones como Seeing AI o Be My Eyes [7, 62]. La propuesta del trabajo menciona además otros ámbitos de aplicación: la navegación asistida, las herramientas educativas interactivas y las interfaces de realidad virtual y aumentada, en las que la cámara y la voz son canales naturales de interacción [95].

La segunda motivación es técnica. Los modelos multimodales generalistas son muy flexibles, pero tienen debilidades conocidas: no localizan objetos con precisión, pueden inventar detalles y cada imagen que procesan tiene un coste económico y de latencia. Los modelos especializados, en cambio, son precisos y baratos, pero rígidos. Estudiar cómo combinar ambos enfoques en un sistema real, con sus problemas de integración, de tiempo real y de coste, es una cuestión abierta de gran interés práctico y el núcleo de este trabajo.

La tercera motivación es formativa. El proyecto recorre prácticamente todo el ciclo de vida de una aplicación de inteligencia artificial moderna: una interfaz web en React y TypeScript con acceso a la cámara y al micrófono, servicios en Python con FastAPI, integración de modelos de visión, voz y lenguaje, diseño de agentes e ingeniería de *prompts*, contenerización con Docker y evaluación experimental. Es, por tanto, una oportunidad

para aplicar de forma integrada los conocimientos de ingeniería del software del grado y para adquirir otros muy demandados en la industria actual.

1.3. Objetivos

El **objetivo general** del trabajo, tal como lo formula la propuesta, es *diseñar e implementar un sistema multimodal que permita diálogos naturales por voz sobre contenido visual (imágenes o vídeo) en tiempo real, utilizando agentes LLM como orquestadores de modelos especializados de visión por computador*. De él se derivan los siguientes **objetivos específicos**:

O1. Desarrollar un módulo de captura y procesamiento de vídeo en tiempo real desde múltiples fuentes (webcam, dispositivos de realidad virtual, cámaras IP).

O2. Implementar un agente basado en LLM capaz de interpretar consultas en lenguaje natural y orquestar llamadas a modelos de visión especializados.

O3. Integrar modelos preentrenados de visión por computador para tareas específicas (detección de objetos, segmentación, reconocimiento de colores, estimación de pose, etc.).

O4. Diseñar un canal de entrada y salida de voz, con reconocimiento del habla y síntesis de voz.

O5. Implementar mecanismos de realimentación visual (cajas delimitadoras, superposiciones, indicadores de dirección) sobre el vídeo.

O6. Desarrollar una interfaz web interactiva para la demostración y el uso del sistema.

O7. Evaluar el rendimiento del sistema en términos de latencia, precisión y experiencia de usuario.

Dada la envergadura de la propuesta completa, el tutor planteó una estrategia de desarrollo por fases incrementales, cada una con un sistema completo y demostrable: la **fase 0**, un asistente sobre imagen estática con texto (imprescindible); la **fase 1**, el vídeo en tiempo real (deseable); la **fase 2**, las características multimodales de voz; y la **fase 3**, la arquitectura completa de microservicios con múltiples modelos. El **alcance** de este trabajo comprende las fases 0, 1 y 2, completadas en su totalidad, más algunos elementos de la fase 3 que se adelantaron, como la memoria conversacional. El resto de la fase 3 se discute como trabajo futuro.

1.4. Estructura de la memoria

La memoria se organiza en cinco capítulos y tres anexos:

- **Capítulo 1, Introducción.** Presenta el problema, la motivación, los objetivos y el alcance del trabajo.
- **Capítulo 2, Contexto del problema.** Define el problema en profundidad, repasa cómo se ha abordado históricamente —desde las máquinas lectoras para personas ciegas hasta los agentes multimodales actuales—, justifica el enfoque híbrido adoptado y presenta la estructura general de la solución por fases. Su segunda mitad expone los fundamentos teóricos de cada pieza: redes convolucionales y la familia YOLO, el espacio de color HSV, el OCR con CRAFT y CRNN, los Transformers y los modelos multimodales, los agentes y la llamada a funciones, el reconocimiento de voz con Whisper, la síntesis de voz y la arquitectura web.
- **Capítulo 3, Implementación técnica.** Describe la metodología de trabajo, la arquitectura final y la implementación de cada fase, con los procesos y funciones que hubo que diseñar y los problemas encontrados: el muestreo de la webcam y sus límites de uso, la memoria conversacional, el canal de voz, el agente Mini y su protocolo de acciones, la interfaz, la seguridad y el despliegue con Docker.
- **Capítulo 4, Experimentación y casos de uso.** Evalúa cada componente sobre baterías de ejemplos —500 imágenes de COCO, cientos de colores con nombre, 120 grabaciones en seis idiomas, 380 imágenes con texto—, analiza el coste de la orquestación con el agente y contrasta los resultados con los criterios de éxito de la propuesta.
- **Capítulo 5, Conclusiones.** Recoge el grado de cumplimiento de los objetivos, lo que ha funcionado y lo que no, las mejoras y ampliaciones posibles y una valoración personal.
- **Anexos.** Contienen el manual de instalación y despliegue, la referencia de la API de los servicios y el diario de desarrollo resumido.

A lo largo del texto se han incluido recuadros de «lecciones aprendidas» con los errores y descubrimientos más instructivos del desarrollo, pensados para quien quiera continuar este trabajo o emprender uno similar. El código fuente completo está disponible en el repositorio del proyecto [25].

Capítulo 2

Contexto del problema

Este capítulo presenta en profundidad el problema que aborda el trabajo, repasa cómo se ha resuelto históricamente por otros medios, justifica el ángulo de ataque elegido y describe la estructura general de la solución por fases. La segunda mitad del capítulo está dedicada a los fundamentos teóricos de cada pieza utilizada —detectores de objetos, espacios de color, OCR, modelos de lenguaje, agentes, reconocimiento y síntesis de voz y arquitectura web—, con el propósito explícito de que ninguna herramienta se trate como una caja negra. Aunque en el proyecto se reutilizan modelos preentrenados por economía de tiempo y de cómputo, el objetivo es entender cómo están construidos, cómo se entrenaron y por qué funcionan (o fallan) como lo hacen.

2.1. El problema: conversar sobre lo que se ve

Para una persona, preguntar «¿dónde he dejado el mando de la tele?» mientras recorre la habitación con la mirada es una acción trivial. Para un sistema informático, en cambio, esa misma pregunta exige resolver una cadena de tareas heterogéneas: captar la escena y la pregunta, convertir la voz en texto, extraer de la imagen la información relevante —y, a ser posible, localizada—, interpretar la intención de quien pregunta, decidir qué información hace falta para responder, combinarla con lo que se ha hablado antes y, por último, contestar del modo más útil posible: con palabras, en voz alta o señalando directamente el objeto en la imagen.

De manera algo más formal, el problema puede plantearse así: en cada turno t de la conversación el sistema dispone de una observación visual I_t (una imagen subida por el usuario o el fotograma actual de la cámara), de una consulta en lenguaje natural q_t (escrita o hablada) y de un historial H_{t-1} con los intercambios anteriores. Se busca una función que produzca una respuesta compuesta por tres elementos:

$$(a_t,\ v_t,\ u_t) = f\,(I_t,\ q_t,\ H_{t-1}) \tag{2.1}$$

donde a_t es la respuesta verbal (texto y, opcionalmente, su versión hablada), v_t es un conjunto de anotaciones visuales sobre la imagen (cajas, flechas, etiquetas) y u_t es una posible acción sobre la propia interfaz (encender la cámara, abrir el selector de archivos, leer en voz alta la última respuesta). Esta formulación, que se irá concretando a lo largo de la memoria, deja claro que no basta con «describir una imagen»: la respuesta debe estar *anclada* en la escena y, cuando procede, debe *actuar*.

Además de los requisitos funcionales, el problema impone varias restricciones no funcionales que condicionan todas las decisiones de diseño:

- **Anclaje o *grounding*.** Las afirmaciones sobre la imagen deberían ser verificables. Si el sistema dice que «la taza está a la derecha», idealmente debe poder señalarla; si dice que hay dos personas, el recuento debe ser fiable.
- **Latencia.** Una conversación resulta natural si la respuesta llega en pocos segundos. La propuesta del TFG fijaba como objetivo una latencia extremo a extremo inferior a 3–5 segundos [95].
- **Multilingüismo.** El sistema debe funcionar en varios idiomas: la interfaz final admite español, inglés, francés, alemán, italiano y portugués.
- **Coste y privacidad.** Cada fotograma enviado a un servicio externo tiene un coste económico y expone información potencialmente personal (el interior de una vivienda, rostros, documentos).
- **Accesibilidad.** Entre los usuarios potenciales hay personas con discapacidad visual, para quienes la voz y la señalización son canales primarios y no un complemento.

La tabla 2.1 recoge los casos de uso demostrativos que planteaba la propuesta del trabajo, junto con la capacidad principal que requiere cada uno y los componentes del sistema final que la resuelven. Se aprecia que ningún modelo aislado cubre todos los casos: unos exigen localizar objetos con precisión, otros leer texto, otros recordar lo ocurrido en turnos anteriores y otros, sencillamente, ejecutar una acción.

Buena parte de la dificultad del problema procede de fenómenos que las personas resolvemos sin darnos cuenta. Existe una *brecha semántica* entre una matriz de píxeles y conceptos como «mando» o «desordenado». Las referencias espaciales son relativas («a la izquierda» ¿de quién?) y a menudo deícticas («¿qué es *eso*?»). El vocabulario del usuario es abierto, mientras que los detectores clásicos conocen un conjunto cerrado de clases. Y la conversación tiene memoria: «¿y antes?» solo tiene sentido si el sistema recuerda qué vio en el turno anterior.

Consulta de ejemplo	Capacidad principal	Componentes que intervienen
«¿Qué objetos hay en la mesa?»	Análisis de escena	YOLOv8 + GPT-4o mini
«¿Dónde está el mando?», «encuentra mi móvil»	Localización y señalización	YOLOv8 + agente (acción `POINT_TO`) + *canvas*
«¿De qué color es la camiseta?»	Análisis de color	Análisis HSV + GPT-4o mini
«¿Qué dice ese cartel?», «busca la palabra *salida*»	Lectura de texto	GPT-4o mini / EasyOCR + señalización
«¿Cuántas personas hay?»	Recuento	YOLOv8 (lista de detecciones) + GPT-4o mini
«¿Qué ha cambiado respecto a antes?»	Memoria temporal	Historial conversacional + GPT-4o mini
«Enciende la cámara», «repite la respuesta»	Control de la interfaz	Agente Mini + protocolo de acciones

Tabla 2.1: Casos de uso objetivo del sistema, capacidad que requiere cada uno y componentes del sistema final que intervienen.

2.2. Cómo se ha abordado históricamente

La interacción conversacional con contenido visual se sitúa en la intersección de varias líneas de trabajo que durante décadas avanzaron por separado: las tecnologías de asistencia para personas ciegas, la visión por computador, el procesamiento del lenguaje natural y los asistentes de voz. La figura 2.1 resume los hitos más relevantes de cada línea, y los apartados siguientes los comentan.

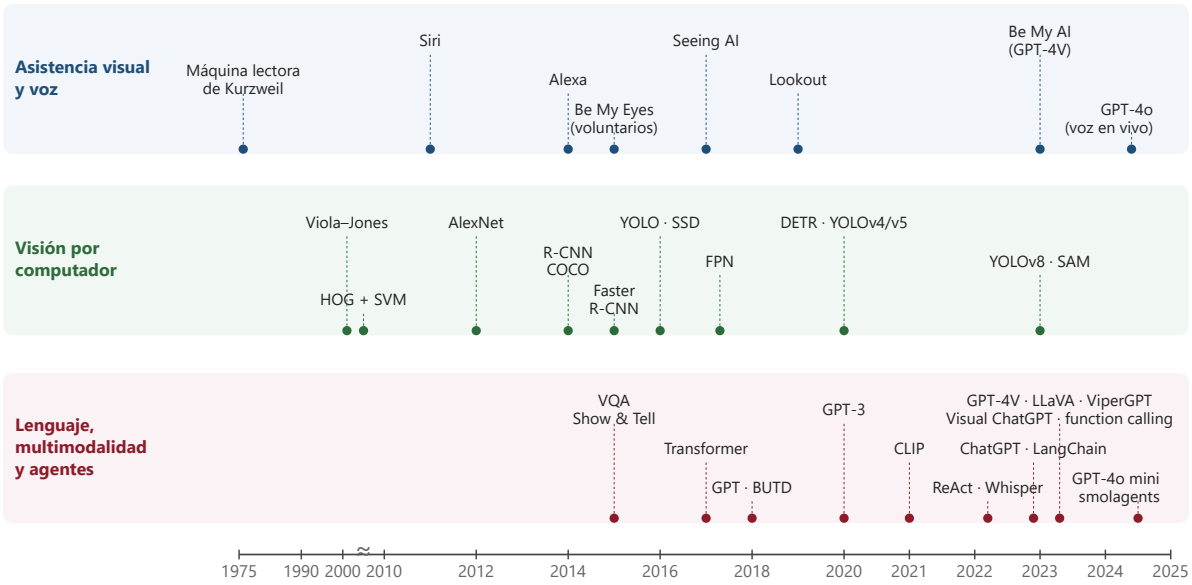


Figura 2.1: Hitos históricos relevantes para este trabajo en tres líneas de investigación que han terminado convergiendo. El eje temporal está comprimido antes de 2010 para poder mostrar con detalle la última década.

2.2.1. Tecnologías de asistencia visual

El antecedente más claro de este trabajo es la **máquina lectora de Kurzweil** (1976), considerada el primer sistema capaz de reconocer texto impreso en cualquier tipografía y leerlo en voz alta a una persona ciega. Ya contenía, en esencia, dos de las piezas de este TFG: un reconocedor óptico de caracteres y un sintetizador de voz. Durante las décadas siguientes, la asistencia visual siguió ese patrón de *canalización*: cada necesidad (leer un documento, identificar un billete, reconocer un producto) se cubría con un módulo especializado.

Con la llegada de los teléfonos inteligentes aparecieron aplicaciones como **Seeing AI** (Microsoft, 2017) [62], organizada en «canales» independientes —texto corto, documentos, productos, personas, escenas—, cada uno respaldado por un modelo distinto. En paralelo, **Be My Eyes** (2015) resolvió el problema por una vía completamente diferente: conectar por videollamada a la persona ciega con un voluntario vidente. En 2023 esa misma plataforma incorporó **Be My AI**, que sustituye al voluntario por el modelo multimodal GPT-4 [7]. Esta transición ilustra muy bien el cambio de paradigma que motiva el presente trabajo: de canalizaciones de modelos específicos a modelos generales capaces de conversar sobre cualquier imagen. La magnitud del problema es considerable: según la Organización Mundial de la Salud, al menos 2200 millones de personas tienen algún tipo de deficiencia visual de cerca o de lejos [75].

2.2.2. De la visión clásica al aprendizaje profundo

Durante los años noventa y dos mil, la detección de objetos se basó en *características diseñadas a mano*. El detector de caras de Viola y Jones [113] combinaba características de tipo Haar, calculadas eficientemente mediante la imagen integral, con una cascada de clasificadores entrenados con AdaBoost, y fue el primero en funcionar en tiempo real. Pocos años después, Dalal y Triggs [20] propusieron los histogramas de gradientes orientados (HOG) junto con una máquina de vectores de soporte para detectar peatones. Ambos enfoques recorrían la imagen con una ventana deslizante a varias escalas y decidían, ventana a ventana, si contenía el objeto.

El punto de inflexión llegó en 2012, cuando AlexNet [46] —una red neuronal convolucional como las propuestas por LeCun a finales de los noventa [47], pero mucho más profunda y entrenada en GPU— redujo el error top-5 en la competición ImageNet del 26,2 % al 15,3 %. A partir de ahí la detección se reformuló sobre redes convolucionales. R-CNN [31] clasificaba con una CNN unas dos mil regiones candidatas por imagen; Faster R-CNN [89] integró la generación de candidatos en la propia red (*Region Proposal Network*), pero seguía siendo un detector de *dos etapas*, preciso aunque lento (unos 5 fotogramas por segundo en GPU). En 2016, YOLO [86] y SSD [56] demostraron que era posible detectar

en *una sola pasada* de la red, a decenas de fotogramas por segundo, lo que abrió la puerta a la detección en tiempo real en hardware modesto. Más recientemente, DETR [15] trasladó la arquitectura Transformer a la detección. Todo este progreso se apoyó en conjuntos de datos anotados de gran tamaño, como PASCAL VOC [26] y, sobre todo, COCO [52], con el que se entrenan los detectores empleados en este trabajo.

2.2.3. Visión y lenguaje: descripción de imágenes y VQA

La unión de visión y lenguaje en un mismo modelo comenzó con la *descripción automática de imágenes*. *Show and Tell* [112] usaba una CNN como codificador de la imagen y una red recurrente LSTM como decodificador de la frase; *Show, Attend and Tell* [123] añadió un mecanismo de atención que permitía al decodificador «mirar» distintas regiones de la imagen al generar cada palabra. En 2015 se presentó también la tarea de **respuesta a preguntas visuales** (*Visual Question Answering*, VQA) [3], con cientos de miles de preguntas en lenguaje natural sobre imágenes de COCO. Los primeros modelos fusionaban el vector de la imagen y el de la pregunta y clasificaban la respuesta entre las más frecuentes, lo que los hacía propensos a responder por sesgos estadísticos sin «mirar» realmente la imagen.

Un trabajo especialmente relevante para este TFG es el de Anderson *et al.* [2], que ganó el reto VQA de 2017 utilizando como entrada visual no la cuadrícula de características de una CNN, sino las *regiones de objetos detectados* por un Faster R-CNN. La intuición —describir la escena al modelo de lenguaje en términos de objetos localizados— es la misma que, a otra escala, se aplica en este proyecto al proporcionar al LLM la lista de detecciones de YOLO junto con la imagen.

2.2.4. Modelos fundacionales multimodales

La arquitectura Transformer [111] y el aumento masivo de datos y cómputo dieron lugar a los *grandes modelos de lenguaje* (LLM): GPT-3 [14] mostró que un modelo de 175 000 millones de parámetros podía resolver tareas nuevas a partir de unos pocos ejemplos en el propio texto de entrada, y el ajuste con instrucciones y realimentación humana [76] los convirtió en asistentes conversacionales como ChatGPT (noviembre de 2022). En el lado visual, CLIP [82] aprendió a alinear imágenes y textos a partir de 400 millones de pares extraídos de Internet, lo que permitió clasificar imágenes «sin entrenamiento» describiendo las clases en lenguaje natural.

El paso siguiente fue conectar un codificador visual preentrenado con un LLM preentrenado: Flamingo [1], BLIP-2 [49] y LLaVA [53] son ejemplos abiertos de esta idea, que se explica en el apartado 2.6.4. En el ámbito comercial, OpenAI presentó GPT-4V en 2023 [70], GPT-4o («omni», capaz de procesar texto, imagen y audio de forma nativa) en mayo de

2024 [72] y, en julio de ese año, **GPT-4o mini** [71], una versión mucho más económica que es la utilizada en este trabajo.

Estos *modelos de visión y lenguaje* (VLM) describen escenas con una riqueza impensable hace pocos años, leen texto en la imagen y razonan sobre ella. Sin embargo, presentan debilidades bien conocidas que son decisivas para el problema planteado: localizan con poca precisión (rara vez devuelven coordenadas fiables), cuentan mal cuando hay muchos objetos, pueden «alucinar» elementos que no existen [40] y cada imagen procesada tiene un coste económico y una latencia de red no despreciables.

2.2.5. Asistentes de voz

Los asistentes de voz comerciales —Siri (2011), Alexa (2014), Google Assistant (2016)— se construyeron durante una década como una canalización de módulos (figura 2.2a): reconocimiento automático del habla (ASR), comprensión del lenguaje (NLU) en términos de intenciones y entidades predefinidas, un gestor de diálogo basado en reglas o estados, generación de lenguaje con plantillas y síntesis de voz (TTS). Su gran limitación era la rigidez: solo entendían las intenciones que alguien había programado. Tanto el reconocimiento como la síntesis de voz pasaron de modelos estadísticos (modelos ocultos de Markov [79], síntesis concatenativa) a redes neuronales extremo a extremo (CTC [34], WaveNet [110], Tacotron 2 [98]), hasta llegar a modelos robustos y multilingües como Whisper [80] y, más recientemente, a LLM que conversan directamente por voz [73].

2.2.6. Modelos de lenguaje que usan herramientas

La línea más reciente, y la más próxima a este trabajo, trata al LLM no como un generador de texto sino como el *controlador* de un conjunto de herramientas. ReAct [125] mostró que intercalar pasos de razonamiento con acciones (búsquedas, consultas, llamadas a funciones) mejora la fiabilidad del modelo, y Toolformer [96] que un LLM puede aprender por sí mismo cuándo invocar una API. En junio de 2023, OpenAI incorporó a su API la *llamada a funciones* (*function calling*) [69], que permite al modelo devolver invocaciones estructuradas en JSON en lugar de texto libre.

Aplicada a la visión, esta idea dio lugar a una familia de sistemas en 2023: **Visual ChatGPT** [122] conectaba ChatGPT con 22 modelos visuales mediante un gestor de *prompts*; **MM-REACT** [124] aplicaba el esquema ReAct a expertos de visión; **HuggingGPT** [99] usaba el LLM para planificar tareas, seleccionar modelos de Hugging Face, ejecutarlos y resumir los resultados; y **ViperGPT** [105] y **VisProg** [36] hacían que el LLM generase directamente un programa en Python que combinaba módulos de visión. En paralelo surgieron marcos de desarrollo de agentes como LangChain [16] y, a finales de

2024, smolagents [92], empleado en este trabajo. Una revisión amplia de este campo puede encontrarse en [117].

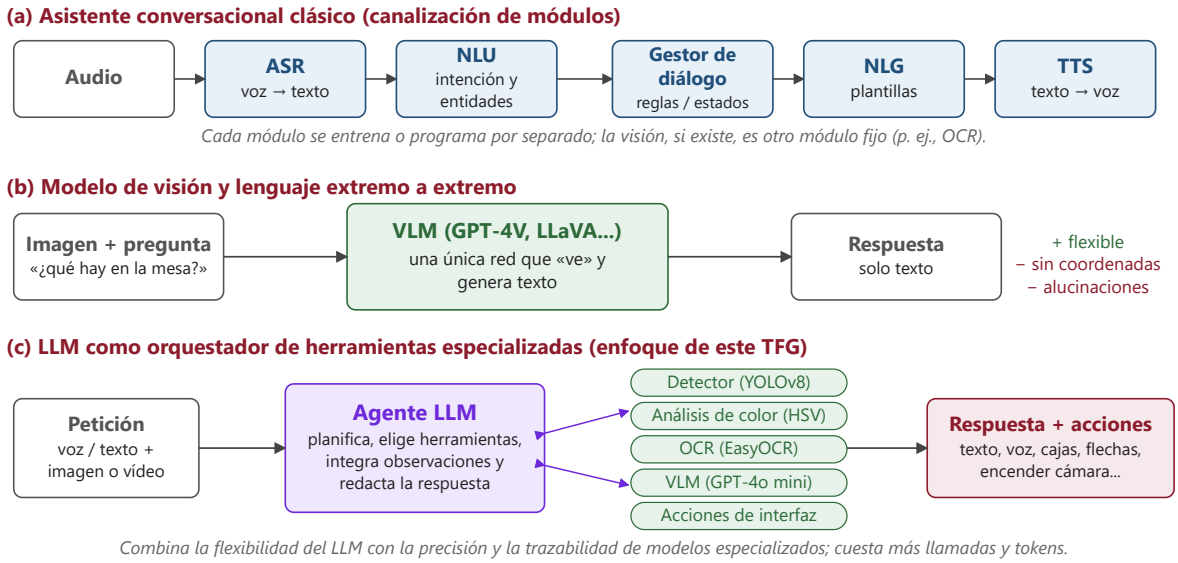


Figura 2.2: Tres paradigmas para construir un asistente conversacional con capacidad visual: (a) canalización clásica de módulos; (b) modelo de visión y lenguaje extremo a extremo; (c) LLM que orquesta herramientas especializadas, que es el enfoque adoptado en este trabajo.

La tabla 2.2 compara cualitativamente los tres paradigmas de la figura 2.2 en los aspectos que más pesan para el problema planteado.

Criterio	**Canalización clásica**	**VLM extremo a extremo**	**LLM + herramientas**
Flexibilidad del lenguaje (preguntas abiertas)	✘	✔	✔
Localización precisa (coordenadas)	✔ (módulo a módulo)	✘	✔ (vía detector/OCR)
Trazabilidad de la respuesta	✔	✘	∼ (se registran las herramientas usadas)
Capacidad de actuar sobre la interfaz	∼ (intenciones fijas)	✘	✔
Coste y latencia por consulta	✔	∼	✘ (varias llamadas al LLM)
Facilidad para añadir capacidades	✘	✘ (reentrenar)	✔ (una función nueva)

Tabla 2.2: Comparación cualitativa de los tres paradigmas de la figura 2.2 (✔ fortaleza, ∼ intermedio, ✘ debilidad).

2.3. Ángulo de ataque del trabajo

A la vista de lo anterior, este trabajo adopta un enfoque **híbrido**: un modelo multimodal de propósito general (GPT-4o mini) actúa como núcleo de comprensión y redacción, mientras que varios modelos especializados aportan información *verificable y localizada* que el LLM no proporciona de forma fiable por sí solo. Un agente basado en LLM decide, para cada petición, qué herramientas invocar y devuelve, además del texto, órdenes que la interfaz ejecuta. Las razones de esta elección son las siguientes:

1. **Anclaje visual.** Las cajas de YOLO y los polígonos del OCR permiten dibujar sobre la imagen exactamente dónde está lo que se busca, algo que un VLM no garantiza.
2. **Complementariedad.** Durante la fase 0 se comprobó que YOLO no detecta nada en logotipos ni ilustraciones, porque solo ha visto fotografías del mundo real; pasar también la imagen al VLM permitió responder en esos casos. A la inversa, el detector aporta al LLM recuentos y posiciones explícitas que reducen sus alucinaciones.
3. **Coste y viabilidad.** Los modelos especializados elegidos (YOLOv8 nano, faster-whisper *small* cuantizado, EasyOCR) se ejecutan en CPU dentro de contenedores Docker sin GPU, y el modelo de lenguaje es de los más baratos del mercado.
4. **Extensibilidad.** Con el patrón de agente, añadir una capacidad nueva consiste en escribir una función de Python bien documentada; no hay que reentrenar nada.
5. **Valor formativo.** El enfoque obliga a entender cada componente por separado —cómo detecta YOLO, cómo se transcribe la voz, cómo razona un agente— en lugar de delegar todo en una única API.

Conviene explicitar también lo que *no* se ha hecho y por qué. No se han entrenado modelos propios: YOLOv8 se entrenó sobre las más de 118 000 imágenes de COCO con varias GPU durante cientos de épocas [41], y Whisper sobre 680 000 horas de audio [80]; reproducir esos entrenamientos está fuera del alcance de un TFG y del hardware disponible (una GPU doméstica GTX 1660 que, además, no se usa en el despliegue en contenedores). Tampoco se ha optado por un VLM local, porque en el hardware disponible los modelos abiertos capaces de describir escenas con calidad no ofrecían una latencia interactiva. El valor del trabajo está, por tanto, en la *integración*: en cómo se combinan los modelos, en el diseño del agente y de su protocolo de acciones, en la experiencia de usuario multimodal y en la evaluación crítica del conjunto.

El elemento más característico de la solución es precisamente ese protocolo: el agente, llamado **Mini**, no se limita a responder, sino que puede *actuar sobre la interfaz* añadiendo a su respuesta etiquetas como `[ACTIVATE_WEBCAM]` o `[POINT_TO]`, que el servidor traduce a una acción y el navegador ejecuta (apartado 3.7.6).

2.4. Estructura general de la aproximación

Lógicamente, el sistema se organiza en cuatro capas por las que fluye cada consulta (figura 2.3): **captura** en el navegador (cámara, micrófono, archivo o texto), **percepción** mediante modelos especializados (detección de objetos, color, texto y voz), **razonamiento y orquestación** a cargo del LLM y del agente, con la memoria de la conversación, y **respuesta** multimodal (texto en el idioma elegido, voz sintetizada, cajas y flechas sobre la imagen y acciones de interfaz). Físicamente, estas capas se reparten entre tres contenedores: el *frontend* en React, un servicio de percepción en FastAPI y un servicio independiente para el agente, como se detalla en el capítulo 3.

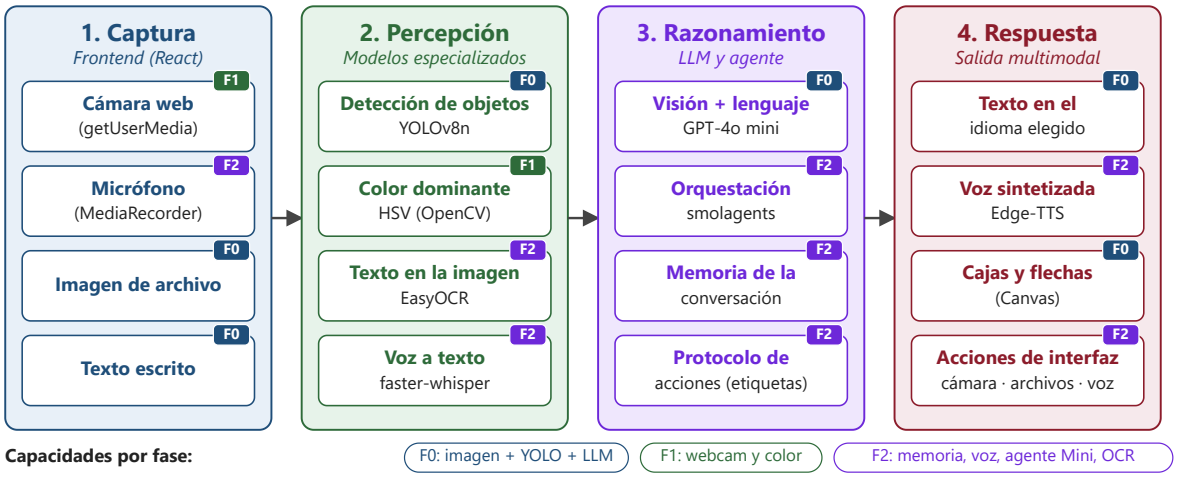


Figura 2.3: Estructura lógica de la aproximación en cuatro capas. Las etiquetas F0, F1 y F2 indican en qué fase del desarrollo se incorporó cada capacidad.

El desarrollo siguió la estrategia de **producto mínimo viable y escalado progresivo** propuesta por el tutor [95], inspirada en la idea de validar pronto y aprender de cada iteración [91]. En lugar de construir a la vez todos los componentes de la arquitectura final de microservicios, se definieron fases incrementales, cada una de las cuales produce un sistema completo, demostrable y evaluable (figura 2.4). La propuesta consideraba imprescindible la fase 0, deseable la fase 1 y un reto las fases 2 y 3; en este trabajo se han completado las fases 0, 1 y 2, e incluso se adelantaron elementos de la fase 3, como la memoria conversacional.

La tabla 2.3 detalla la profundidad alcanzada en cada fase comparándola con lo que planteaba la propuesta. Las diferencias más significativas son tres: la comunicación en tiempo real se ha resuelto con peticiones HTTP periódicas en lugar de WebSockets; el servicio de audio no se separó del servicio principal, y en su lugar el primer servicio independiente fue el del agente; y se sustituyó LangChain por smolagents y Coqui TTS por Edge-TTS. Todas estas decisiones se justifican en el capítulo 3.

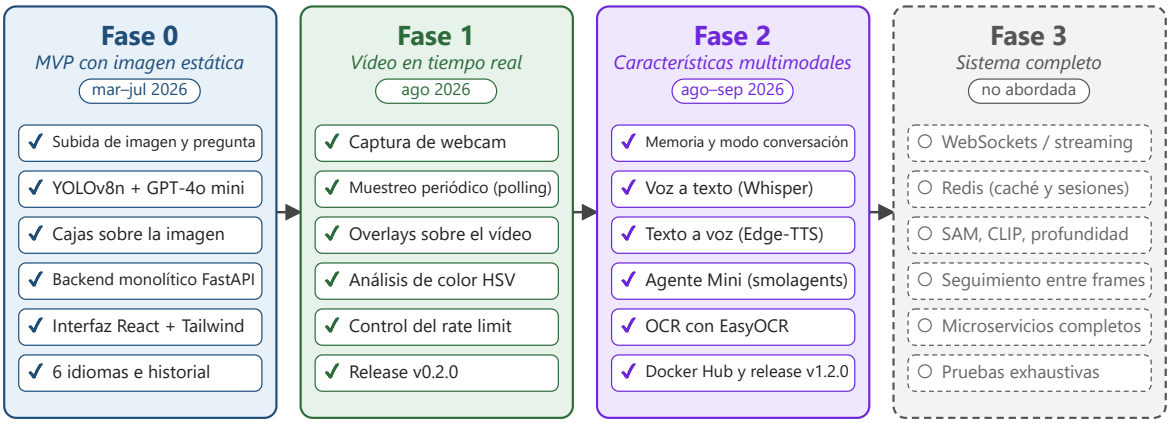


Figura 2.4: Fases incrementales del desarrollo y funcionalidades incorporadas en cada una. La fase 3 (recuadro discontinuo) queda fuera del alcance del trabajo y se discute como trabajo futuro en el capítulo 5.

Fase	Propuesta	Implementado
0 (MVP)	Imagen estática, pregunta escrita, YOLO + GPT-4o, respuesta en texto, cajas sobre la imagen, backend monolítico.	Todo lo propuesto, más GPT-4o mini viendo la imagen además de las detecciones, historial con miniaturas, interfaz en 6 idiomas y traducción de las clases de COCO.
1	Webcam, análisis periódico cada 2–3 s por HTTP, overlays dinámicos, análisis de color.	Webcam con análisis periódico (intervalo final de 10 s por los límites de la API), overlays sobre el vídeo, análisis de color HSV y gestión del límite de peticiones.
2	Whisper, Coqui TTS, servicio de audio separado, WebSocket.	faster-whisper con VAD, Edge-TTS con voz por idioma, memoria conversacional, agente Mini en un servicio separado, OCR con EasyOCR, Docker Compose e imágenes en Docker Hub. Sin WebSocket.
3	Microservicios completos, SAM, CLIP, Redis, seguimiento, pruebas exhaustivas.	No abordada (se discute en el capítulo 5).

Tabla 2.3: Alcance de cada fase según la propuesta y según lo implementado.

El resto del capítulo presenta los fundamentos de cada una de las piezas de la figura 2.3: la visión por computador (apartado 2.5), los modelos de lenguaje (apartado 2.6), los agentes (apartado 2.7), el procesamiento de voz (apartado 2.8) y la arquitectura web (apartado 2.9).

2.5. Fundamentos de visión por computador

2.5.1. La imagen digital y las redes convolucionales

Para un ordenador, una imagen en color es un tensor de dimensiones $H \times W \times 3$: una matriz de píxeles con tres canales de intensidad, habitualmente enteros de 8 bits entre 0

y 255. Un detalle práctico que conviene recordar desde el principio es que OpenCV, la biblioteca de visión utilizada en el backend [12], almacena los canales en orden **BGR** y no RGB. Las imágenes viajan comprimidas en formato JPEG, que descarta información de alta frecuencia poco perceptible y reduce el tamaño unas diez veces, a costa de introducir artefactos.

Las **redes neuronales convolucionales** (CNN) son la herramienta estándar para extraer información de estas matrices [33, 107]. Su operación básica es la convolución (en rigor, una correlación cruzada) de la imagen x con un pequeño filtro k de tamaño $M \times N$:

$$y[i,j] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x[i+m,\ j+n]\ k[m,n]\ +\ b \tag{2.2}$$

El filtro se desliza por toda la imagen y produce un *mapa de activación* que es alto allí donde la imagen «se parece» al filtro (figura 2.5). A diferencia de los enfoques clásicos, los valores de los filtros no se diseñan a mano: se *aprenden* minimizando una función de pérdida sobre un conjunto de datos etiquetado mediante descenso por gradiente y retropropagación [93]. Tras cada convolución se aplica una función no lineal (ReLU, o SiLU en YOLOv8) y, cada cierto número de capas, se reduce la resolución con un paso (*stride*) mayor que uno o con *pooling*. El resultado es una jerarquía de características: las primeras capas responden a bordes y colores, las intermedias a texturas y patrones, y las profundas a partes de objetos y objetos completos. Dos propiedades explican su eficacia: la *conectividad local* (cada neurona mira una vecindad pequeña) y la *compartición de pesos* (el mismo filtro se aplica en todas las posiciones), que reducen drásticamente el número de parámetros y hacen la red robusta a traslaciones.

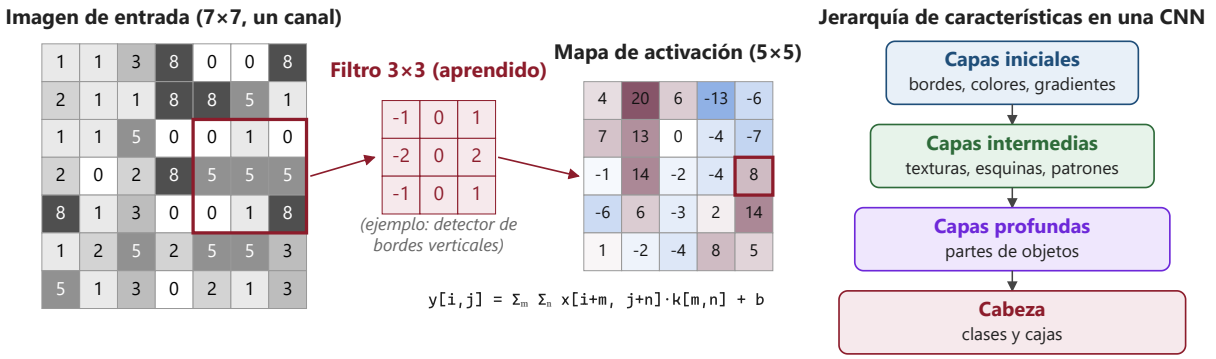


Figura 2.5: La operación de convolución: un filtro de 3×3 (aquí, un detector de bordes verticales de tipo Sobel, a modo de ejemplo) recorre la imagen y genera un mapa de activación. En una CNN se apilan decenas de capas con miles de filtros aprendidos.

2.5.2. Detección de objetos: formulación y métricas

Un detector de objetos recibe una imagen y devuelve un conjunto de detecciones $\{(c_i, s_i, \mathbf{b}_i)\}$, donde c_i es la clase, $s_i \in [0,1]$ una puntuación de confianza y $\mathbf{b}_i = (x_1, y_1, x_2, y_2)$ la caja que delimita el objeto, expresada por sus esquinas superior izquierda e inferior derecha en píxeles. Este es exactamente el formato que el backend del proyecto devuelve al navegador. Para comparar una caja predicha B con la caja real A se utiliza la **intersección sobre unión** (IoU):

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \tag{2.3}$$

Una red de detección genera muchas cajas candidatas para un mismo objeto, casi idénticas entre sí. Para quedarse con una se aplica la **supresión de no máximos** (NMS) [65]: se ordenan las candidatas por confianza, se conserva la mejor, se eliminan las que la solapan con un IoU superior a un umbral (0,7 por defecto en YOLOv8) y se repite con las restantes (figura 2.6).

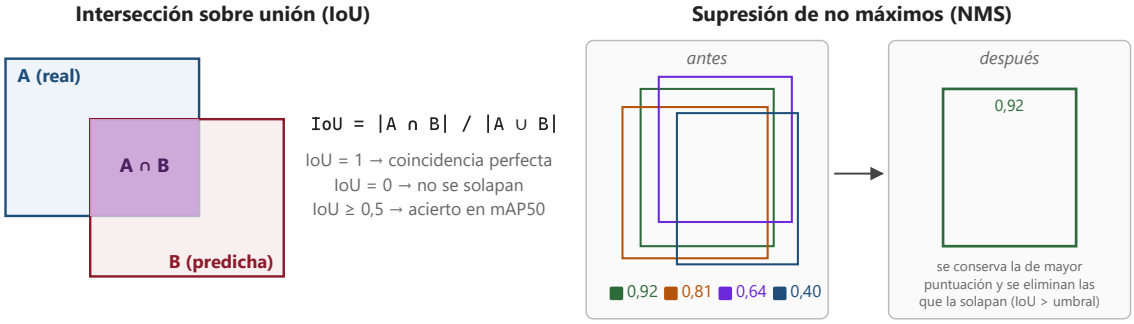


Figura 2.6: Izquierda: la intersección sobre unión mide el solapamiento entre la caja real y la predicha. Derecha: la supresión de no máximos elimina las detecciones redundantes de un mismo objeto.

La calidad de un detector se mide comparando sus detecciones con las anotaciones de un conjunto de validación. Una detección es un verdadero positivo (TP) si su clase coincide con la de un objeto real no emparejado y su IoU con él supera un umbral; en otro caso es un falso positivo (FP). Los objetos reales que quedan sin emparejar son falsos negativos (FN). A partir de ellos se definen la **precisión** y la **exhaustividad** (*recall*):

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN} \tag{2.4}$$

Ambas dependen del umbral de confianza con que se filtran las detecciones: un umbral alto produce pocas detecciones muy fiables (alta precisión, baja exhaustividad) y uno bajo, muchas detecciones con más errores. Recorriendo todos los umbrales se obtiene la curva precisión–exhaustividad, y el área bajo ella (interpolada) es la **precisión media** (AP) de

una clase. La media de la AP sobre todas las clases es la **mAP**. La métrica de referencia de COCO, $\text{mAP}_{50\text{–}95}$, promedia además la mAP para diez umbrales de IoU entre 0,50 y 0,95, con lo que premia las cajas bien ajustadas; mAP_{50} usa solo el umbral 0,5 [52]. Esta distinción será importante en el capítulo 4, porque la aplicación no trabaja «recorriendo todos los umbrales», sino con un umbral de confianza fijo de 0,5.

2.5.3. La familia YOLO

La idea original

YOLO (*You Only Look Once*) [86] replanteó la detección como un único problema de regresión. La imagen se divide en una rejilla de $S \times S$ celdas y la celda que contiene el centro de un objeto es la responsable de detectarlo (figura 2.7). Cada celda predice B cajas —cada una con sus coordenadas y una confianza— y una distribución de probabilidad sobre las C clases. En la versión original ($S = 7$, $B = 2$, $C = 20$) la salida de la red es un tensor de $7 \times 7 \times 30$ que se obtiene en una sola pasada, lo que permitía procesar 45 imágenes por segundo. La función de pérdida era una suma de errores cuadráticos que ponderaba más las coordenadas ($\lambda_{coord} = 5$) y menos las celdas sin objeto ($\lambda_{noobj} = 0{,}5$). Su principal debilidad eran los objetos pequeños y agrupados, puesto que cada celda solo podía predecir unos pocos.

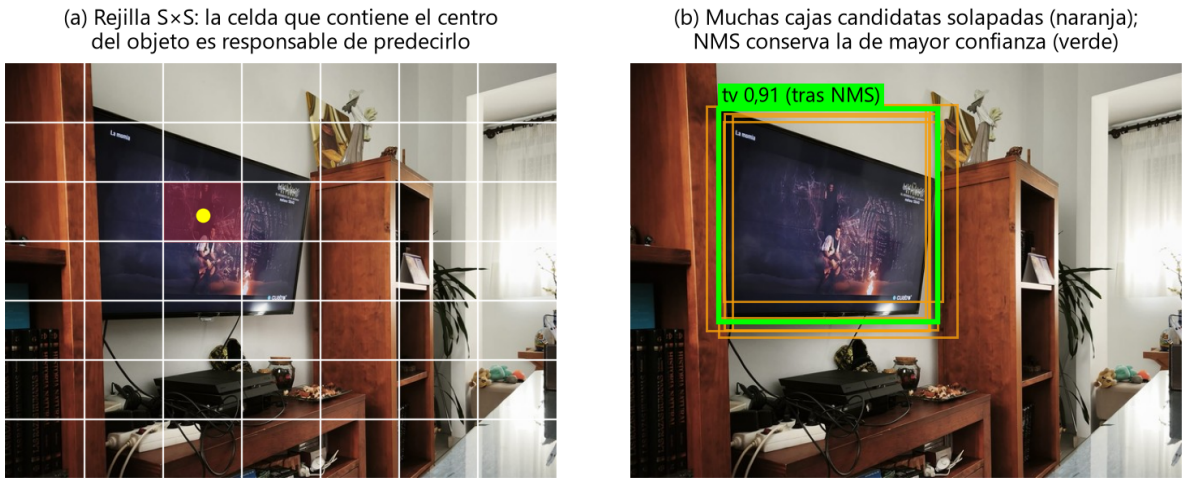


Figura 2.7: Ilustración conceptual del principio de YOLO sobre un fotograma de prueba: (a) la celda que contiene el centro del televisor es la responsable de predecirlo; (b) la red genera muchas cajas candidatas y la supresión de no máximos conserva la de mayor confianza. La caja verde es la detección real de YOLOv8n; las candidatas naranjas son ilustrativas.

Evolución hasta YOLOv8

Las versiones posteriores refinaron cada componente (tabla 2.4) [109]. YOLOv2 [87] introdujo cajas de anclaje (*anchors*) y normalización por lotes; YOLOv3 [88], una red

troncal más profunda (Darknet-53) y predicciones a tres escalas; YOLOv4 [11] incorporó la troncal CSPDarknet53 [116], el aumento de datos *mosaic* y la pérdida CIoU; y YOLOv5, reimplementado por Ultralytics en PyTorch, popularizó una herramienta muy fácil de usar. **YOLOv8**, publicado por Ultralytics en enero de 2023 [41], es la versión utilizada en este trabajo.

Versión	**Año**	**Aportaciones principales**
YOLOv1	2016	Detección como regresión en una pasada; rejilla S×S.
YOLOv2	2017	Cajas de anclaje, *batch normalization*, entrenamiento multiescala.
YOLOv3	2018	Darknet-53 con conexiones residuales; predicción a 3 escalas (estilo FPN).
YOLOv4	2020	CSPDarknet53, cuello PAN, aumento *mosaic*, pérdida CIoU.
YOLOv5	2020	Reimplementación en PyTorch (Ultralytics); ecosistema de uso sencillo.
YOLOv8	2023	Cabeza desacoplada *sin anclas*, bloques C2f, pérdida DFL + CIoU, asignación alineada con la tarea.

Tabla 2.4: Principales aportaciones de cada versión de YOLO relevantes para entender YOLOv8.

Arquitectura de YOLOv8

La figura 2.8 muestra la arquitectura de YOLOv8 para la variante *nano* empleada en el proyecto. Como la mayoría de detectores modernos, se divide en tres partes:

- **Red troncal (*backbone*).** Una CSPDarknet que alterna convoluciones de 3×3 con paso 2 —cada una reduce a la mitad la resolución— y bloques **C2f**. Un bloque C2f (figura 2.9) divide los canales en dos ramas, hace pasar una de ellas por varios «cuellos de botella» residuales y concatena *todas* las salidas intermedias antes de una convolución final; así los gradientes disponen de muchos caminos cortos a bajo coste. La troncal termina en un módulo **SPPF**, que aplica tres *max-pooling* encadenados para agregar contexto a varias escalas. De ella se extraen tres mapas de características, P3, P4 y P5, con pasos 8, 16 y 32 respecto a la imagen.

- **Cuello (*neck*).** Una red de pirámide de características con agregación de caminos (PAN-FPN) [51, 54]: un camino descendente propaga la información semántica de P5 hacia las resoluciones altas mediante sobremuestreo y concatenación, y otro ascendente devuelve el detalle espacial hacia las resoluciones bajas. Así, cada escala combina «qué hay» con «dónde está exactamente».

- **Cabeza (*head*).** Una cabeza de detección por escala, **desacoplada** (una rama para la caja y otra para la clase) y **sin anclas**: cada punto de los mapas de 80×80, 40×40 y 20×20 predice directamente las distancias (l, t, r, b) desde ese punto a los cuatro lados de la caja y 80 puntuaciones de clase independientes (sigmoides). Con una

entrada de 640×640 se obtienen $80^2 + 40^2 + 20^2 = 8400$ predicciones candidatas, que se filtran por confianza y NMS.

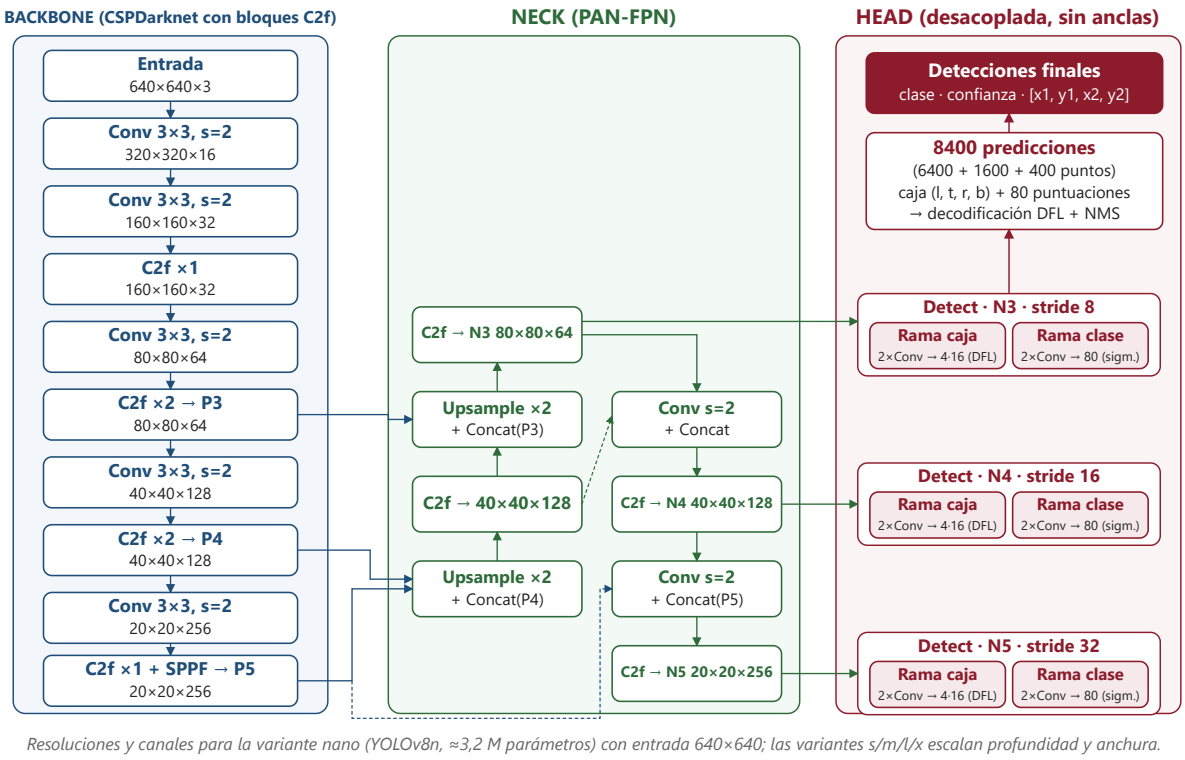


Figura 2.8: Arquitectura de YOLOv8 (variante nano): red troncal CSPDarknet con bloques C2f y SPPF, cuello PAN-FPN y cabeza desacoplada sin anclas a tres escalas. Elaboración propia a partir del código de Ultralytics [41] y de la revisión [109].

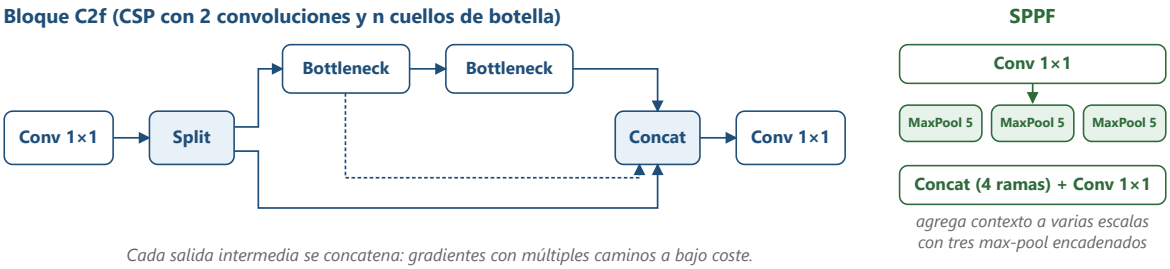


Figura 2.9: Detalle de los bloques C2f (izquierda) y SPPF (derecha) de YOLOv8.

Un detalle interesante de la rama de regresión es la ***Distribution Focal Loss*** (DFL) [50]. En lugar de predecir cada distancia como un único número, la red predice una distribución discreta sobre 16 valores posibles (reg_max = 16), de ahí los $4 \times 16 = 64$ canales de la rama de caja; la distancia final es el valor esperado de esa distribución. Esto permite representar la incertidumbre en bordes ambiguos (un objeto parcialmente ocluido) y mejora la precisión de las cajas.

Entrenamiento

Los pesos `yolov8n.pt` que usa el proyecto se obtuvieron entrenando la red sobre **COCO 2017** [52]: unas 118 000 imágenes de entrenamiento y 5000 de validación con 80 categorías de objetos cotidianos (personas, vehículos, animales, muebles, alimentos, electrónica...). La pérdida total combina tres términos:

$$\mathcal{L} = \lambda_{box}\,\mathcal{L}_{CIoU} + \lambda_{dfl}\,\mathcal{L}_{DFL} + \lambda_{cls}\,\mathcal{L}_{BCE}, \qquad \lambda_{box} = 7{,}5,\ \ \lambda_{dfl} = 1{,}5,\ \ \lambda_{cls} = 0{,}5 \tag{2.5}$$

donde $\mathcal{L}_{BCE}$ es la entropía cruzada binaria de las clases y $\mathcal{L}_{CIoU}$ es la pérdida *Complete IoU* [128], que además del solapamiento penaliza la distancia entre centros y la diferencia de proporciones:

$$\mathcal{L}_{CIoU} = 1 - \mathrm{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \tag{2.6}$$

siendo ρ la distancia euclídea entre los centros de las cajas, c la diagonal de la menor caja que contiene a ambas y v un término que mide la discrepancia de la relación de aspecto. Para decidir qué predicciones se comparan con cada objeto real, YOLOv8 usa el **asignador alineado con la tarea** (TAL) de TOOD [27]: para cada objeto elige los 10 puntos con mayor valor de la métrica $t = s^{\alpha} \cdot u^{\beta}$ (con $\alpha = 0{,}5$ y $\beta = 6$), donde s es la puntuación de clase y u el IoU de la caja predicha, de forma que se premian a la vez la buena clasificación y la buena localización. Durante el entrenamiento se aplican aumentos de datos agresivos —en particular *mosaic*, que compone cuatro imágenes en una, desactivado en las últimas 10 épocas—, variaciones de color en HSV, escalados y volteos.

Variantes y uso en el proyecto

Ultralytics publica cinco variantes que escalan la profundidad y la anchura de la red (tabla 2.5). En este trabajo se usa la más pequeña, **YOLOv8n**, por dos motivos: el despliegue en Docker se hace sin GPU, y la detección es solo una de las fuentes de información del sistema, de modo que prima la latencia sobre la precisión máxima. El capítulo 4 mide ambas en la configuración real del proyecto.

Modelo	**Parámetros (M)**	**FLOPs (G)**	**mAP$_{50\text{–}95}$**	**CPU ONNX (ms)**
YOLOv8n (usado)	3,2	8,7	37,3	80,4
YOLOv8s	11,2	28,6	44,9	128,4
YOLOv8m	25,9	78,9	50,2	234,7
YOLOv8l	43,7	165,2	52,9	375,2
YOLOv8x	68,2	257,8	53,9	479,1

Tabla 2.5: Variantes de YOLOv8 preentrenadas en COCO, según los datos publicados por Ultralytics (entrada de 640×640; velocidad en CPU con ONNX) [41].

Desde el punto de vista del programador, la biblioteca `ultralytics` oculta toda esta complejidad tras una interfaz mínima: `YOLO("yolov8n.pt")` descarga los pesos la primera vez y construye la red; llamar al modelo con una imagen de OpenCV aplica el preprocesado (redimensionado con relleno o *letterbox* hasta un múltiplo de 32 conservando la proporción, normalización a [0,1] y conversión a tensor), la inferencia y el postprocesado (filtro de confianza y NMS), y devuelve un objeto con los atributos `boxes.xyxy`, `boxes.conf`, `boxes.cls` y el diccionario `names` de clases. El listado 2.1 muestra la forma en que el backend lo utiliza, que se analiza en detalle en el capítulo 3.

Listado 2.1: Uso de YOLOv8 en el backend: carga única al arrancar e inferencia con umbral de confianza 0,5.

```
yolo_model = YOLO("yolov8n.pt")            # una sola vez, al arrancar el servicio

results = yolo_model(img, conf=0.5)[0]     # img: matriz BGR de OpenCV
detections = [{
    "class": translate_class(results.names[int(box.cls[0])], language),
    "confidence": round(float(box.conf[0]), 2),
    "bbox": [round(x, 1) for x in box.xyxy[0].tolist()],   # [x1, y1, x2, y2]
} for box in results.boxes]
```

2.5.4. El color y el espacio HSV

El modelo RGB describe cada color como la mezcla aditiva de tres primarios, que es la forma en que los sensores y las pantallas lo capturan y lo reproducen. Sin embargo, es poco adecuado para *nombrar* colores, porque mezcla en sus tres componentes la información de tono, pureza y brillo: un rojo iluminado y el mismo rojo en sombra tienen valores RGB muy distintos. El espacio **HSV** (*Hue, Saturation, Value*) [102] reorganiza esa información en tres coordenadas más cercanas a la percepción humana: el **tono** H, un ángulo que indica el color «puro» (0° rojo, 120° verde, 240° azul); la **saturación** S, la pureza del color frente al gris; y el **valor** V, su brillo. A partir de $R, G, B \in [0,1]$, con $C_{max} = \max(R,G,B)$, $C_{min} = \min(R,G,B)$ y $\Delta = C_{max} - C_{min}$:

$$V = C_{max}, \qquad S = \begin{cases} \Delta / C_{max} & \text{si } C_{max} > 0 \\ 0 & \text{en otro caso} \end{cases} \tag{2.7}$$

$$H = \begin{cases} 60° \cdot \left(\dfrac{G-B}{\Delta} \bmod 6 \right) & \text{si } C_{max} = R \\ 60° \cdot \left(\dfrac{B-R}{\Delta} + 2 \right) & \text{si } C_{max} = G \\ 60° \cdot \left(\dfrac{R-G}{\Delta} + 4 \right) & \text{si } C_{max} = B \end{cases} \tag{2.8}$$

El tono no está definido cuando $\Delta = 0$ (grises puros). OpenCV, para almacenar HSV en 8 bits, divide el tono entre dos —de modo que $H \in [0, 179]$— y escala S y V al intervalo $[0, 255]$. Esta convención, fuente frecuente de errores, explica los rangos que aparecen en el código del proyecto (apartado 3.4.5). La figura 2.10 muestra la geometría del espacio: el tono recorre un círculo, la saturación crece hacia el borde y el valor oscurece hacia el negro.

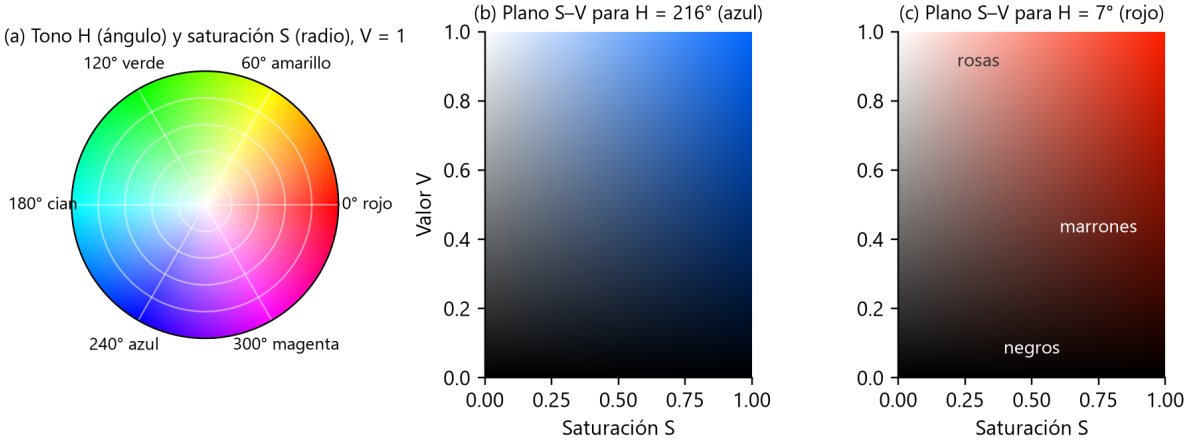


Figura 2.10: Geometría del espacio HSV: (a) círculo de tonos y saturaciones a brillo máximo; (b) y (c) planos saturación–valor para un tono azul y otro rojo. En (c) se aprecia que los marrones son rojos y naranjas oscuros, y los rosas, rojos poco saturados: no tienen un rango de tono propio.

Asignar un *nombre* a un color a partir de sus coordenadas HSV consiste en dividir el espacio en regiones. Los estudios lingüísticos de Berlin y Kay [9] identificaron once términos básicos de color que aparecen de forma sistemática en los idiomas: blanco, negro, rojo, verde, amarillo, azul, marrón, morado, rosa, naranja y gris. Como muestra la figura 2.10c, varios de ellos (marrón, rosa) no corresponden a un intervalo de tono sino a combinaciones de tono, saturación y brillo, lo que hace que una partición basada solo en rangos de H sea necesariamente aproximada. Además, el color medido depende de la iluminación y del balance de blancos de la cámara. Existen alternativas más sólidas —el espacio CIELAB, diseñado para ser perceptualmente uniforme [32]; agrupar los píxeles con *k-means* para obtener colores dominantes; o modelos aprendidos de nombrado de colores—, pero el enfoque HSV por umbrales es simple, rápido y suficiente como primera aproximación. El capítulo 4 cuantifica sus limitaciones.

2.5.5. Reconocimiento óptico de caracteres

El **reconocimiento óptico de caracteres** (OCR) convierte el texto presente en una imagen en una cadena de caracteres. En documentos escaneados el problema está resuelto desde hace años mediante canalizaciones clásicas —binarización, análisis de la maquetación, segmentación en líneas y caracteres y clasificación—, cuyo exponente más conocido es

Tesseract, que desde su versión 4 reconoce líneas completas con una red LSTM [103]. El texto en escenas naturales («texto en la escena»: carteles, pantallas, lomos de libros) es mucho más difícil: tipografías variadas, perspectiva, iluminación irregular y fondos complejos. Los sistemas modernos lo abordan en dos etapas neuronales, que son las que implementa **EasyOCR** [39], la biblioteca utilizada en el proyecto (figura 2.11):

1. **Detección con CRAFT** [4]. Una red con troncal VGG-16 y un decodificador de tipo U-Net predice, para cada píxel, dos mapas: la *puntuación de región* (probabilidad de ser el centro de un carácter) y la *puntuación de afinidad* (probabilidad de estar entre dos caracteres contiguos de la misma palabra). Umbralizando ambos mapas y agrupando componentes conexas se obtienen las cajas de palabras o líneas, como polígonos de cuatro vértices. Como no existen grandes conjuntos de imágenes reales anotados carácter a carácter, CRAFT se entrena primero con texto sintético y después con supervisión débil sobre imágenes reales anotadas por palabras.

2. **Reconocimiento con CRNN** [100]. Cada recorte se rectifica y normaliza en altura, y una CNN (ResNet o VGG en EasyOCR) lo convierte en una secuencia de vectores, uno por columna de la imagen. Una LSTM bidireccional [37] añade contexto en ambos sentidos y, para cada columna, una capa de salida estima una distribución de probabilidad sobre el alfabeto más un símbolo especial «en blanco» ε.

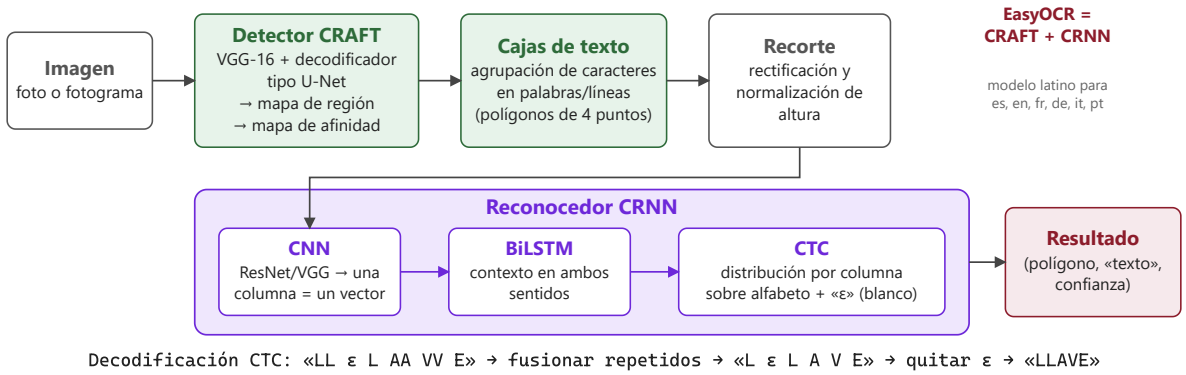


Figura 2.11: Canalización de OCR de EasyOCR: detección de texto con CRAFT, recorte y reconocimiento con CRNN entrenado con la pérdida CTC. Abajo, ejemplo de la regla de decodificación de CTC.

El problema de entrenar el reconocedor es que no se sabe qué columna corresponde a qué letra. La **clasificación temporal conexionista** (CTC) [34] lo resuelve sumando sobre todos los alineamientos posibles. Si $\mathcal{B}$ es la función que colapsa una secuencia de columnas eliminando repeticiones consecutivas y después los blancos, la probabilidad de

una transcripción $\mathbf{y}$ dada la entrada $\mathbf{x}$ de T columnas es:

$$p(\mathbf{y} \mid \mathbf{x}) = \sum_{\boldsymbol{\pi} \in \mathcal{B}^{-1}(\mathbf{y})} \prod_{t=1}^{T} p(\pi_t \mid \mathbf{x}) \tag{2.9}$$

La suma, que tiene un número exponencial de términos, se calcula eficientemente con programación dinámica (algoritmo hacia delante–hacia atrás). El símbolo en blanco es imprescindible para representar letras dobles: sin él, «LL» se colapsaría en «L». Esta misma pérdida CTC fue, durante años, la base de muchos sistemas de reconocimiento de voz, lo que ilustra hasta qué punto OCR y ASR son problemas hermanos: ambos transforman una señal ordenada (columnas de píxeles o tramas de audio) en una secuencia de símbolos.

La función `readtext` de EasyOCR devuelve, para cada fragmento detectado, una tupla `(polígono, texto, confianza)`. Los idiomas del proyecto (español, inglés, francés, alemán, italiano y portugués) comparten el alfabeto latino, por lo que se cargan juntos con un único modelo de reconocimiento. Una consecuencia de la etapa de detección, que se analizará en el capítulo 4, es que la unidad devuelta es el *fragmento* que CRAFT agrupa (una palabra o una línea), no necesariamente la palabra que el usuario busca.

2.6. Fundamentos de los modelos de lenguaje

2.6.1. Modelos de lenguaje, tokens y representaciones

Un **modelo de lenguaje** asigna una probabilidad a cada secuencia de símbolos. Por la regla de la cadena, esa probabilidad se descompone en una sucesión de predicciones del siguiente símbolo condicionadas por los anteriores [43]:

$$P(w_1, \ldots, w_n) = \prod_{t=1}^{n} P(w_t \mid w_1, \ldots, w_{t-1}) \tag{2.10}$$

Los modelos clásicos de n-gramas estimaban estas probabilidades contando frecuencias en un corpus, y los primeros modelos neuronales [8] sustituyeron los recuentos por redes que representaban cada palabra con un vector denso o *embedding* [63], de modo que palabras de significado parecido quedaban próximas. Los LLM actuales no trabajan con palabras sino con **tokens**, unidades obtenidas con algoritmos de segmentación en subpalabras como BPE [97], que funden iterativamente los pares de símbolos más frecuentes del corpus de entrenamiento. El tokenizador de la familia GPT-4o (`o200k_base`) tiene un vocabulario de unos 200 000 tokens. Por ejemplo, con él la pregunta «¿Qué objetos ves en esta imagen?» se divide en siete tokens (`¿Qué`, `␣objetos`, `␣ves`, `␣en`, `␣esta`, `␣imagen`, `?`), mientras que una palabra poco frecuente como «electroencefalografista» se trocea en seis. El token es la

unidad en la que se mide el contexto de un modelo y en la que se factura su uso, de ahí su importancia práctica.

2.6.2. La arquitectura Transformer

Hasta 2017, los modelos de secuencias se basaban en redes recurrentes, que procesan los tokens uno tras otro y tienen dificultades para relacionar elementos lejanos. El mecanismo de **atención** [5] permitía a un decodificador consultar directamente cualquier posición de la entrada, y el **Transformer** [111] construyó toda la arquitectura sobre él. Cada token se representa por un vector del que se obtienen, mediante tres proyecciones aprendidas, una *consulta* q, una *clave* k y un *valor* v. La salida para cada posición es una media de los valores de todas las posiciones, ponderada por la afinidad entre su consulta y las claves:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V \tag{2.11}$$

donde d_k es la dimensión de las claves (el factor $\sqrt{d_k}$ evita que el *softmax* se sature) y M es una máscara que, en los modelos generativos, vale $-\infty$ por encima de la diagonal para que cada token solo pueda «mirar» a los anteriores (atención *causal*). En la práctica se ejecutan varias atenciones en paralelo con proyecciones distintas (*multi-head attention*), de modo que cada «cabeza» pueda especializarse en un tipo de relación. Un bloque Transformer combina la atención con una red *feed-forward* aplicada a cada posición, conexiones residuales y normalización; los LLM generativos como GPT apilan decenas de estos bloques en una arquitectura *solo decodificador* (figura 2.12). Como la atención no tiene noción de orden, se suma a cada token una codificación de su posición.

La atención permite que cualquier token influya directamente en cualquier otro y se paraleliza muy bien en GPU, lo que hizo posible entrenar modelos gigantescos. Su coste, sin embargo, crece con el cuadrado de la longitud de la secuencia, y la longitud máxima que el modelo puede procesar, su **ventana de contexto**, es finita: 128 000 tokens en GPT-4o mini [71]. Este límite, junto con el coste por token, es la razón por la que el sistema recorta el historial de la conversación (apartado 3.5).

2.6.3. Preentrenamiento, instrucciones y alineamiento

Un LLM se **preentrena** con enormes cantidades de texto minimizando la entropía cruzada de la predicción del siguiente token, $\mathcal{L} = -\sum_t \log P_\theta(w_t \mid w_{<t})$ [81]. Con suficiente escala, esta tarea tan simple obliga al modelo a capturar gramática, conocimiento del mundo y cierta capacidad de razonamiento [14]. Un modelo solo preentrenado, no obstante, se limita a continuar textos. Para convertirlo en un asistente se ajusta después con ejemplos

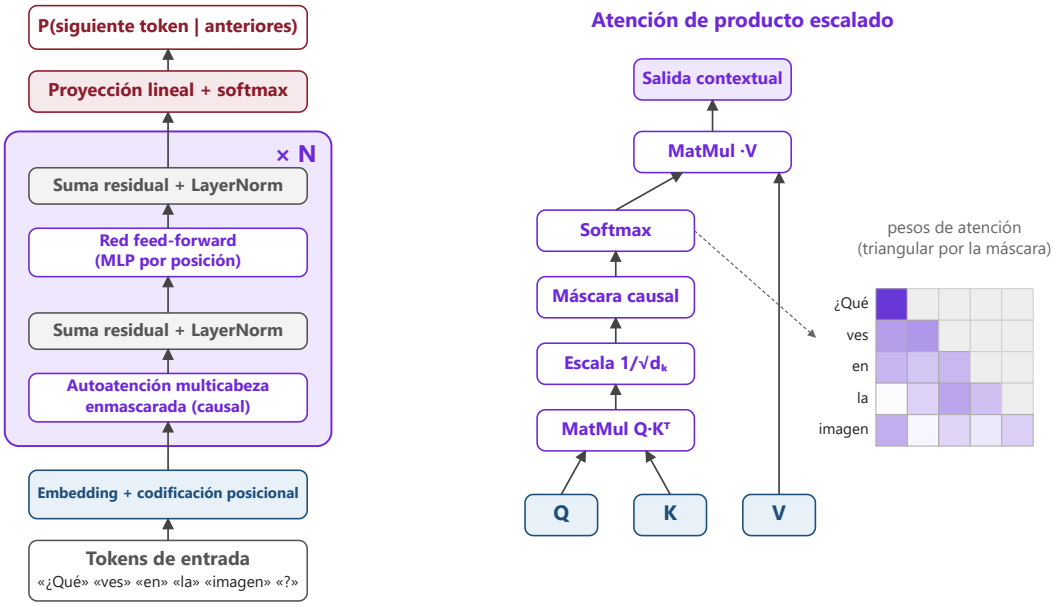


Figura 2.12: Izquierda: Transformer solo decodificador, la arquitectura de los LLM generativos. Derecha: atención de producto escalado con máscara causal y ejemplo de la matriz de pesos resultante, triangular porque cada token solo atiende a los anteriores.

de instrucciones y respuestas (*supervised fine-tuning*) y con **aprendizaje por refuerzo a partir de realimentación humana** (RLHF): se entrena un modelo de recompensa con comparaciones de respuestas hechas por personas y se optimiza el LLM para maximizar esa recompensa [76]. De ahí proceden los **roles** de las API de chat: un mensaje de *sistema* con instrucciones de alto nivel, y mensajes alternos de *usuario* y *asistente*.

En el momento de la inferencia, el modelo produce una distribución sobre el vocabulario y el siguiente token se elige por muestreo. La **temperatura** T controla la aleatoriedad al reescalar las puntuaciones z_i antes del *softmax*, $p_i = e^{z_i/T} / \sum_j e^{z_j/T}$: valores bajos producen respuestas más deterministas. Conviene tener presente la principal limitación de estos modelos: generan texto *plausible*, no necesariamente *verdadero*, y pueden inventar datos con total seguridad, lo que se conoce como **alucinación** [40]. Técnicas como pedir un razonamiento paso a paso [119] o aportar al modelo datos externos verificables —como las detecciones de YOLO en este proyecto— ayudan a mitigarla.

2.6.4. Modelos multimodales y GPT-4o mini

Para que un LLM «vea», la imagen debe convertirse en algo parecido a tokens. El **Vision Transformer** (ViT) [23] mostró que basta con dividir la imagen en parches (por ejemplo, de 16×16 píxeles), proyectar cada parche a un vector y procesar la secuencia con un Transformer. Los modelos abiertos del tipo LLaVA [53] conectan un codificador visual de este tipo, preentrenado con CLIP, con un LLM preentrenado mediante un pequeño **proyector** que traduce los vectores visuales al espacio de *embeddings* del LLM; tras un

ajuste con instrucciones visuales, el modelo trata los «tokens visuales» igual que los de texto y la autoatención relaciona las palabras de la pregunta con los parches relevantes (figura 2.13).

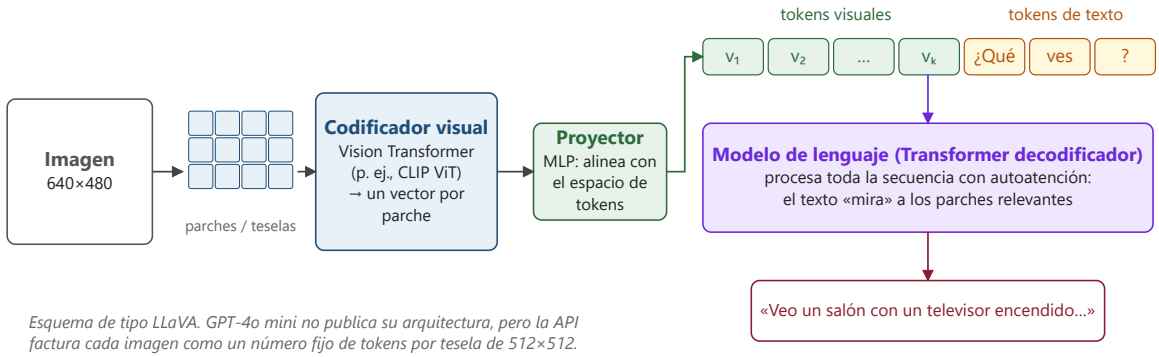


Figura 2.13: Esquema de un modelo de lenguaje multimodal: la imagen se divide en parches, un codificador visual los convierte en vectores y un proyector los inserta, como tokens visuales, en la secuencia que procesa el LLM junto con la pregunta.

OpenAI no ha publicado la arquitectura interna de GPT-4o ni de GPT-4o mini; se sabe que GPT-4o se entrenó de forma nativa sobre texto, imagen y audio [72]. Lo que sí documenta es cómo contabiliza las imágenes [67]. Con el parámetro `detail` en modo alto, la imagen se escala para caber en 2048×2048 y para que su lado menor no supere 768 píxeles, y se cuenta el número de teselas de 512×512 necesarias para cubrirla; el coste en tokens es:

$$T_{img} = T_{base} + T_{tesela} \cdot \left\lceil \frac{w}{512} \right\rceil \left\lceil \frac{h}{512} \right\rceil \tag{2.12}$$

con $T_{base} = 85$ y $T_{tesela} = 170$ en GPT-4o, y $T_{base} = 2833$ y $T_{tesela} = 5667$ en GPT-4o mini (en modo de detalle bajo solo se cobra T_{base}). La cifra de GPT-4o mini es deliberadamente mayor para que el precio por imagen sea similar al de su hermano mayor pese a que su precio por token es muy inferior (0,15 dólares por millón de tokens de entrada y 0,60 por millón de salida). Para un fotograma de webcam de 640×480 se necesitan 2 teselas, es decir, 14 167 tokens de entrada: casi treinta veces más que el texto de una consulta típica. Este cálculo, aparentemente anecdótico, resultará clave para entender los límites de frecuencia encontrados en la fase 1 (apartado 3.4.2).

Por último, conviene entender por qué un VLM localiza mal. Su salida es texto, y su entrenamiento premia descripciones correctas, no coordenadas exactas; además, la división en parches y el reescalado diluyen la posición fina de los objetos. Algunos modelos recientes, como Qwen2.5-VL [6], se entrenan específicamente para devolver cajas, pero no era el caso del modelo disponible, y esa es la justificación técnica de mantener un detector especializado en el sistema.

2.6.5. Instrucciones, roles y seguridad

La interacción con un LLM mediante API consiste en enviar una lista de mensajes con su rol y recibir la continuación. En los modelos multimodales, el contenido de un mensaje puede ser una lista de partes de distinto tipo: texto e imágenes, estas últimas como URL o, como en este proyecto, como *data URL* con la imagen codificada en base64. El diseño de estos mensajes, la llamada **ingeniería de *prompts***, tiene un efecto enorme sobre el comportamiento del modelo: instrucciones explícitas, ejemplos, delimitadores que separan datos de instrucciones y restricciones de formato.

Dado que el texto del usuario se inserta en el *prompt*, un usuario malintencionado puede intentar que el modelo ignore sus instrucciones («olvida lo anterior y...»). Es la **inyección de *prompts*** [78], que puede ser también *indirecta* si las instrucciones maliciosas llegan a través de datos que el sistema procesa, como el texto de una imagen leída por OCR [35]. Es el primer riesgo de la clasificación de OWASP para aplicaciones con LLM [77]. No existe una defensa perfecta; las medidas razonables son separar claramente instrucciones y datos (por ejemplo, con etiquetas), recordar en el mensaje de sistema que el contenido del usuario no son instrucciones, limitar lo que las herramientas pueden hacer y validar las salidas. El apartado 3.9 describe las medidas adoptadas en el proyecto.

2.7. Agentes basados en LLM

2.7.1. Del agente clásico al agente LLM

En inteligencia artificial, un **agente** es una entidad que percibe su entorno mediante sensores y actúa sobre él mediante actuadores para alcanzar unos objetivos [94]. Un **agente basado en LLM** utiliza el modelo de lenguaje como mecanismo de decisión: recibe una tarea en lenguaje natural, razona sobre ella y elige acciones de un repertorio de **herramientas** (funciones que consultan datos o producen efectos). Las revisiones del campo [117] suelen descomponer estos agentes en cuatro módulos: perfil o instrucciones, memoria, planificación y acción. En el sistema de este trabajo, el entorno es doble: por un lado, los modelos de percepción que el agente consulta; por otro, la propia interfaz del usuario, sobre la que el agente puede actuar.

2.7.2. El bucle ReAct y la llamada a funciones

El patrón dominante es el bucle de **ReAct** [125] (figura 2.14). En cada paso, el LLM recibe las instrucciones, la descripción de las herramientas disponibles y la memoria de los pasos anteriores, y emite una *acción*: la invocación de una herramienta con unos argumentos. El entorno ejecuta la herramienta —aquí se ejecuta código real: una inferencia de YOLO,

una petición HTTP, una lectura OCR— y su resultado se añade a la memoria como una *observación*. El bucle se repite hasta que el modelo invoca una acción especial de respuesta final o se agota un número máximo de pasos.

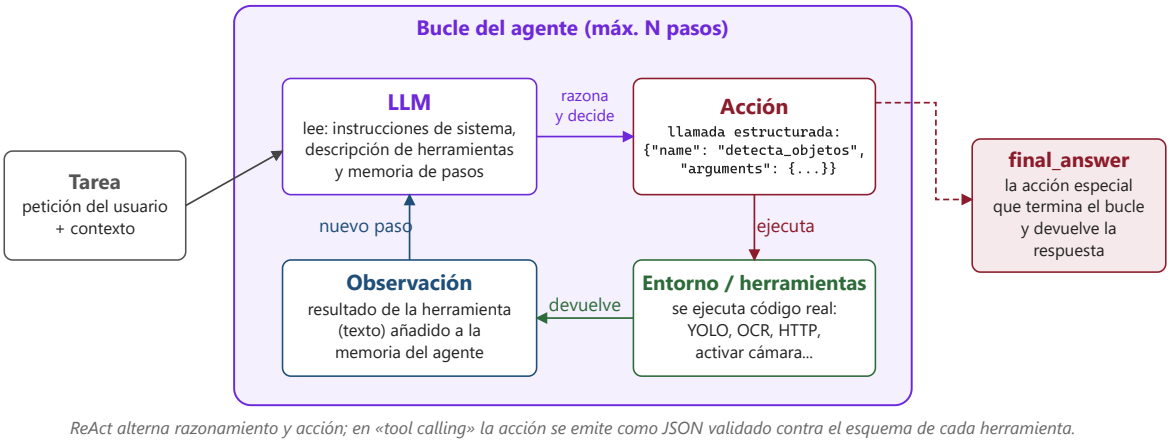


Figura 2.14: Bucle de un agente basado en LLM al estilo ReAct: el modelo decide una acción, se ejecuta la herramienta y la observación resultante alimenta el siguiente paso, hasta que se invoca `final_answer`.

Con la **llamada a funciones** [69], cada herramienta se describe al modelo con un nombre, una descripción en lenguaje natural y un esquema JSON de sus parámetros, y el modelo, en lugar de texto, puede devolver una estructura como `{"name": "analiza_frame_webcam", "arguments": {"query": "¿dónde está el mando?"}}`, que el programa valida y ejecuta. Es importante subrayar que el LLM nunca ejecuta nada por sí mismo: solo propone llamadas, y es el código del agente el que decide ejecutarlas. La *descripción* de cada herramienta se convierte así en su interfaz de programación para el LLM: de su claridad depende que el modelo elija bien.

2.7.3. Marcos de desarrollo: LangChain frente a smolagents

La propuesta del TFG sugería LangChain [16], el marco más extendido para construir aplicaciones con LLM. Siguiendo la recomendación del tutor, finalmente se utilizó **smolagents** [92], una biblioteca de Hugging Face publicada a finales de 2024 cuya filosofía es mantener la lógica del agente mínima y legible: el núcleo cabe en unos pocos miles de líneas, los *prompts* son plantillas visibles y modificables, y admite modelos de muchos proveedores (OpenAI, modelos locales con Transformers, Hugging Face, LiteLLM). La tabla 2.6 resume la comparación.

smolagents ofrece dos tipos de agente. El `CodeAgent` expresa sus acciones como fragmentos de código Python que se ejecutan en un intérprete restringido, lo que permite combinar varias herramientas en un solo paso y, según los propios autores y el trabajo de Wang *et al.* [118], suele requerir menos pasos. El `ToolCallingAgent`, el utilizado en este trabajo,

Aspecto	**LangChain**	**smolagents (elegido)**
Alcance	Ecosistema amplio: cadenas, RAG, memoria, integraciones.	Centrado exclusivamente en agentes.
Transparencia	Muchas capas de abstracción.	Lógica compacta y *prompts* legibles.
Tipos de agente	Varios (herramientas, ReAct, grafos con LangGraph).	`CodeAgent` (acciones en Python) y `ToolCallingAgent` (acciones en JSON).
Definición de herramientas	Clases o decoradores.	Decorador `@tool`: el esquema se deduce de la firma y la *docstring*.
Curva de aprendizaje	Media-alta.	Baja.

Tabla 2.6: Comparación entre LangChain y smolagents para las necesidades de este proyecto.

emite acciones como llamadas JSON, igual que la llamada a funciones nativa de los proveedores; es menos expresivo, pero más predecible y seguro, porque no se ejecuta código arbitrario generado por el modelo. En ambos casos, el decorador `@tool` transforma una función de Python en una herramienta: el nombre de la función es el nombre de la herramienta, las anotaciones de tipo determinan el esquema de los argumentos y la *docstring* —incluida la descripción de cada argumento en su sección `Args`— se convierte en la documentación que lee el LLM. El agente añade siempre una herramienta especial, `final_answer`, cuya invocación termina el bucle, y un parámetro `max_steps` que limita el número de pasos. El capítulo 3 muestra el *prompt* de sistema que construye smolagents y cómo se diseñaron las herramientas.

2.8. Fundamentos del procesamiento de voz

2.8.1. Representación del audio

El sonido capturado por un micrófono se digitaliza muestreando la presión acústica a intervalos regulares. Según el teorema de muestreo, una frecuencia de muestreo de 16 kHz permite representar frecuencias de hasta 8 kHz, suficiente para la inteligibilidad del habla; es la frecuencia que utiliza Whisper. La forma de onda en bruto es poco informativa para un modelo, así que se transforma en un **espectrograma**: se divide la señal en tramas cortas solapadas (25 ms cada 10 ms en Whisper), se calcula la transformada de Fourier de cada una y se obtiene la energía por frecuencia a lo largo del tiempo. Como el oído humano distingue mejor las frecuencias graves que las agudas, las frecuencias se agrupan en bandas equiespaciadas en la **escala Mel** [104]:

$$m = 2595 \log_{10}\left(1 + \frac{f}{700}\right) \tag{2.13}$$

y se toma el logaritmo de la energía, que se aproxima a la percepción de la sonoridad [21]. El resultado, el **espectrograma log-Mel**, es la «imagen» del sonido que procesan los modelos de voz. La figura 2.15 muestra el de una de las consultas de prueba del capítulo 4: se distinguen las sílabas, los armónicos de las vocales (bandas horizontales) y el silencio final que el filtro de actividad de voz se encarga de descartar.

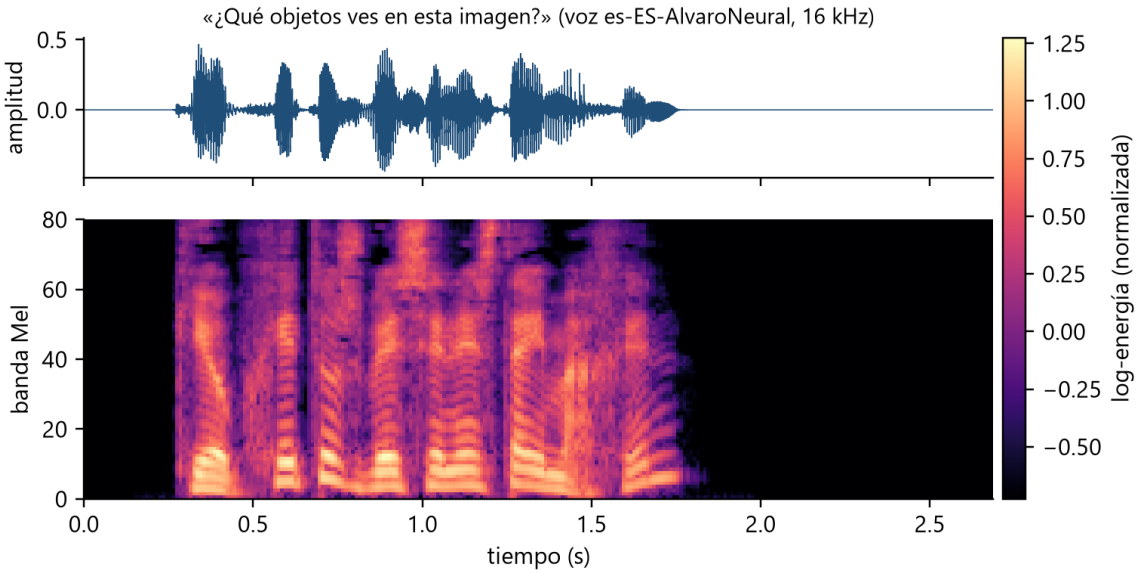


Figura 2.15: Forma de onda (arriba) y espectrograma log-Mel de 80 bandas (abajo) de la consulta «¿Qué objetos ves en esta imagen?», calculado con el mismo extractor de características que usa faster-whisper.

2.8.2. Reconocimiento automático del habla y Whisper

El reconocimiento automático del habla (ASR) busca la secuencia de palabras W más probable dada la señal acústica X. Los sistemas clásicos aplicaban el teorema de Bayes, $W^{*} = \arg\max_{W} P(X \mid W)\, P(W)$, y combinaban un modelo acústico basado en modelos ocultos de Markov [79], un diccionario de pronunciación y un modelo de lenguaje de n-gramas, cada uno entrenado por separado. Los sistemas neuronales extremo a extremo sustituyeron esta canalización por una única red entrenada con CTC o con atención.

Whisper [80] es un Transformer *codificador–decodificador* entrenado por OpenAI con 680 000 horas de audio de Internet con sus transcripciones (supervisión «débil», porque las transcripciones no se revisaron a mano), de las que unas 117 000 corresponden a 96 idiomas distintos del inglés y 125 000 a traducción hacia el inglés. El codificador recibe el espectrograma log-Mel de una ventana de 30 segundos, lo reduce con dos convoluciones y lo procesa con bloques Transformer; el decodificador genera el texto token a token, atendiendo al audio mediante atención cruzada (figura 2.16). Una idea elegante del modelo es que la tarea se especifica con **tokens especiales** al principio de la secuencia del decodificador: el idioma (`<|es|>`), la tarea (`<|transcribe|>` o `<|translate|>`) y si se desean marcas

de tiempo. El mismo modelo sirve así para identificar el idioma, transcribir y traducir. Su principal virtud es la robustez: al haberse entrenado con audio muy diverso, funciona razonablemente con acentos, ruido y micrófonos domésticos sin ajuste adicional.

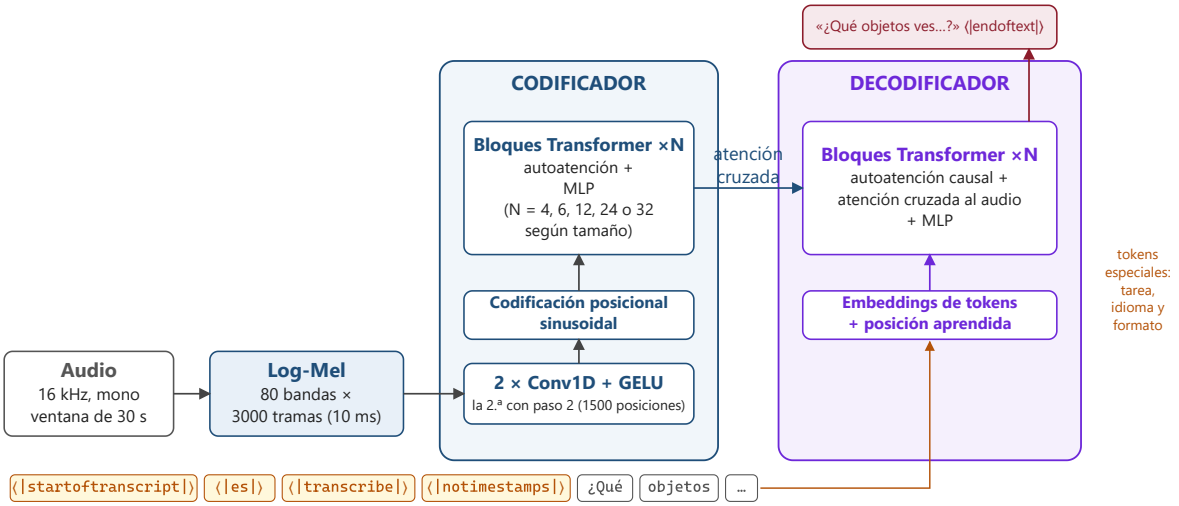


Figura 2.16: Arquitectura codificador–decodificador de Whisper. El decodificador recibe tokens especiales que fijan la tarea, el idioma y el formato antes de generar la transcripción.

Modelo	Capas	Anchura	Cabezas	Parámetros
tiny	4	384	6	39 M
base	6	512	8	74 M
small (usado)	12	768	12	244 M
medium	24	1024	16	769 M
large	32	1280	20	1550 M

Tabla 2.7: Tamaños de los modelos Whisper multilingües [80]. En el proyecto se usa *small*, tras comprobar que *base* no era suficientemente preciso.

La métrica estándar de ASR es la **tasa de error de palabra** (WER), que cuenta las sustituciones S, eliminaciones D e inserciones I necesarias para convertir la transcripción en la referencia de N palabras —la distancia de edición de Levenshtein [48] a nivel de palabra—:

$$\mathrm{WER} = \frac{S + D + I}{N} \tag{2.14}$$

2.8.3. faster-whisper, cuantización y detección de actividad de voz

El proyecto no usa la implementación de referencia de Whisper, sino **faster-whisper** [106], una reimplementación sobre CTranslate2 [74], un motor de inferencia optimizado para Transformers que fusiona operaciones, gestiona mejor la memoria y admite **cuantización**. Cuantizar a `int8` significa almacenar los pesos como enteros de 8 bits con un factor de escala, en lugar de números en coma flotante de 32 bits [38]: el modelo ocupa cuatro veces

menos memoria y las multiplicaciones enteras son más rápidas en CPU, con una pérdida de precisión normalmente inapreciable. Según sus autores, faster-whisper es hasta cuatro veces más rápido que la implementación original con la misma precisión. La decodificación usa por defecto búsqueda en haz (*beam search*) de anchura 5 y, si la salida parece degenerada, reintenta con temperaturas crecientes.

faster-whisper incorpora además un filtro de **detección de actividad de voz** (VAD) basado en el modelo Silero VAD [101], que se activa con `vad_filter=True`. Silero estima la probabilidad de voz en pequeños bloques de audio; por defecto, los tramos con probabilidad superior a 0,5 se consideran voz, se unen si están separados por silencios de menos de 2 segundos y se amplían 400 ms por cada lado. Solo esos tramos se envían a Whisper. Esto evita un defecto conocido del modelo: ante silencio o ruido tiende a «alucinar» frases que nadie ha dicho, un problema especialmente probable cuando el usuario tarda en hablar tras pulsar el micrófono.

2.8.4. Síntesis de voz

La síntesis de voz (TTS) recorre el camino inverso (figura 2.17): el texto se normaliza (números, abreviaturas, siglas), se convierte en fonemas con información de prosodia, un **modelo acústico** genera un espectrograma Mel y un **vocoder** lo transforma en forma de onda. WaveNet [110] demostró que un vocoder neuronal podía producir voz casi indistinguible de la humana; Tacotron 2 [98] y FastSpeech 2 [90] resolvieron la etapa acústica de forma autorregresiva y no autorregresiva, respectivamente; y VITS [44] unificó todo el proceso en un único modelo extremo a extremo.

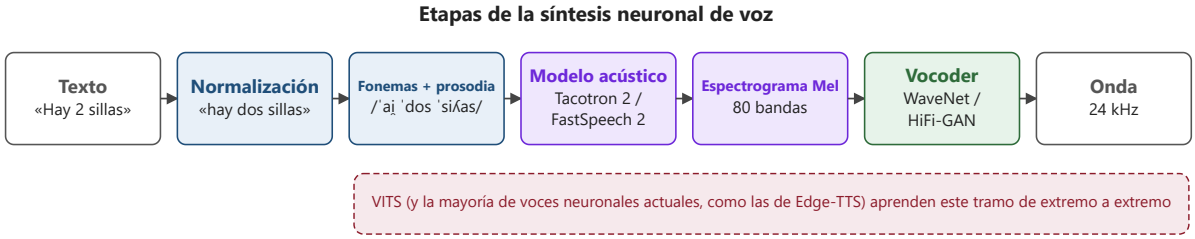


Figura 2.17: Etapas de la síntesis neuronal de voz. Los sistemas actuales suelen aprender de extremo a extremo el tramo que va de los fonemas a la forma de onda.

La propuesta planteaba usar **Coqui TTS** [19], una biblioteca abierta con modelos VITS que se ejecutan localmente. Sin embargo, la empresa que la mantenía cerró a comienzos de 2024 y su última versión no es compatible con Python 3.13, el intérprete del entorno de desarrollo. Se optó por **Edge-TTS** [85], un cliente de Python del servicio de lectura en voz alta del navegador Microsoft Edge, que da acceso gratuito y sin clave a las voces neuronales de Microsoft en decenas de idiomas y devuelve MP3 de 24 kHz a 48 kbit/s. La

contrapartida, que se discute en el capítulo 5, es que la síntesis deja de ser local: requiere conexión a Internet, envía el texto de las respuestas a un servicio externo y depende de un servicio no documentado oficialmente para este uso.

2.9. Fundamentos de la arquitectura web del sistema

2.9.1. Cliente, servidor y formato de los datos

El sistema es una aplicación web cliente-servidor. El navegador y los servicios se comunican mediante **HTTP** [29] siguiendo el estilo REST [30]: cada funcionalidad es un recurso con una URL (`/api/analyze`, `/api/stt/transcribe`...) al que se envían peticiones `POST`. Como las peticiones incluyen ficheros —imágenes y audio— junto con campos de texto, se codifican como `multipart/form-data` [59], el mismo formato que usan los formularios HTML con ficheros adjuntos. Las respuestas son documentos JSON o, en el caso de la síntesis de voz, un fichero MP3.

Los fotogramas de la webcam se envían como texto en **base64** [42] dentro de una *data URL* [58] (`...`), que es lo que produce directamente el método `toDataURL` del *canvas*. La codificación base64 representa cada 3 bytes con 4 caracteres ASCII, por lo que el tamaño transmitido crece un 33 %:

$$n_{b64} = 4 \left\lceil \frac{n}{3} \right\rceil \tag{2.15}$$

Como el frontend (servido en el puerto 5173) y los servicios (puertos 8000 y 8002) tienen orígenes distintos, el navegador aplica la política del mismo origen y solo permite las peticiones si el servidor lo autoriza explícitamente mediante cabeceras **CORS** [120]. Por último, la propuesta sugería WebSockets [28], un protocolo que mantiene abierta una conexión bidireccional; la alternativa más sencilla, adoptada en este trabajo, es el *polling*: el cliente repite peticiones HTTP a intervalos regulares.

2.9.2. FastAPI y la programación asíncrona

Los dos servicios de Python están escritos con **FastAPI** [83], un marco web que se apoya en Starlette para el enrutado y en Pydantic para la validación de datos, y que genera automáticamente la documentación OpenAPI de todos los *endpoints* (accesible en `/docs`). FastAPI sigue la especificación **ASGI** [22] y se ejecuta sobre el servidor **Uvicorn**, que atiende las peticiones en un bucle de eventos de `asyncio` (figura 2.18). Una función declarada con `async def` puede suspenderse mientras espera una operación de entrada/salida —una respuesta de la API de OpenAI, la lectura de un fichero— y, entretanto, el bucle atiende otras peticiones.

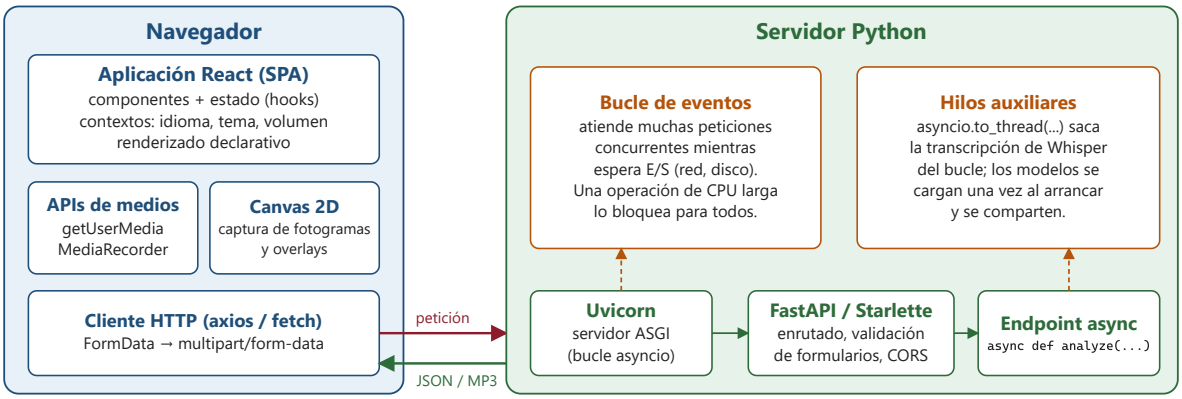


Figura 2.18: Pila web del sistema: la aplicación React se comunica por HTTP con servicios FastAPI que se ejecutan sobre el servidor ASGI Uvicorn. Las operaciones de CPU largas deben sacarse del bucle de eventos para no bloquearlo.

Este modelo tiene una trampa que conviene conocer: el bucle de eventos es un único hilo, y una operación de cálculo intenso ejecutada directamente dentro de una función `async` —una inferencia de red neuronal, por ejemplo— lo bloquea para todas las peticiones hasta que termina. La solución es ejecutar esas operaciones en un hilo auxiliar con `asyncio.to_thread`, como hace el proyecto con la transcripción de Whisper. Otra buena práctica, también aplicada, es cargar los modelos una sola vez al arrancar el servicio y compartirlos entre peticiones, en lugar de cargarlos en cada llamada.

2.9.3. React y las API multimedia del navegador

La interfaz es una **aplicación de página única** (SPA) construida con **React** [61] y TypeScript [10], empaquetada con Vite [126] y maquetada con Tailwind CSS [108]. En React la interfaz se describe de forma declarativa como una función del estado: cada componente devuelve lo que debe mostrarse para su estado y sus propiedades, y la biblioteca calcula y aplica los cambios mínimos en el DOM. El estado se gestiona con *hooks*: `useState` para valores que provocan un nuevo renderizado, `useEffect` para efectos secundarios (temporizadores, acceso a la cámara), `useRef` para valores mutables que *no* deben provocarlo (como el elemento de vídeo o el último estado leído por un temporizador), `useCallback` para memorizar funciones, y la **API de contexto** para compartir datos globales —idioma, tema, volumen— sin pasarlos componente a componente. El capítulo 3 muestra cómo algunas sutilezas de este modelo, como las *clausuras obsoletas* en temporizadores, causaron errores reales.

El navegador ofrece además las API que hacen posible la captura multimodal: `navigator.mediaDevices.getUserMedia` [114] da acceso a la cámara y al micrófono tras el permiso del usuario —y solo en contextos seguros, es decir, HTTPS o `localhost`—; `MediaRecorder` [115] graba el audio comprimido (en Chrome, Opus dentro de un

contenedor WebM); y el elemento `<canvas>` con su contexto 2D [121] permite copiar el fotograma actual del vídeo (`drawImage`), exportarlo como JPEG (`toDataURL`) y dibujar cajas, textos y flechas sobre la imagen.

2.9.4. Contenedores y microservicios

Una **arquitectura de microservicios** divide la aplicación en servicios pequeños, desplegables de forma independiente y comunicados por la red [66]. Frente a un monolito, facilita escalar y actualizar cada parte por separado y aislar fallos, a costa de más complejidad operativa. **Docker** [60] empaqueta cada servicio con su sistema operativo base, sus bibliotecas y sus modelos en una *imagen* inmutable, construida por capas a partir de un `Dockerfile`; al ejecutarla se obtiene un *contenedor* aislado. **Docker Compose** describe en un único fichero YAML todos los contenedores de la aplicación, sus puertos, variables de entorno y dependencias, y los arranca juntos en una red privada en la que cada servicio es accesible por su nombre. Las imágenes pueden publicarse en un registro como Docker Hub para que cualquiera pueda desplegar la aplicación sin instalar Python ni Node.js, que es exactamente lo que se hizo al cerrar la fase 2 (apartado 3.10).

Capítulo 3

Implementación técnica

Este capítulo recorre la implementación del sistema siguiendo el orden en que se construyó: primero la metodología de trabajo y la arquitectura final, y después cada una de las fases, deteniéndose en los procesos y en las funciones que hubo que diseñar específicamente, así como en los problemas que surgieron y en cómo se resolvieron. No se reproduce el código completo —que está disponible en el repositorio del proyecto [25]—, sino los fragmentos imprescindibles para entender cada decisión.

3.1. Metodología y organización del trabajo

3.1.1. Desarrollo incremental por sesiones

El trabajo se desarrolló entre finales de marzo y finales de septiembre de 2026 siguiendo la estrategia incremental descrita en el apartado 2.4: cada fase debía terminar con un sistema completo y demostrable antes de empezar la siguiente. Para mantener la trazabilidad del progreso se llevó un **diario de sesiones** (`docs/diary.md`) en el que, al final de cada sesión de desarrollo, se anotaban la duración, los logros, los problemas encontrados y los próximos pasos. La figura 3.1 resume la cronología que se desprende de ese diario, y el anexo C lo reproduce de forma resumida.

La figura muestra una fase 0 extendida en el tiempo con sesiones cortas y una concentración del esfuerzo en agosto y septiembre, cuando se abordaron las fases 1 y 2. El diario resultó útil por dos motivos: obligaba a cerrar cada sesión con un objetivo concreto para la siguiente y, a posteriori, ha permitido reconstruir con precisión los problemas encontrados, que se citan a lo largo de este capítulo.

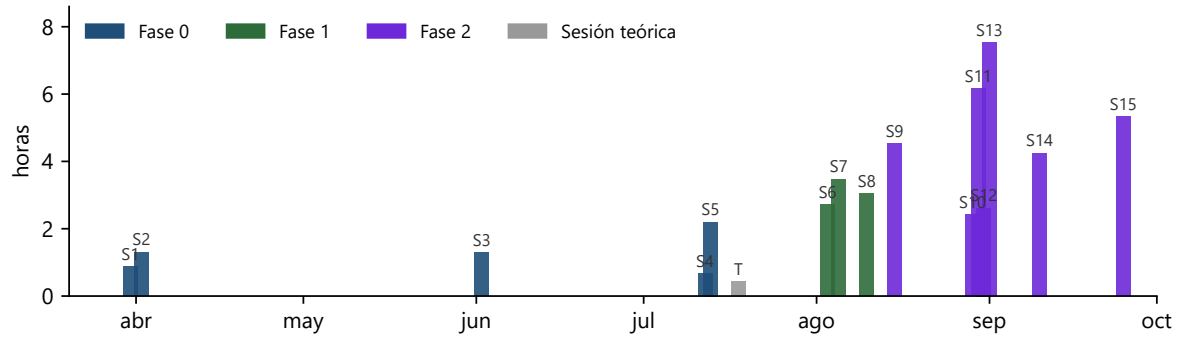


Figura 3.1: Cronología de las sesiones de desarrollo registradas en el diario del proyecto (S1–S15) y de la sesión teórica sobre smolagents (T), coloreadas según la fase a la que contribuyeron. La altura de cada barra es la duración anotada de la sesión; el diario no recoge el tiempo dedicado al estudio de la documentación ni a la redacción de esta memoria.

3.1.2. Control de versiones y buenas prácticas

El código se gestionó con Git y GitHub siguiendo una versión simplificada de *Git Flow* [24], documentada en el fichero `CONTRIBUTING.md` del repositorio (figura 3.2). La rama `main` contiene siempre una versión estable y entregable; `develop` es la rama de integración; y cada tarea se desarrolla en una rama efímera con prefijo según su tipo (`feature/`, `fix/`, `test/` o `docs/`), que se fusiona con `develop` y se borra al terminar. Cuando un conjunto de funcionalidades forma una versión estable, `develop` se fusiona con `main` y se etiqueta: así nacieron la versión **v0.2.0** (5 de agosto, webcam en tiempo real) y la **v1.2.0** (25 de septiembre, cierre de la fase 2).

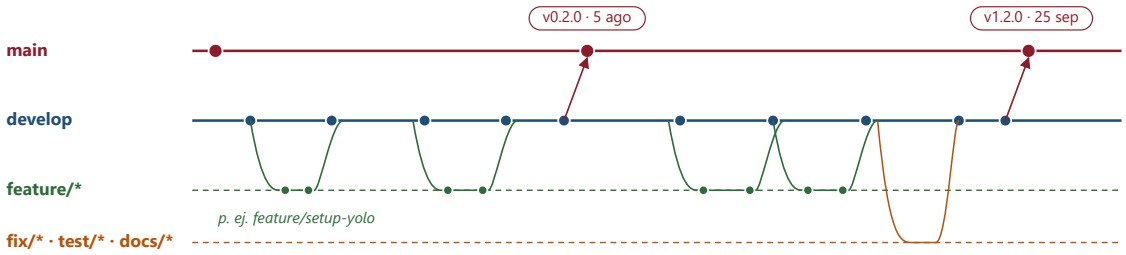


Figura 3.2: Estrategia de ramas seguida en el proyecto (Git Flow simplificado) y versiones publicadas.

Los mensajes de *commit* siguen la convención *Conventional Commits* [18] (`feat:`, `fix:`, `docs:`, `refactor:`, `style:`, `test:`), reforzada con una plantilla (`.gitmessage`) que Git muestra al escribir cada mensaje. Las dependencias de Python se aíslan en un entorno virtual y se congelan con `pip freeze`. Un detalle curioso, útil para quien trabaje en Windows: al generar `requirements.txt` con la redirección `>` de PowerShell, el fichero se guarda en UTF-16 con BOM en lugar de UTF-8; `pip` lo interpreta correctamente gracias al BOM, pero otras herramientas no, así que es preferible usar `pip freeze | Out-File -Encoding utf8`.

LECCIÓN APRENDIDA

En la sesión 3 se subió por error al repositorio el fichero de pesos `yolov8n.pt`. Los modelos y los secretos no deben versionarse: se añadió `*.pt` y `.env` al `.gitignore` (y al `.dockerignore`) y se dejó que Ultralytics descargue los pesos automáticamente en el primer arranque. El fichero `.env.example` documenta las variables necesarias sin exponer la clave real.

3.2. Arquitectura del sistema final

La figura 3.3 muestra la arquitectura desplegada al final de la fase 2. Se compone de tres servicios, cada uno en su propio contenedor Docker:

- **frontend**: la aplicación React que se ejecuta en el navegador del usuario. Captura imagen, vídeo y audio, dibuja los resultados y gestiona el estado de la conversación.
- **backend**: el *servicio de percepción*, en FastAPI. Carga una sola vez YOLOv8n y el modelo Whisper, y expone seis *endpoints* de análisis de imagen, color, voz a texto y texto a voz. Es el único que habla con GPT-4o mini para analizar imágenes.
- **agent-service**: el *servicio del agente* Mini, también en FastAPI, con un único *endpoint*. Construye la tarea, ejecuta el agente de smolagents y, cuando este necesita «ver», delega en el backend por HTTP.

La tabla 3.1 resume la interfaz de los dos servicios. Todas las peticiones se envían como `multipart/form-data` e incluyen el idioma elegido en la interfaz, que se propaga a todas las capas: el LLM responde en ese idioma, las clases de YOLO y los nombres de colores se traducen, Whisper transcribe forzando ese idioma y la síntesis usa una voz nativa. La especificación detallada de cada *endpoint* se recoge en el anexo B.

Una decisión de diseño transversal es que **los servicios no guardan estado**. No hay base de datos ni Redis: el historial de la conversación viaja en cada petición como un documento JSON que el servidor devuelve actualizado y que el navegador conserva hasta la siguiente. Esta decisión simplifica el despliegue y hace que cualquier instancia de un servicio pueda atender cualquier petición, a costa de que el cliente controle el historial (apartado 3.5). Comparada con la propuesta, la arquitectura se desvía en tres puntos que se justifican en las secciones correspondientes: el *polling* HTTP sustituye a los WebSockets; no hay capa de caché ni mensajería; y el primer servicio separado fue el del agente, no el de audio, porque sus dependencias (smolagents, EasyOCR) y su ritmo de evolución eran los más distintos de los del resto del sistema.

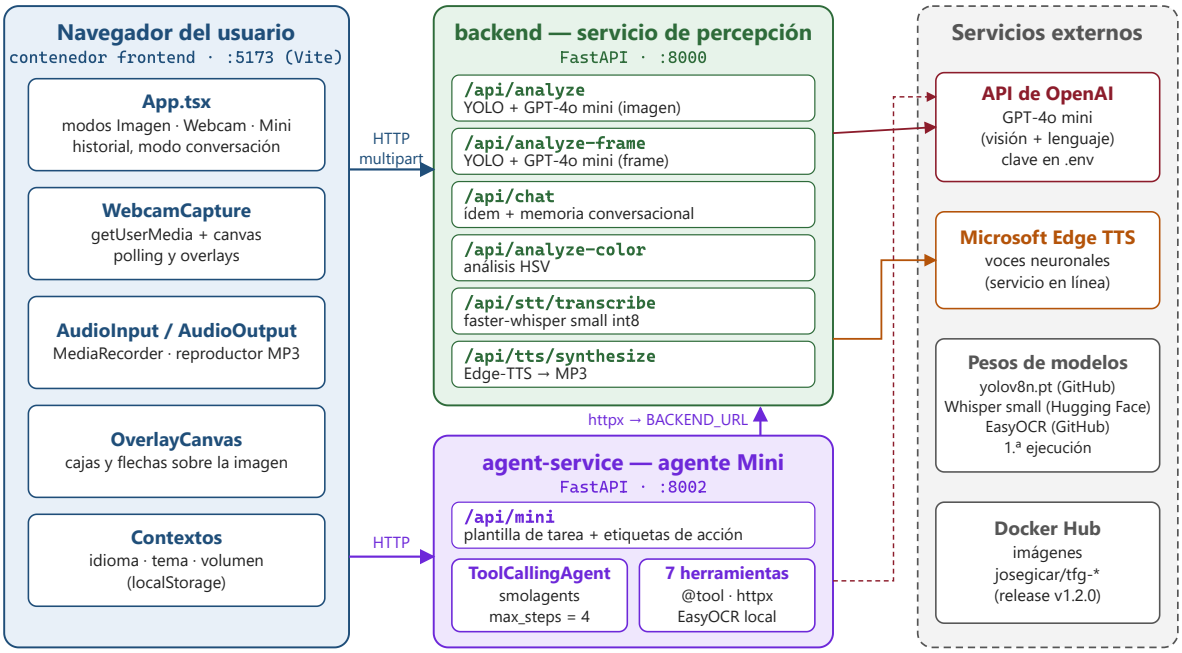


Figura 3.3: Arquitectura desplegada del sistema al cierre de la fase 2: tres contenedores (frontend, backend de percepción y servicio del agente) y los servicios externos de los que dependen.

Endpoint	Entrada	Salida	Usado por
backend (puerto 8000)			
`POST /api/analyze`	image, query, language	answer, detections	modo Imagen, herramienta del agente
`POST /api/analyze-frame`	frame_base64, query, language	answer, detections	modo Webcam, herramienta del agente
`POST /api/chat`	image o frame_base64, query, language, history	answer, detections, history	modo conversación
`POST /api/analyze-color`	frame_base64, language	colors, dominant	herramienta del agente
`POST /api/stt/transcribe`	audio, language	text, language	micrófono (todos los modos)
`POST /api/tts/synthesize`	text, language	fichero MP3	botones de altavoz
agent-service (puerto 8002)			
`POST /api/mini`	query, language, image?, frame_base64?, history?	answer, action, detections, history	modo Mini

Tabla 3.1: Interfaz de los servicios del sistema.

3.3. Fase 0: asistente visual sobre imagen estática

3.3.1. Flujo de una consulta

La fase 0 estableció el recorrido básico de una consulta, que el resto del sistema reutiliza. La figura 3.4 muestra el diagrama de secuencia de `/api/analyze`. El backend lee los bytes del fichero recibido, los convierte en una matriz de NumPy y la decodifica con `cv2.imdecode`, lo que evita escribir la imagen en disco. A continuación ejecuta YOLOv8n con un umbral de confianza de 0,5 y construye la lista de detecciones: la clase (traducida al idioma del usuario), la confianza redondeada a dos decimales y la caja en formato `[x1, y1, x2, y2]` en píxeles de la imagen original. Por último, vuelve a codificar la imagen como JPEG en base64 y llama a GPT-4o mini con un mensaje de sistema y un mensaje de usuario compuesto por dos partes: la imagen y un texto con las detecciones y la pregunta. La respuesta se limita a 300 tokens.

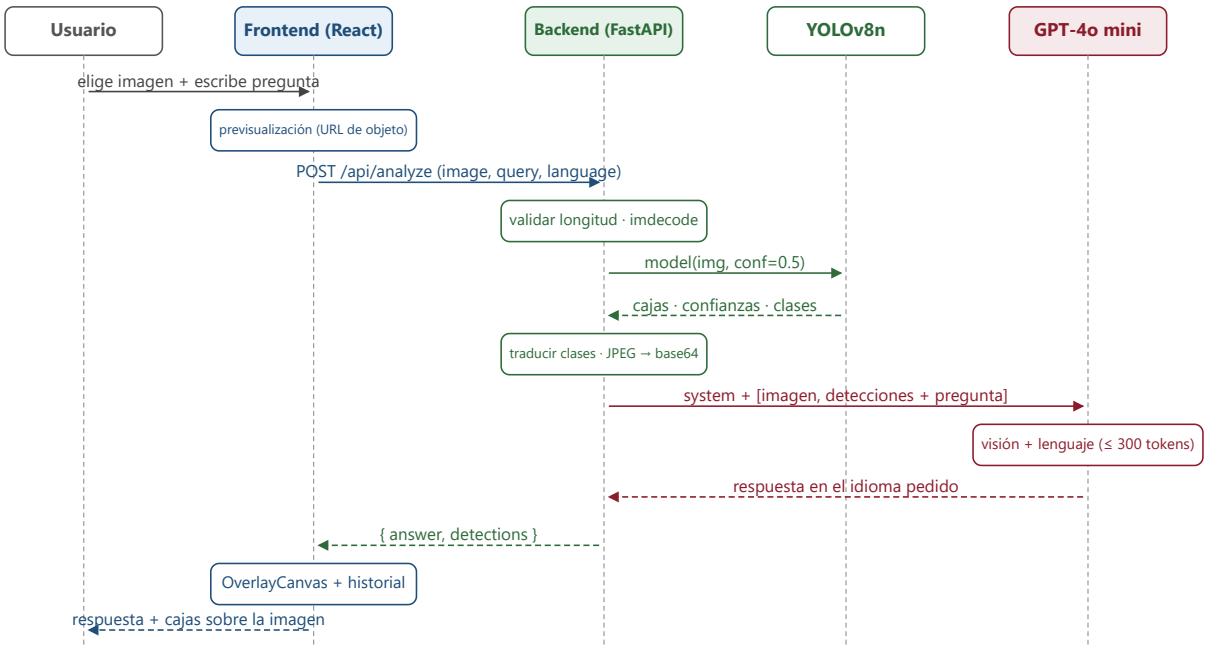


Figura 3.4: Diagrama de secuencia de una consulta en modo Imagen (`/api/analyze`).

3.3.2. Doble canal de información hacia el LLM

La primera versión del backend, la que proponía el MVP, enviaba al LLM *solo* la lista de detecciones de YOLO, sin la imagen. En la sesión 5 se detectó su principal carencia: al probar con logotipos e ilustraciones, YOLO no detectaba nada —su entrenamiento en COCO solo incluye fotografías— y el LLM, ciego, no podía responder. La solución fue enviar *también* la imagen a GPT-4o mini, de modo que el modelo recibe dos canales complementarios:

- la **imagen**, que le permite describir cualquier cosa, incluidos objetos fuera de las 80 clases de COCO, ilustraciones, texto o colores;
- las **detecciones**, que le proporcionan recuentos y posiciones explícitas y fiables, útiles para preguntas como «¿cuántas personas hay?» o «¿qué hay a la izquierda?».

Además, las detecciones se devuelven al navegador para dibujarlas, de modo que el usuario puede contrastar visualmente lo que el sistema afirma. En términos de la ecuación 2.1, la respuesta verbal a_t la produce el LLM y las anotaciones visuales v_t proceden del detector, lo que garantiza que lo que se dibuja existe realmente en la imagen.

3.3.3. Internacionalización de las clases de COCO

YOLOv8 devuelve los nombres de las clases en inglés (`person`, `dining table`...). Para que tanto el LLM como la interfaz trabajen en el idioma del usuario, se creó el módulo `class_translations.py`, con un diccionario que traduce las 80 clases de COCO a español, francés, alemán, italiano y portugués (el inglés se devuelve sin cambios). La función `translate_class` se aplica a cada detección *antes* de construir el *prompt*, de manera que el modelo lee, por ejemplo, «televisor» y «planta en maceta» y no mezcla idiomas en su respuesta. Si una clase no tuviera traducción, se devolvería el nombre original, lo que hace la función robusta ante cambios de modelo.

3.3.4. Dibujo de las detecciones

En el navegador, el componente `OverlayCanvas` dibuja la imagen y sus detecciones en un `<canvas>`. El detalle clave es que el tamaño *intrínseco* del canvas se fija igual al de la imagen original (`canvas.width = img.width`), de modo que las coordenadas de las cajas, expresadas en píxeles de la imagen original, pueden usarse directamente; el CSS (`max-width: 100%`) se encarga después de escalar el conjunto para que quepa en pantalla. Como el grosor de las líneas y el tamaño de la letra se expresan también en píxeles de la imagen, en una fotografía de 4000 píxeles de ancho unas etiquetas de 16 píxeles resultarían ilegibles una vez reducida; por eso se escalan con el ancho de la imagen:

$$s = \frac{w_{img}}{1000}, \qquad \text{letra} = \text{máx}(16,\ 22\,s), \qquad \text{línea} = \text{máx}(2,\ 3\,s) \tag{3.1}$$

La etiqueta de cada caja se dibuja sobre un rectángulo de fondo cuyo ancho se calcula con `measureText`. La figura 3.5 muestra la interfaz de la fase 0 con una consulta real realizada durante el desarrollo. Obsérvese que las clases aparecen todavía en inglés («chair», «tv»), porque la captura es anterior a la traducción de clases, y que YOLO solo detecta la silla y el televisor, mientras que la respuesta de GPT-4o mini menciona además la lámpara, las

cortinas, la escultura y el suelo de madera: una ilustración directa de la complementariedad de ambos canales.

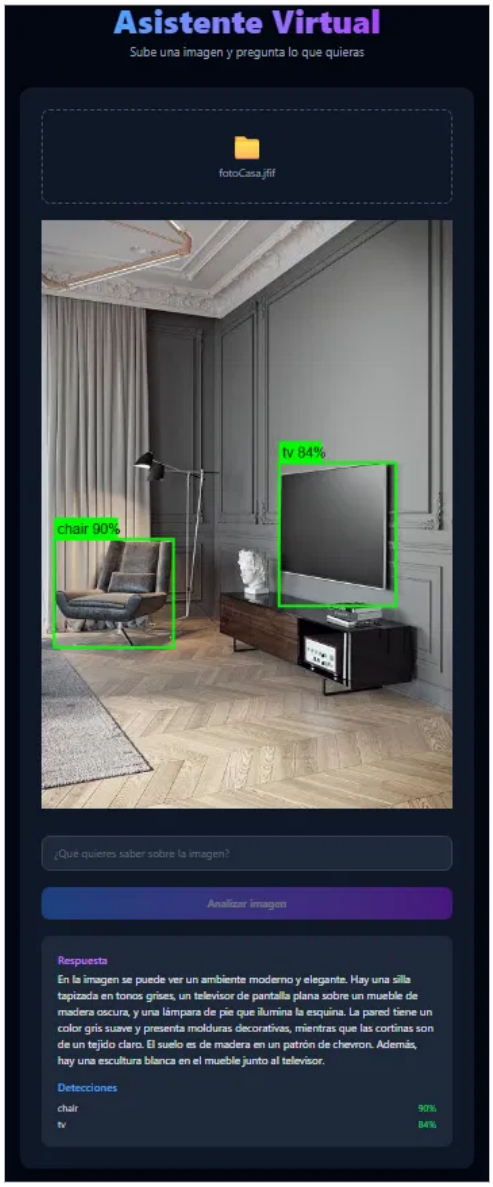


Figura 3.5: Interfaz de la fase 0 (captura del desarrollo). YOLO detecta la silla (90 %) y el televisor (84 %); el LLM, que ve también la imagen, describe además elementos que no pertenecen a las clases de COCO.

3.3.5. Problemas encontrados en la fase 0

- **Acceso a la API.** Usar GPT-4o mini exige crear una clave y cargar un saldo mínimo (5 euros). La clave se guarda en un fichero `.env` que se lee con `python-dotenv` y nunca se versiona.
- **Tailwind CSS v4.** La versión 4 cambió la configuración: basta una línea (`@import "tailwindcss";`) en el CSS y el complemento de Vite, sin fichero de configuración.

Además, los estilos en línea del prototipo eran incompatibles con el nuevo diseño y hubo que sustituirlos por completo.

- **Modelos ciegos a lo que no han visto.** El fallo de YOLO con ilustraciones, comentado arriba, motivó el doble canal hacia el LLM.

3.4. Fase 1: vídeo en tiempo real

3.4.1. Captura de fotogramas

La fase 1 añadió la webcam. El componente `WebcamCapture` solicita el flujo de vídeo con `getUserMedia({ video: { width: 640, height: 480, facingMode: "user" } })` y lo asocia a un elemento `<video>`. Para capturar un fotograma, se dibuja el vídeo en un canvas oculto con `drawImage` y se exporta con `toDataURL('image/jpeg', 0.7)`. El resultado es una *data URL* que se envía como campo de texto del formulario al nuevo *endpoint* `/api/analyze-frame` (figura 3.6). La calidad JPEG de 0,7 es un compromiso entre tamaño y detalle: medido en Chrome sobre la propia aplicación, un fotograma de 640×480 genera una *data URL* de unos 51 400 caracteres, es decir, unos 38 KB de JPEG convertidos en unos 50 KB de base64, un 33 % más, como predice la ecuación 2.15. La captura y la codificación cuestan unos 5 ms en el navegador.

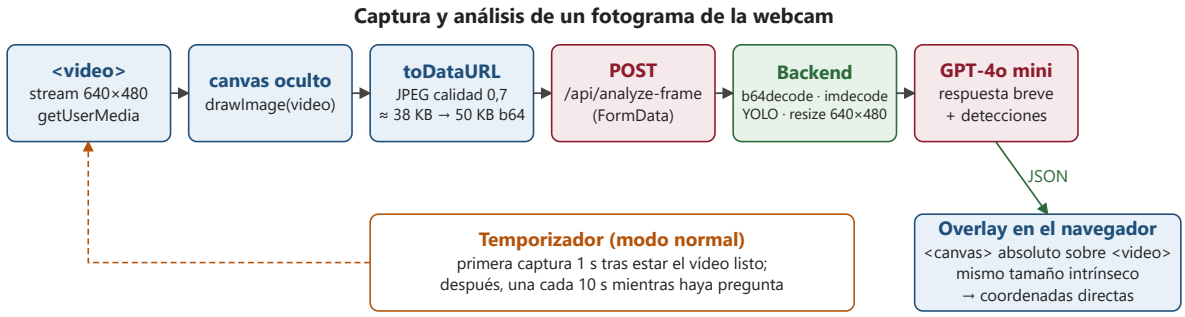
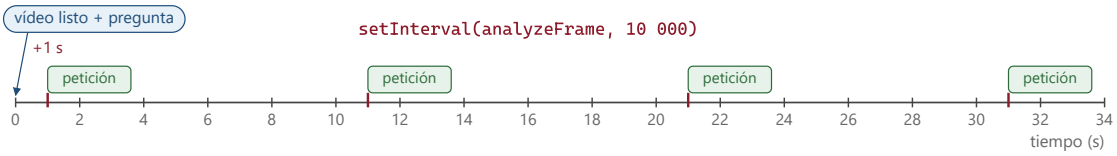


Figura 3.6: Canalización de captura y análisis de un fotograma de la webcam, desde el elemento de vídeo hasta el dibujo del resultado.

En el servidor, el nuevo *endpoint* separa la cabecera de la *data URL* (``), limpia saltos de línea y espacios que algunos clientes introducen, decodifica el base64 y la imagen, y comprueba que el resultado no esté vacío antes de continuar. Se reutiliza el mismo análisis que en la fase 0, con dos diferencias: el fotograma se redimensiona a 640×480 antes de enviarlo a GPT-4o mini, para acotar el número de teselas y, por tanto, de tokens, y el mensaje de sistema pide respuestas concisas, apropiadas para un análisis continuo.

3.4.2. Muestreo periódico y límites de la API

Con la cámara activa y una pregunta escrita, el componente analiza un fotograma de forma periódica. La lógica, resumida en la figura 3.7, se implementa con un efecto de React que se activa solo cuando se cumplen cinco condiciones —cámara activa, pregunta no vacía, vídeo listo, modo conversación desactivado y modo Webcam visible—: espera un segundo, analiza el primer fotograma y programa los siguientes con `setInterval`. Si todavía hay una petición en curso cuando vence el intervalo, la nueva captura se descarta para no acumular peticiones.



Tokens por minuto enviados a la API ≈ (60 / Δt) · (14 167 tokens de imagen + ≈ 200 de texto)

Δt = 3 s	20 fotogramas/min	**≈ 287 000 TPM**	supera un límite del orden de 200 000 TPM → errores 429
Δt = 10 s (final)	6 fotogramas/min	**≈ 86 000 TPM**	margen suficiente; coste ≈ 0,8 $ por hora de análisis continuo

Figura 3.7: Arriba: temporización del muestreo periódico de la webcam. Abajo: estimación de los tokens por minuto enviados a la API según el intervalo, usando el coste por imagen de la ecuación 2.12.

La elección del intervalo fue uno de los problemas más instructivos del proyecto. La propuesta sugería analizar un fotograma cada 2–3 segundos, y durante las sesiones 6 y 7 se probaron intervalos entre 3 y 10 segundos. Con los intervalos cortos aparecían errores `429 Too Many Requests` de la API de OpenAI (`RateLimitError`). A primera vista resultaba sorprendente, porque 20 peticiones por minuto están muy lejos del límite de peticiones de la cuenta. La explicación está en los *tokens* y no en las peticiones: como se vio en el apartado 2.6.4, cada fotograma de 640×480 enviado a GPT-4o mini en modo de detalle alto cuesta 14 167 tokens de entrada. El consumo por minuto es:

$$\mathrm{TPM}(\Delta t) \approx \frac{60}{\Delta t} \cdot (T_{img} + T_{texto}) \tag{3.2}$$

Con $\Delta t = 3$ s se envían unos 287 000 tokens por minuto, por encima de límites del orden de 200 000 tokens por minuto como los que OpenAI asigna a las cuentas de los primeros niveles de uso [68]; con $\Delta t = 10$ s, unos 86 000, con margen suficiente. El valor final quedó fijado en 10 segundos (`POLLING_INTERVAL = 10000`), y el backend captura `RateLimitError` y devuelve un mensaje explicativo en el idioma del usuario en lugar de fallar. La misma ecuación permite estimar el coste: a 10 segundos por fotograma, el análisis continuo cuesta del orden de 0,8 dólares por hora, un dato relevante si se piensa en un uso prolongado. Como se discute en el capítulo 5, fijar explícitamente el modo de detalle bajo

o analizar solo cuando la escena cambie reduciría este coste en un orden de magnitud.

> **LECCIÓN APRENDIDA**
>
> Con modelos multimodales, el recurso escaso no suele ser el número de peticiones sino el número de *tokens*, y una imagen puede costar tanto como decenas de páginas de texto. Antes de fijar la frecuencia de análisis de un flujo de vídeo conviene calcular los tokens por imagen y compararlos con los límites de la cuenta.

3.4.3. Temporizadores en React: el problema de las clausuras obsoletas

Implementar el muestreo periódico en React obliga a resolver un problema sutil. La función que analiza el fotograma se crea en un renderizado concreto y «captura» los valores de las variables de ese momento (la pregunta, el idioma, el historial...). Si se pasa a `setInterval`, el temporizador seguirá llamando a esa versión de la función aunque el usuario cambie después la pregunta o el idioma: es el problema de las **clausuras obsoletas** (*stale closures*). Recrear el intervalo cada vez que cambia cualquier dato tampoco es una buena solución, porque reiniciaría la cuenta en cada pulsación de tecla. La solución adoptada es el patrón de **referencia espejo** (listado 3.1): un `useRef` guarda en todo momento los valores actuales de las propiedades, y la función del temporizador los lee de la referencia en cada ejecución.

Listado 3.1: Patrón de referencia espejo para evitar clausuras obsoletas en el temporizador (WebcamCapture.tsx, simplificado).

```
const requestRefs = useRef({ query, language, conversationMode, conversationHistory, onResult })

useEffect(() => {        // en cada renderizado se actualiza la referencia...
  requestRefs.current = { query, language, conversationMode, conversationHistory, onResult }
}, [query, language, conversationMode, conversationHistory, onResult])

const analyzeFrame = useCallback(async () => {
  const { query, language } = requestRefs.current   // ...y el temporizador lee siempre lo último
  if (!query || loading) return
  // capturar fotograma, enviar y dibujar el resultado
}, [loading, drawOverlay])
```

Otros tres detalles completan la robustez del componente. Primero, al activar la cámara el vídeo tarda unos instantes en tener datos —en la sesión 6 se comprobó que el primer fotograma se capturaba antes de que el vídeo estuviera listo—, así que se espera al evento `onCanPlay` y se comprueba que `video.readyState` $\geq$ `2` (`HAVE_CURRENT_DATA`), además del retardo inicial de un segundo. Segundo, una respuesta puede llegar después de que el usuario haya apagado la cámara; una referencia `activeRef` permite descartarla. Tercero, al desmontar el componente o detener la cámara se paran todas las pistas del `MediaStream` y

se cancelan los temporizadores, de modo que el indicador de cámara del sistema operativo se apaga realmente.

3.4.4. Dibujo sobre el vídeo

Para superponer las detecciones al vídeo en directo se coloca un segundo canvas con posición absoluta exactamente encima del elemento `<video>`. Como en la fase 0, su tamaño intrínseco se iguala al del vídeo (`videoWidth × videoHeight`, es decir, 640×480), el mismo que el del fotograma enviado, y ambos elementos se escalan con el mismo CSS; así, las coordenadas devueltas por el backend caen exactamente sobre los objetos. Las cajas permanecen dibujadas hasta que llega el siguiente resultado, lo que con un intervalo de 10 segundos supone que, si la cámara se mueve, pueden quedar desalineadas durante un tiempo: es una limitación inherente al muestreo espaciado que el seguimiento de objetos entre fotogramas (fase 3) resolvería.

3.4.5. Análisis de color en HSV

Para responder preguntas de color de forma cuantitativa se implementó el *endpoint* `/api/analyze-color`, basado en los fundamentos del apartado 2.5.4. El algoritmo (figura 3.8) reduce la imagen a 150×150 píxeles, la convierte a HSV y, para cada uno de los diez colores definidos, construye una máscara binaria con `cv2.inRange` como unión de sus rangos. El porcentaje de cada color es la fracción de píxeles de su máscara; solo se informa de los colores que superan el 5 % y el dominante es el de mayor porcentaje. Los nombres se traducen al idioma del usuario.

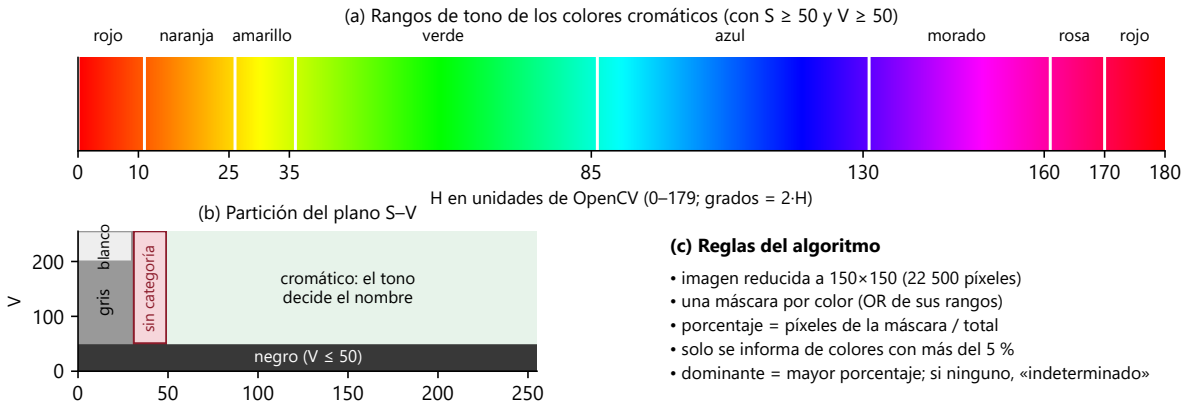


Figura 3.8: Definición de los colores en el análisis HSV del proyecto: (a) rangos de tono de los colores cromáticos; (b) partición del plano saturación–valor en negro, gris, blanco y colores cromáticos; (c) reglas del algoritmo.

Varias decisiones merecen comentario. La reducción a 150×150 (22 500 píxeles) es más que suficiente para estimar proporciones y hace que el análisis completo, incluida la

decodificación del fotograma, tarde unos 3 ms. El rojo necesita dos rangos porque el tono es circular: está tanto al principio (0–10) como al final (170–180) de la escala. Los acromáticos se definen por saturación y brillo: negro si $V \leq 50$, blanco si $S \leq 30$ y $V \geq 200$, gris en medio. Esta definición tiene dos consecuencias poco evidentes: los píxeles con saturación entre 31 y 49 no pertenecen a ningún color, y los que tienen exactamente $V = 50$ cuentan a la vez como negro y como su color cromático, así que los porcentajes no suman necesariamente el 100 %. La figura 3.9 muestra el proceso sobre un fotograma real: los muebles de madera aparecen como «rojo» (24,5 %), porque la paleta no incluye el marrón, una limitación que el capítulo 4 cuantifica.



Figura 3.9: Análisis de color sobre un fotograma real: (a) imagen original; (b) píxeles que activan las cuatro máscaras principales; (c) porcentajes de cada color y umbral de notificación. La madera se clasifica como «rojo» y en menor medida «naranja».

El *endpoint* de color se creó en la fase 1, pero no se conectó a ningún botón de la interfaz clásica: en los modos Imagen y Webcam las preguntas de color las responde GPT-4o mini mirando la imagen. Su papel quedó definido en la fase 2, cuando se convirtió en una herramienta del agente (apartado 3.7).

3.5. Memoria conversacional

Entre las fases 1 y 2 se incorporó la memoria conversacional, que la propuesta situaba en la fase 3 pero que resultó imprescindible para preguntas de seguimiento como «¿y ahora?» o «¿qué ha cambiado?». Se implementó en el *endpoint* `/api/chat`, que se usa en los modos Imagen y Webcam cuando el interruptor «Modo conversación» está activado.

3.5.1. Un historial que viaja con cada petición

Como se ha dicho, el servidor no guarda estado. El navegador envía, junto con la pregunta y la imagen, el historial serializado en JSON; el backend construye con él la lista de

mensajes para GPT-4o mini, obtiene la respuesta, actualiza el historial y lo devuelve; y el navegador lo guarda para la siguiente petición (figura 3.10). Activar o desactivar el modo, o pulsar «Limpiar», reinicia el historial.

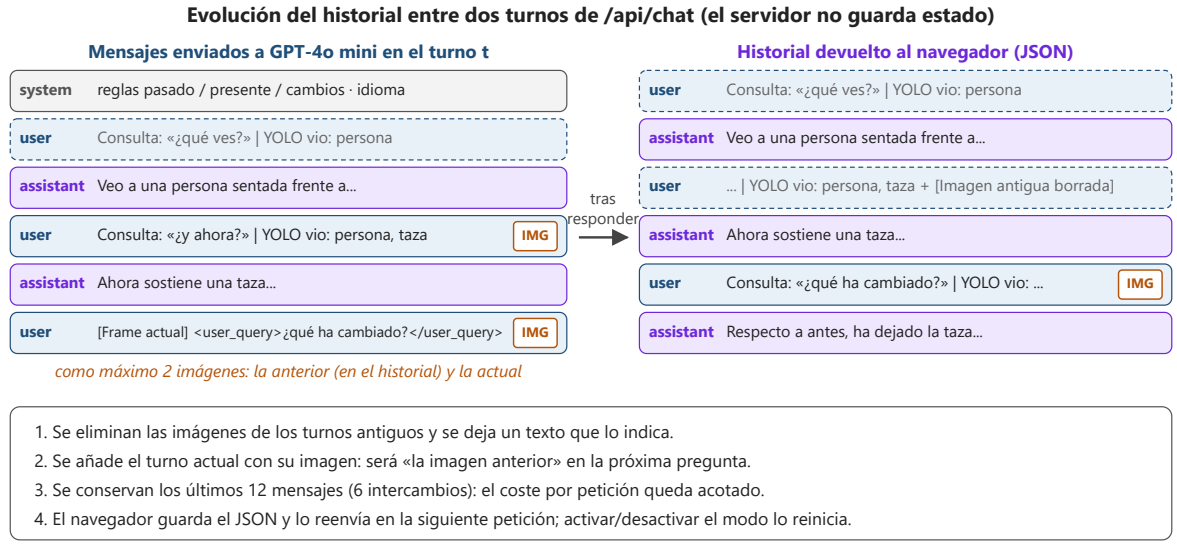


Figura 3.10: Evolución del historial entre dos turnos del modo conversación. Solo se conserva la imagen del turno más reciente y un máximo de 12 mensajes.

Este diseño tiene ventajas claras —el backend puede reiniciarse o replicarse sin perder conversaciones y no hace falta ningún almacén de sesiones— y dos inconvenientes que conviene explicitar. El primero es de seguridad: el historial es un dato del cliente y un usuario podría manipularlo, por ejemplo inyectando mensajes falsos del asistente. En una demo local no es un problema; en producción, el historial debería firmarse o guardarse en el servidor. El segundo es de rendimiento: si el historial contuviera todas las imágenes, cada petición crecería sin límite. De ahí la política de memoria visual del apartado siguiente.

3.5.2. Política de memoria visual: «una imagen como máximo»

Cada imagen cuesta más de 14 000 tokens (apartado 2.6.4), así que un historial con seis imágenes superaría los 85 000 tokens por petición. Tras varias pruebas se adoptó una política sencilla (listado 3.2): después de cada respuesta, se eliminan las imágenes de todos los turnos anteriores, sustituyéndolas por una nota textual; se añade el turno actual *con* su imagen, que se convertirá en «la imagen anterior» de la siguiente pregunta; y se conservan solo los 12 últimos mensajes (seis intercambios). En cada petición, por tanto, el modelo ve como mucho dos imágenes —la anterior y la actual—, lo que basta para responder a comparaciones, y el resto del contexto se conserva en forma de texto: las preguntas, lo que YOLO detectó en cada turno y las respuestas del propio asistente. El coste por petición

queda acotado por:

$$T_{\text{petición}} \leq T_{sistema} + 2\,T_{img} + \sum_{k=1}^{12} T_{texto,k} \tag{3.3}$$

Listado 3.2: Política de memoria visual del endpoint /api/chat (simplificado).

```
# 1. Borrar las imágenes de los turnos antiguos
for msg in conversation_history:
    if msg["role"] == "user" and isinstance(msg["content"], list):
        text_only = [p for p in msg["content"] if p.get("type") == "text"]
        text_only.append({"type": "text", "text": "[Imagen antigua borrada para liberar memoria]"
    })
        msg["content"] = text_only

# 2. Añadir el turno actual CON su imagen (será "la anterior" en la próxima pregunta)
conversation_history.append({"role": "user", "content": [image_content,
    {"type": "text", "text": f"Consulta: '{query}' | YOLO vio: {yolo_summary}"}]})
conversation_history.append({"role": "assistant", "content": answer})

# 3. Ventana deslizante: últimos 6 intercambios
conversation_history = conversation_history[-12:]
```

3.5.3. Pasado frente a presente

La primera versión de la memoria «funcionaba» técnicamente, pero no en la práctica: ante preguntas como «¿qué veías antes?», el modelo describía la imagen actual, porque es la información visual más saliente del mensaje. El problema se resolvió en el *prompt*, con un mensaje de sistema que distingue explícitamente tres tipos de pregunta y da ejemplos de las expresiones temporales que los delatan: si se pregunta por el **pasado** («¿qué veías?», «¿qué había hace un rato?»), debe usarse el historial y no solo la imagen actual; si se pregunta por el **presente** («¿qué ves ahora?»), la imagen y las detecciones actuales; y si se pide una **comparación** («¿qué ha cambiado?»), ambas cosas. El mensaje de sistema aclara además qué imagen es cuál: la del historial es la inmediatamente anterior y la del mensaje actual es la nueva. Esta es una buena muestra de cómo, con LLM, muchos «errores de programa» se corrigen con instrucciones y no con código.

3.5.4. El modo conversación en la webcam

Con el modo conversación activo, el muestreo periódico de la webcam se desactiva: no tendría sentido añadir al historial un turno cada diez segundos con la misma pregunta. En su lugar aparece un botón «Enviar» y, al pulsarlo (o al pulsar `Enter`), se incrementa un contador de estado, `webcamTrigger`, que el componente de la webcam observa en un efecto para capturar y analizar un único fotograma. Usar un contador en lugar de un

booleano garantiza que cada pulsación produzca un cambio de valor y, por tanto, un disparo. Cada turno guarda la miniatura de su fotograma en el historial visible de la interfaz. La figura 3.11 muestra este modo en una captura del desarrollo.

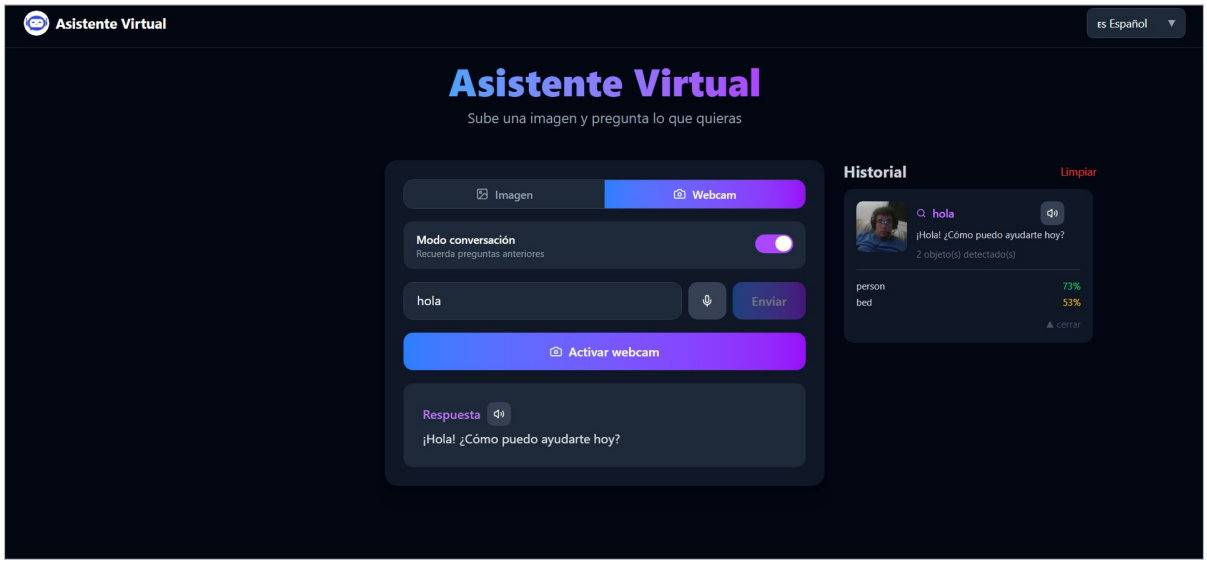


Figura 3.11: Modo Webcam con el modo conversación activado (captura del desarrollo, fase 2). El historial de la derecha guarda la miniatura del fotograma de cada turno y las detecciones de YOLO («person» 73 %, «bed» 53 %).

3.6. Fase 2: interacción por voz

3.6.1. Voz a texto

El reconocimiento de voz sigue el flujo de la parte superior de la figura 3.12. El componente `AudioInput` muestra un botón de micrófono: la primera pulsación solicita el micrófono con `getUserMedia({ audio: true })` e inicia un `MediaRecorder` que acumula fragmentos de audio comprimido; la segunda detiene la grabación, compone un `Blob` de tipo `audio/webm`, libera el micrófono y lo envía a `/api/stt/transcribe` junto con el idioma de la interfaz.

En el servidor, el audio se guarda en un fichero temporal con su extensión original —faster-whisper lo decodifica con FFmpeg a través de PyAV y necesita un fichero o un flujo—, se transcribe y el fichero se borra en un bloque `finally`, pase lo que pase. El modelo se carga una sola vez al arrancar el backend con `WhisperModel("small", device="cpu", compute_type="int8")`. La transcripción usa tres opciones deliberadas: se fuerza el idioma de la interfaz (`language=language`), lo que evita errores de identificación del idioma en frases cortas; se activa el filtro VAD (`vad_filter=True`), que descarta los silencios del principio y del final de la grabación y evita alucinaciones; y la inferencia se ejecuta en un hilo auxiliar con `asyncio.to_thread`, porque de lo contrario los segundos de cálculo de Whisper bloquearían el bucle de eventos y, con él, cualquier otra petición al backend

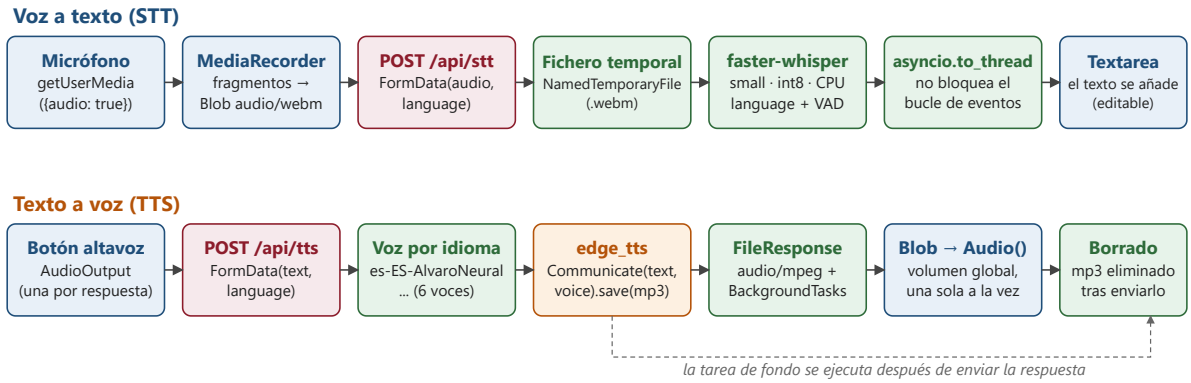


Figura 3.12: Flujos de reconocimiento (arriba) y síntesis de voz (abajo), desde el navegador hasta los modelos y de vuelta.

(listado 3.3).

Listado 3.3: Transcripción sin bloquear el bucle de eventos (backend/main.py, simplificado).

```
whisper_model = WhisperModel("small", device="cpu", compute_type="int8")   # al arrancar

def run_transcription():
    segments, info = whisper_model.transcribe(tmp_path, language=language, vad_filter=True)
    return " ".join(s.text for s in segments), info.language

try:
    text, detected = await asyncio.to_thread(run_transcription)   # hilo auxiliar
    return {"text": text.strip(), "language": detected}
finally:
    os.unlink(tmp_path)                                            # borrar siempre el temporal
```

La primera versión usaba el modelo *base*, pero su precisión resultó insuficiente en las pruebas con micrófono, y en la sesión 11 se cambió a *small* con el filtro VAD. El capítulo 4 cuantifica esta decisión. Una decisión de experiencia de usuario importante es que la transcripción *no se envía automáticamente*: se añade al cuadro de texto, donde el usuario puede revisarla y corregirla antes de enviarla con `Enter`. Así, un error de reconocimiento no desencadena una consulta equivocada. Para agilizar el uso, el atajo global `Ctrl` + `M` activa o detiene el micrófono (salvo cuando el foco está en un campo de texto, para no interferir con la escritura). Los problemas de esta parte fueron más de integración que de algoritmo: el contenedor Docker fallaba porque `faster-whisper` no se había añadido a `requirements.txt`, y el navegador bloqueaba el micrófono hasta conceder el permiso en la configuración del sitio.

3.6.2. Texto a voz

Cada respuesta del asistente, tanto en el panel principal como en el historial, lleva un botón de altavoz (componente `AudioOutput`). Al pulsarlo, el navegador envía el texto a `/`

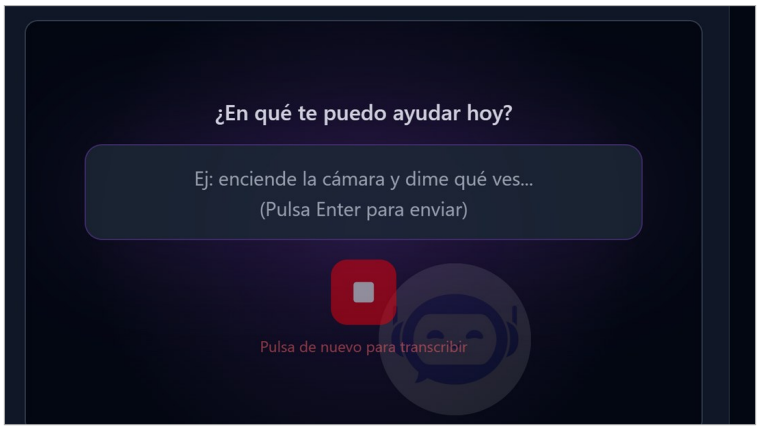


Figura 3.13: Grabación de voz en el modo Mini: el botón se vuelve rojo y pulsante, y la leyenda indica que una nueva pulsación detendrá la grabación y la transcribirá.

`api/tts/synthesize`, que elige una voz neuronal según el idioma (tabla 3.2), genera el MP3 con Edge-TTS en un fichero temporal y lo devuelve con `FileResponse`. Como el fichero no puede borrarse antes de haberse enviado, el borrado se programa como tarea de fondo (`BackgroundTasks`), que FastAPI ejecuta después de terminar la respuesta.

Idioma	**Voz**
Español	`es-ES-AlvaroNeural`
Inglés	`en-US-AriaNeural`
Francés	`fr-FR-DeniseNeural`
Alemán	`de-DE-KatjaNeural`
Italiano	`it-IT-ElsaNeural`
Portugués	`pt-BR-FranciscaNeural`

Tabla 3.2: Voces neuronales de Edge-TTS asignadas a cada idioma de la interfaz.

En el navegador, el MP3 se convierte en una URL de objeto y se reproduce con un elemento `Audio`. Dos requisitos de la interfaz obligaron a diseñar este componente con cuidado. El primero es que solo suene un audio a la vez: si se pulsa el altavoz de otra respuesta, la anterior debe detenerse. Como cada botón es una instancia independiente del componente, la coordinación se hace con dos variables a nivel de módulo (`currentAudio` y `currentStop`) compartidas por todas las instancias, que guardan el audio en reproducción y la función que actualiza el estado del botón que lo lanzó. El segundo es el control de volumen global de la barra de navegación: el volumen vive en un contexto de React y un efecto lo aplica al audio en curso en tiempo real. Durante el desarrollo apareció un error ilustrativo: se intentó guardar el elemento de audio en un estado de React y reiniciarlo con `currentTime = 0`, pero modificar un objeto guardado en el estado no tiene efecto fiable; el elemento debe guardarse en un `useRef`, que es el mecanismo de React para objetos mutables que no forman parte del renderizado. Al terminar la reproducción, la URL de objeto se libera con `revokeObjectURL` para no acumular memoria.

3.7. Fase 2: el agente Mini

Con visión, color, memoria y voz funcionando, faltaba la pieza que diese sentido al conjunto: un asistente al que se pudiera pedir cualquier cosa sin tener que elegir antes el modo adecuado. Siguiendo la recomendación del tutor, tras una sesión de estudio de smolagents, en las sesiones 12 y 13 se construyó el **agente Mini**.

3.7.1. Un servicio independiente

El agente se implementó como un servicio aparte, `agent-service`, por tres razones: sus dependencias (smolagents, EasyOCR y sus modelos) son pesadas y ajenas al resto del backend; separar la *orquestación* (qué hacer) de la *percepción* (cómo ver u oír) permite evolucionarlas por separado; y un fallo del agente no afecta a los modos clásicos, que siguen funcionando contra el backend. La figura 3.14 muestra la organización interna del servicio.

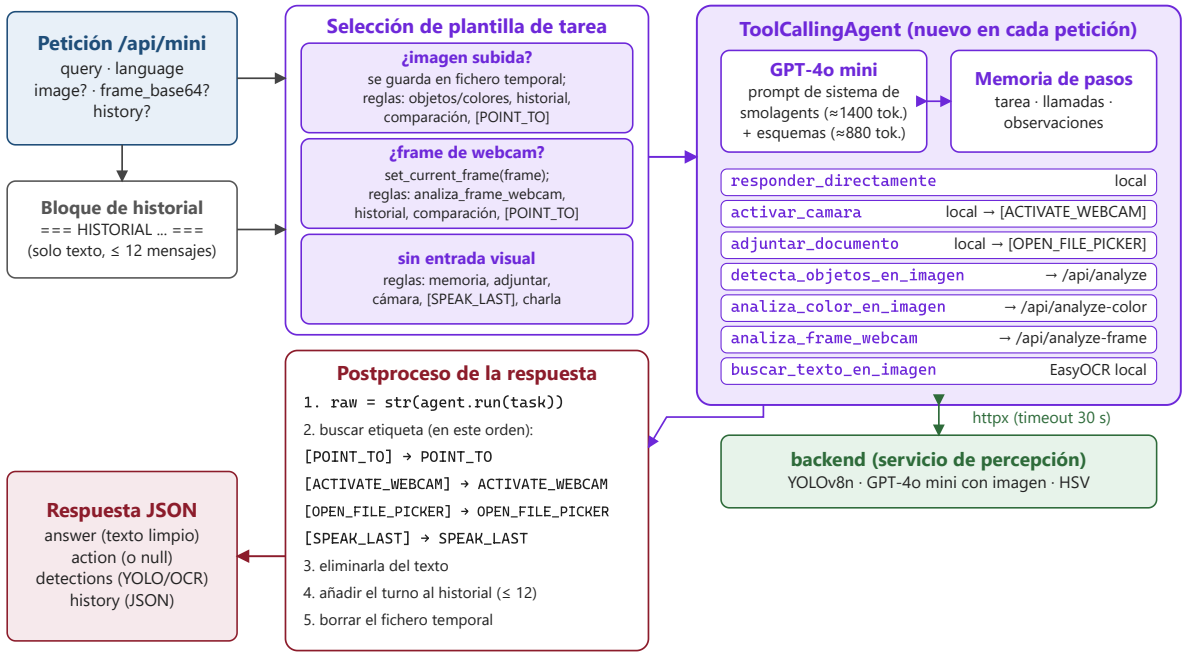


Figura 3.14: Organización interna del servicio del agente: selección de la plantilla de tarea, bucle del agente con sus siete herramientas y postproceso de la respuesta para extraer la acción.

3.7.2. Configuración del agente

El agente es un `ToolCallingAgent` de smolagents con el modelo `gpt-4o-mini` (a través de `OpenAIServerModel`), siete herramientas y un límite de cuatro pasos (`max_steps=4`). Se crea *un agente nuevo en cada petición*: la memoria interna de smolagents acumula los pasos de una ejecución, y reutilizar el mismo objeto entre peticiones mezclaría las

conversaciones de distintos usuarios. La memoria entre turnos se gestiona, igual que en `/api/chat`, con un historial que viaja en la petición, aunque aquí solo contiene texto.

Resulta instructivo examinar lo que smolagents envía realmente al modelo. Su *prompt* de sistema para el `ToolCallingAgent`, en inglés, explica que el modelo debe resolver la tarea mediante llamadas a herramientas, que cada acción produce una observación, incluye tres ejemplos completos con herramientas ficticias, enumera las herramientas disponibles con su descripción y sus parámetros, y termina con reglas como «usa siempre una llamada a herramienta» o «no repitas una llamada con los mismos parámetros». Tokenizado con el tokenizador de GPT-4o, ocupa unos 1 400 tokens, a los que se suman unos 880 de los esquemas JSON de las ocho herramientas (las siete del proyecto más `final_answer`). Es un coste fijo que se paga en cada paso del agente, un dato que el capítulo 4 utiliza para estimar el coste de cada tipo de consulta.

3.7.3. Catálogo de herramientas

La tabla 3.3 resume las siete herramientas. Todas se definen como funciones de Python decoradas con `@tool`, de modo que smolagents deduce su esquema de la firma y su descripción de la *docstring*.

Herramienta	**Cuándo debe usarse (según su descripción)**	**Implementación**
`responder_directamente`	Saludos, charla y preguntas que no requieren ver nada.	Devuelve el mensaje que recibe (respuesta de texto).
`activar_camara`	El usuario pide encender o usar la cámara.	Devuelve una instrucción para que el agente añada `[ACTIVATE_WEBCAM]`.
`adjuntar_documento`	El usuario quiere subir o analizar una foto o documento.	Devuelve una instrucción para que el agente añada `[OPEN_FILE_PICKER]`.
`detecta_objetos_en_imagen`	Preguntas sobre una imagen subida.	Llama a `/api/analyze` (YOLO + GPT-4o mini) y guarda las detecciones.
`analiza_color_en_imagen`	Colores dominantes de una imagen subida.	Llama a `/api/analyze-color`.
`analiza_frame_webcam`	Preguntas sobre lo que ve la cámara ahora.	Llama a `/api/analyze-frame` con el fotograma actual.
`buscar_texto_en_imagen`	Leer, buscar o localizar un texto o palabra.	EasyOCR local; guarda la caja del texto encontrado.

Tabla 3.3: Herramientas del agente Mini.

3.7.4. Diseño de las herramientas

El diseño de las herramientas concentra varias de las lecciones más interesantes del proyecto:

- **La *docstring* es la interfaz de programación para el LLM.** El modelo decide qué herramienta usar leyendo únicamente su nombre y su descripción. Por eso las

descripciones se redactaron como instrucciones de uso («Usa esta herramienta cuando el usuario pida encender, usar, abrir o mirar por la cámara/webcam») y la de la herramienta de OCR incluye incluso la palabra «EXCLUSIVAMENTE» para evitar que se use en preguntas visuales generales.

- **El decorador `@tool` no es opcional.** En la sesión 12 el agente fallaba con un error de validación de `BaseTool`: se le estaban pasando funciones de Python normales, y smolagents exige objetos herramienta. El decorador realiza esa conversión y, de paso, valida que la *docstring* documente todos los argumentos en su sección `Args`.

- **Herramientas que devuelven instrucciones.** Las herramientas de acción (`activar_camara`, `adjuntar_documento`) no tienen efecto en el servidor: no puede encender la cámara del usuario. Lo que hacen es devolver una observación con una instrucción para el propio agente —«Comando de cámara detectado. Responde amigablemente al usuario y añade exactamente el texto [ACTIVATE_WEBCAM] al final de tu respuesta»—. Es una forma sencilla de conseguir que el LLM incluya en su respuesta final la etiqueta que el servidor convertirá en acción. Las herramientas de análisis añaden una instrucción similar para la etiqueta `[POINT_TO]` cuando el usuario pide localizar algo.

- **Un LLM dentro de una herramienta.** Las herramientas visuales no analizan la imagen por sí mismas: llaman al backend, que ejecuta YOLO y consulta a GPT-4o mini *con la imagen*. El agente, que solo maneja texto, delega así la tarea de «ver» y recibe una descripción textual con las detecciones, que después reformula para el usuario. Esta composición es potente —el agente no necesita saber nada de visión— pero multiplica las llamadas al modelo por consulta, un coste que se analiza en el capítulo 4.

- **Estado compartido entre el *endpoint* y las herramientas.** Las herramientas necesitan datos que el LLM no debe manipular: el fotograma de la webcam (unos 50 KB de base64 que no tiene sentido hacer pasar por el modelo) y las detecciones que hay que devolver al navegador. Se guardan en dos variables globales del módulo de herramientas: `_current_frame_base64`, que el *endpoint* fija antes de ejecutar el agente, y `_last_detections`, que se vacía al empezar cada petición y que las herramientas rellenan. Es una solución simple y válida para un único usuario, pero no es segura ante peticiones concurrentes: si dos usuarios usan el agente a la vez, sus fotogramas y detecciones podrían cruzarse. La alternativa correcta sería un contexto por petición, por ejemplo con `contextvars` o pasando un objeto de estado a las herramientas.

3.7.5. Plantillas de tarea

La «tarea» que recibe el agente no es solo la pregunta del usuario: el *endpoint* la envuelve en una plantilla que depende del contexto (figura 3.14). Hay tres plantillas: con imagen subida (que se guarda en un fichero temporal cuya ruta se incluye en la tarea para que la herramienta pueda leerla), con fotograma de webcam y sin entrada visual. Todas empiezan, si lo hay, por un bloque con el historial delimitado por marcas explícitas («=== HISTORIAL DE MENSAJES ANTERIORES (NO es la pregunta actual) ===»), seguido de la instrucción de responder únicamente a la pregunta nueva. A continuación aparecen reglas específicas de cada contexto, redactadas en mayúsculas en sus puntos críticos. Estas reglas, ajustadas durante las pruebas de las sesiones 12 y 13, previenen comportamientos indeseados concretos:

- Que el agente confunda la pregunta actual con la última del historial, o conteste que no tiene registro cuando sí lo tiene: de ahí los delimitadores y la regla que define exactamente cuál es el último mensaje del usuario.
- Que entre en bucles llamando varias veces a la misma herramienta, o que mezcle herramientas: se le ordena llamar a `final_answer` inmediatamente después de la primera herramienta y no repetir herramientas.
- Que, ante «repite tu última respuesta en voz alta», se disculpe diciendo que no puede hablar, la respuesta habitual de un LLM que solo produce texto. La plantilla le informa de que *sí* tiene esa capacidad, porque el navegador activará el altavoz, y le ordena responder únicamente con la etiqueta `[SPEAK_LAST]`.
- Para comparaciones («¿qué ha cambiado?»), se le indica el orden: primero la herramienta de análisis para obtener la descripción actual y después `responder_directamente` combinándola con el historial.

3.7.6. Protocolo de acciones

La respuesta final del agente es texto. Para que pueda *actuar* sobre la interfaz se diseñó un protocolo mínimo basado en etiquetas que el agente añade al final de su respuesta. Tras ejecutar el agente, el servidor busca las etiquetas en un orden de prioridad fijo, extrae la primera que encuentra como campo `action`, la elimina del texto y devuelve al navegador un JSON con la respuesta limpia, la acción, las detecciones acumuladas por las herramientas y el historial actualizado (listado 3.4). El navegador ejecuta cada acción como se describe en la tabla 3.4.

Listado 3.4: Extracción de la acción a partir de las etiquetas de la respuesta del agente (agent-service/main.py, simplificado).

```
raw_answer = str(agent.run(task))
action, clean_answer = None, raw_answer
for tag in ("POINT_TO", "ACTIVATE_WEBCAM", "OPEN_FILE_PICKER", "SPEAK_LAST"):   # prioridad
    if f"[{tag}]" in raw_answer:
        action = tag
        clean_answer = raw_answer.replace(f"[{tag}]", "").strip()
        break
return {"answer": clean_answer, "action": action,
        "detections": tools._last_detections, "history": json.dumps(new_history)}
```

Acción	Cuándo la emite el agente	Reacción del navegador
`ACTIVATE_WEBCAM`	«Enciende la cámara».	Cambia al modo Webcam y activa la cámara; no muestra la respuesta como resultado.
`OPEN_FILE_PICKER`	«Quiero subir una foto».	Cambia al modo Imagen y, 100 ms después, abre el selector de archivos pulsando por programa un `<input type="file">` oculto.
`SPEAK_LAST`	«Repite tu última respuesta».	Pulsa por programa el primer botón de altavoz de la página, que corresponde a la respuesta más reciente del historial; no añade una entrada nueva.
`POINT_TO`	«¿Dónde está...?», «busca...».	Cambia al modo Imagen o Webcam (según haya imagen o cámara) y dibuja flechas «¡Aquí!» sobre las detecciones en lugar de cajas.

Tabla 3.4: Acciones del protocolo y reacción de la interfaz a cada una.

La figura 3.15 muestra el recorrido completo de una consulta de localización con la webcam: el agente necesita dos pasos (la llamada a la herramienta y la respuesta final), la herramienta provoca a su vez una llamada al backend con su propia consulta a GPT-4o mini, y la respuesta vuelve al navegador con la acción `POINT_TO`. La figura 3.16 reproduce cómo se dibuja el resultado con el mismo código de dibujo de la aplicación.

La figura 3.16 revela una limitación del protocolo que conviene reconocer: la etiqueta `[POINT_TO]` no dice *a qué* señalar. El navegador dibuja una flecha sobre cada detección devuelta, y las herramientas de análisis devuelven todas las detecciones de YOLO de la imagen, no solo la del objeto buscado. Cuando solo hay un objeto detectado, o cuando la búsqueda es de texto (la herramienta de OCR devuelve solo la coincidencia), el resultado es exacto; cuando hay varios, se señalan todos. La mejora natural es que la etiqueta lleve un argumento, por ejemplo `[POINT_TO:televisor]`, y filtrar las detecciones por clase. Otro riesgo, identificado al analizar el flujo, afecta a `OPEN_FILE_PICKER`: los navegadores solo permiten abrir el selector de archivos durante la *activación transitoria* que sigue a una acción del usuario, que en Chrome dura unos pocos segundos. Si el agente tarda más en responder, el navegador puede bloquear la apertura, y el usuario tendría que pulsar el área de subida manualmente.

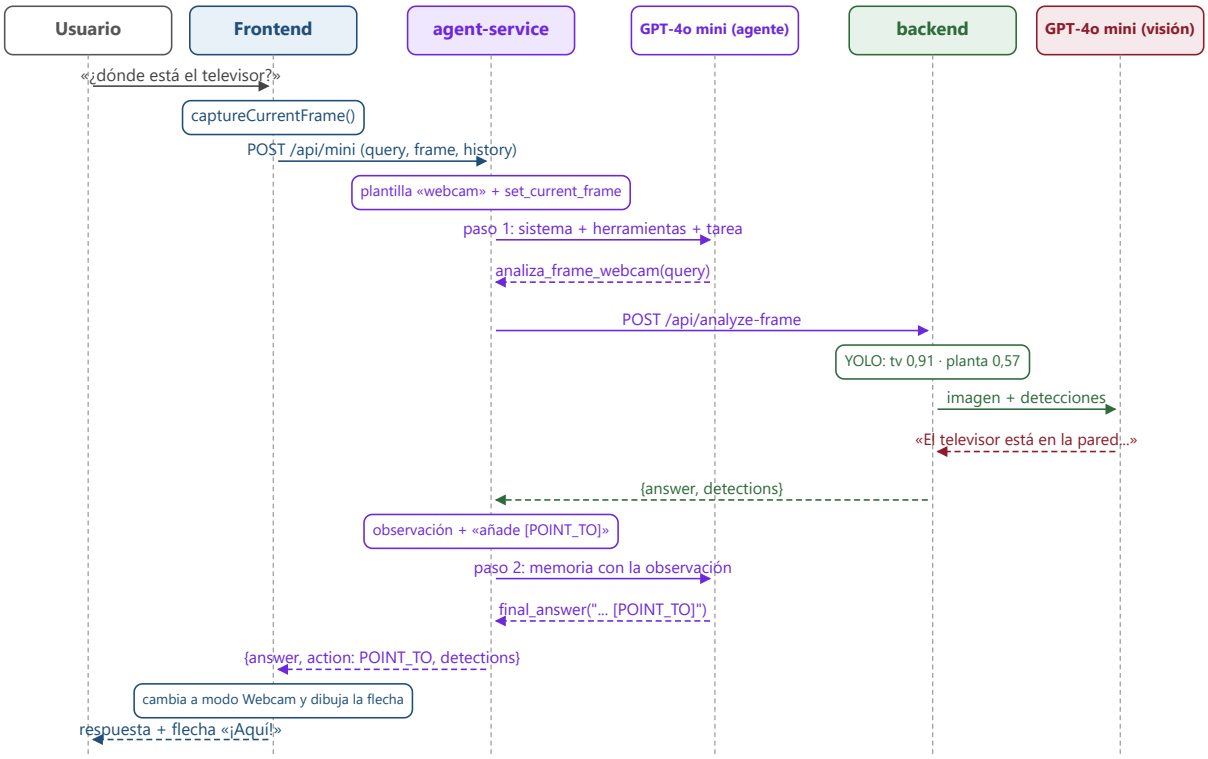


Figura 3.15: Secuencia de una consulta de localización en el modo Mini con la webcam activa: tres llamadas a GPT-4o mini (dos del agente y una del backend) y una acción `POINT_TO` ejecutada por el navegador.



Figura 3.16: Dibujo de las detecciones reales de YOLOv8n sobre un fotograma de prueba, reproducido con la misma geometría que los componentes `OverlayCanvas` y `WebcamCapture`: (a) cajas normales; (b) y (c) flechas de la acción `POINT_TO` en los modos Imagen y Webcam. La flecha se dibuja sobre *todas* las detecciones, también sobre la planta, aunque la pregunta fuera por el televisor.

3.7.7. Búsqueda de texto con OCR

La última herramienta, incorporada en la sesión 15, permite localizar una palabra en la imagen o en la webcam y señalarla. Su funcionamiento es el siguiente:

1. **Carga diferida.** El lector de EasyOCR se crea la primera vez que se usa la herramienta, no al arrancar el servicio, para no retrasar el arranque ni ocupar memoria si nunca se busca texto. Se cargan juntos los seis idiomas de la interfaz, que comparten el modelo latino. Se solicita GPU (`gpu=True`), pero en los contenedores, que usan PyTorch para CPU, EasyOCR recurre automáticamente a la CPU.

2. **Fuente de la imagen.** Si la tarea incluye la ruta de una imagen subida, se lee de disco; si no, se decodifica el fotograma de la webcam guardado en el estado compartido.

3. **Búsqueda normalizada.** Se ejecuta `readtext` y se recorre la lista de fragmentos buscando uno que *contenga* el texto buscado, comparando ambos en minúsculas y sin espacios. Quitar los espacios fue la mitigación que se aplicó al observar que, en textos muy espaciados, el OCR partía o unía palabras de forma impredecible.

4. **Conversión de coordenadas.** EasyOCR devuelve un polígono de cuatro vértices; la herramienta calcula su caja envolvente alineada con los ejes (mínimos y máximos de x e y) para obtener el mismo formato `[x1, y1, x2, y2]` que YOLO.

5. **Un texto «disfrazado» de objeto.** La coincidencia se añade a las detecciones con la clase «Texto: ...» y la confianza del OCR. De este modo, el navegador la dibuja con exactamente el mismo código que las detecciones de YOLO, flecha `POINT_TO` incluida, sin haber tenido que modificar el frontend.

Este último punto ilustra el valor de un formato común de detección: una nueva fuente de información visual se integró en la interfaz sin tocar una línea de código del navegador. Las limitaciones de la herramienta —la unidad de búsqueda es el fragmento que devuelve el detector, la caja puede abarcar toda una línea y los textos en vertical o en diagonal no se encuentran— se evalúan en el capítulo 4.

3.7.8. Integración en la interfaz

En el modo Mini, cada petición a `/api/mini` incluye la imagen seleccionada, si la hay; el fotograma actual de la webcam, si está encendida —el componente de la webcam permanece montado aunque no esté visible, precisamente para que Mini pueda usar la cámara desde su propio modo—; y el historial, si el modo conversación está activo. La

respuesta se añade al historial visible con la miniatura del fotograma y, según la acción, la interfaz cambia de modo para mostrar el resultado en contexto. Al elegir una imagen se apaga la cámara y viceversa, para que nunca haya dos fuentes visuales simultáneas y la petición sea inequívoca; este comportamiento se incorporó en la sesión 13.

3.8. La interfaz de usuario

3.8.1. Estructura de componentes

La figura 3.17 muestra la estructura del frontend. El punto de entrada envuelve la aplicación en tres proveedores de contexto —tema, idioma y volumen— y el componente `App` concentra el estado principal (tabla 3.5) y la función `handleSubmit`, que decide a qué *endpoint* enviar cada consulta según el modo y ejecuta la acción devuelta. Los componentes reutilizables encapsulan cada capacidad: `WebcamCapture` expone mediante `forwardRef` y `useImperativeHandle` dos métodos (`captureCurrentFrame` y `stop`) que el resto de la aplicación puede invocar; `AudioInput` y `AudioOutput` gestionan la voz; y `OverlayCanvas` el dibujo sobre imágenes.

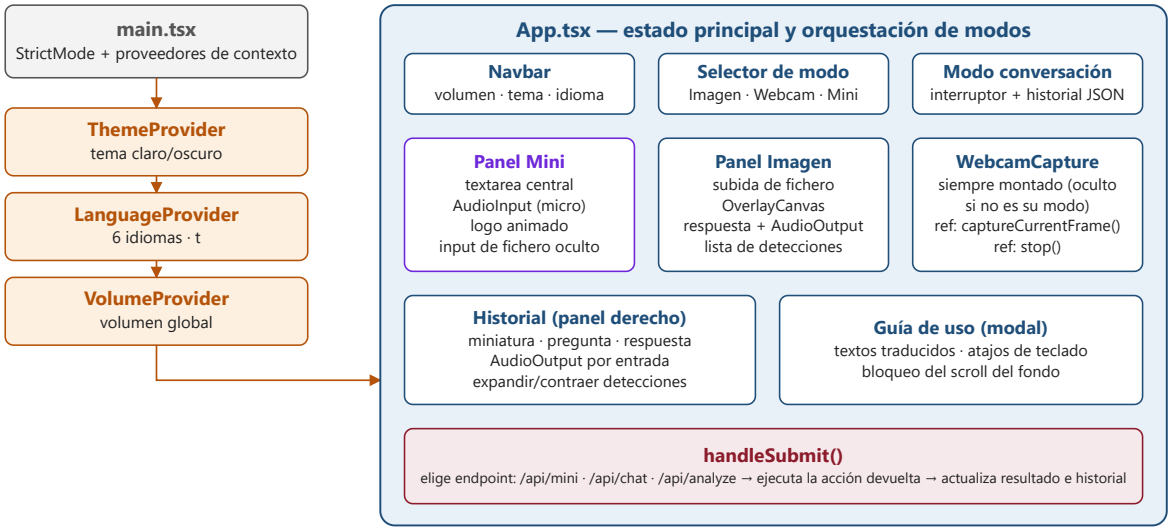


Figura 3.17: Estructura de componentes del frontend y proveedores de contexto.

La figura 3.18 muestra la interfaz final en sus tres modos y la guía de uso integrada, un modal con las explicaciones y los atajos de teclado traducidos a los seis idiomas que, al abrirse, bloquea el desplazamiento de la página de fondo (un problema real: sin ese bloqueo, la rueda del ratón movía la página de detrás y producía saltos extraños).

Estado	Función
`mode`	Modo activo: `'image'`, `'webcam'` o `'mini'`.
`image`, `preview`	Fichero seleccionado y URL de objeto para previsualizarlo.
`query`, `result`, `loading`, `error`	Pregunta en edición, última respuesta y estado de la petición.
`conversationMode`, `conversationHistory`	Interruptor del modo conversación e historial JSON que se reenvía al servidor.
`history`, `expandedIndex`	Historial visible (miniatura, pregunta, respuesta, detecciones) y entrada desplegada.
`isWebcamActive`, `webcamTrigger`	Estado de la cámara y contador que dispara un análisis en el modo conversación.
`isManualOpen`	Visibilidad de la guía de uso.

Tabla 3.5: Principales variables de estado del componente `App`.

3.8.2. Internacionalización y temas

Todos los textos de la interfaz están en un único objeto con seis idiomas y unas sesenta claves cada uno, servido por un contexto de React (`LanguageContext`). Los componentes obtienen del contexto el objeto `t` con los textos del idioma activo, y el idioma elegido se recuerda en `localStorage`. Dos detalles de diseño: los textos que dependen de un número se definen como funciones (`detectedObjects: (n) => ` `${n} objeto(s) detectado(s)` `), y el tipo de TypeScript de las traducciones se deriva del objeto en español, de modo que el compilador detecta si falta una clave en un idioma. Como se ha visto, el idioma no se queda en la interfaz: viaja en cada petición y condiciona la respuesta del LLM, la traducción de clases y colores, la voz de síntesis, el idioma de Whisper y los mensajes de error.

El tema claro u oscuro sigue el mismo patrón con `ThemeContext`, que añade o quita la clase `dark` del elemento raíz y la guarda en `localStorage`. En Tailwind CSS v4, la variante `dark:` se vincula a esa clase con una sola línea de CSS: `@custom-variant dark (&:is(.dark *));`. A partir de ahí, cada elemento declara sus dos aspectos (por ejemplo, `bg-white dark:bg-gray-900`). La figura 3.19 muestra la interfaz con el tema claro y en otros idiomas.

3.8.3. Detalles de la experiencia de usuario

Algunos detalles visuales requirieron soluciones con cierto interés técnico:

- **Sincronización de gradientes animados.** Los botones principales tienen un gradiente animado con un ciclo de 4 segundos. Como cada botón empieza su animación al montarse, al cambiar de modo los gradientes quedaban desfasados. La solución (listado 3.5) aprovecha que un retardo de animación negativo, $-d$, hace que la animación empiece «como si hubiera comenzado hace d ms». Si todos los elementos usan $d = t_{actual}$ mód 4000, todos quedan en la misma fase respecto a un reloj global, independientemente de cuándo se montaron.

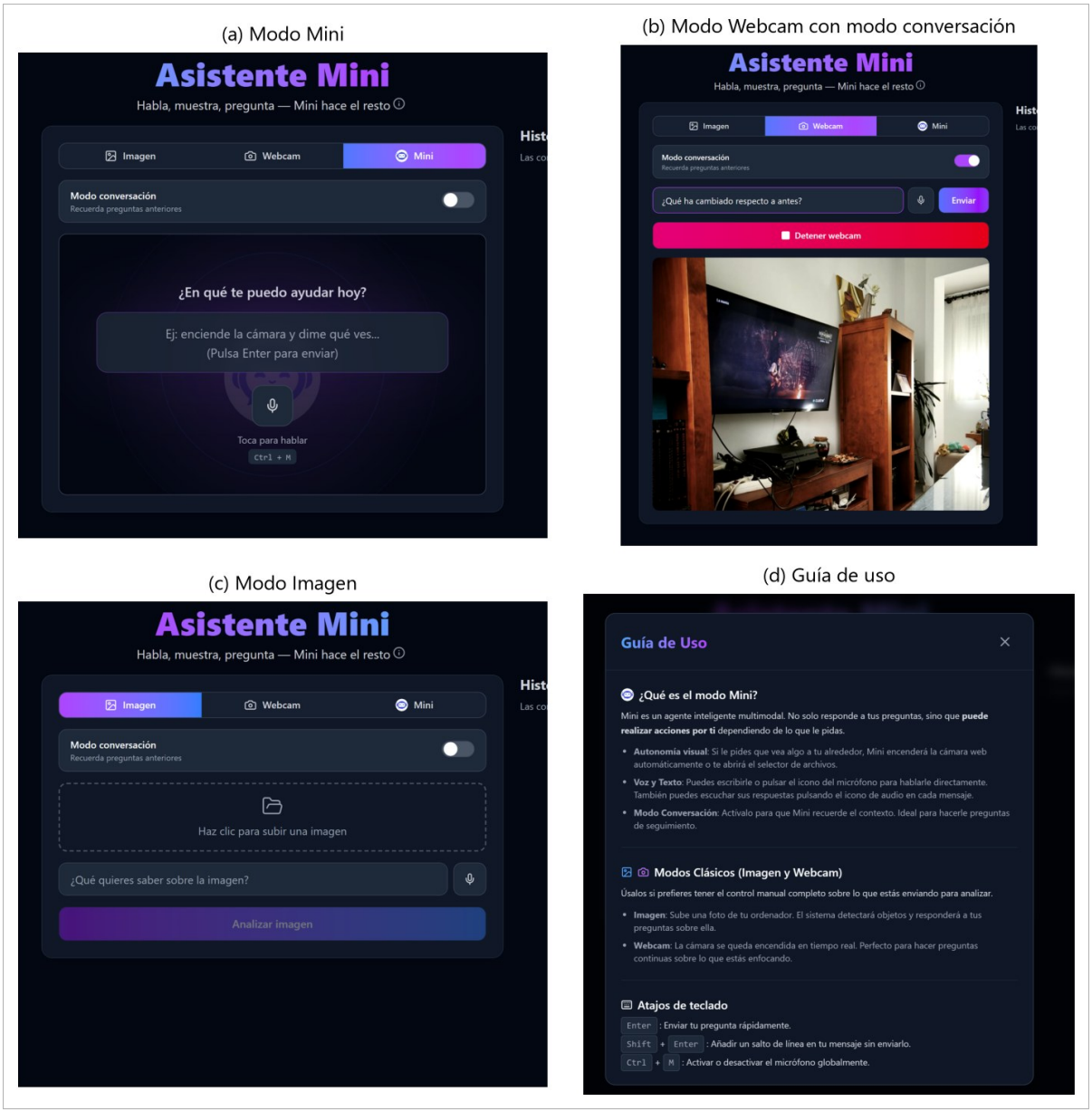


Figura 3.18: Interfaz final de la aplicación (capturas de la versión 1.2.0): (a) modo Mini; (b) modo Webcam con el modo conversación activado; (c) modo Imagen; (d) guía de uso integrada.

- **Logotipo flotante del modo Mini.** El logotipo rebota por el panel como el clásico salvapantallas de los reproductores de DVD. Se anima con `requestAnimationFrame` modificando directamente la transformación CSS del elemento a través de una referencia —sin pasar por el estado de React, que provocaría sesenta renderizados por segundo— e invirtiendo la velocidad al tocar un borde.

- **Cuadro de texto autoajustable.** Crece con el contenido fijando su altura a `scrollHeight` en cada cambio, con un máximo y una barra de desplazamiento oculta por CSS.

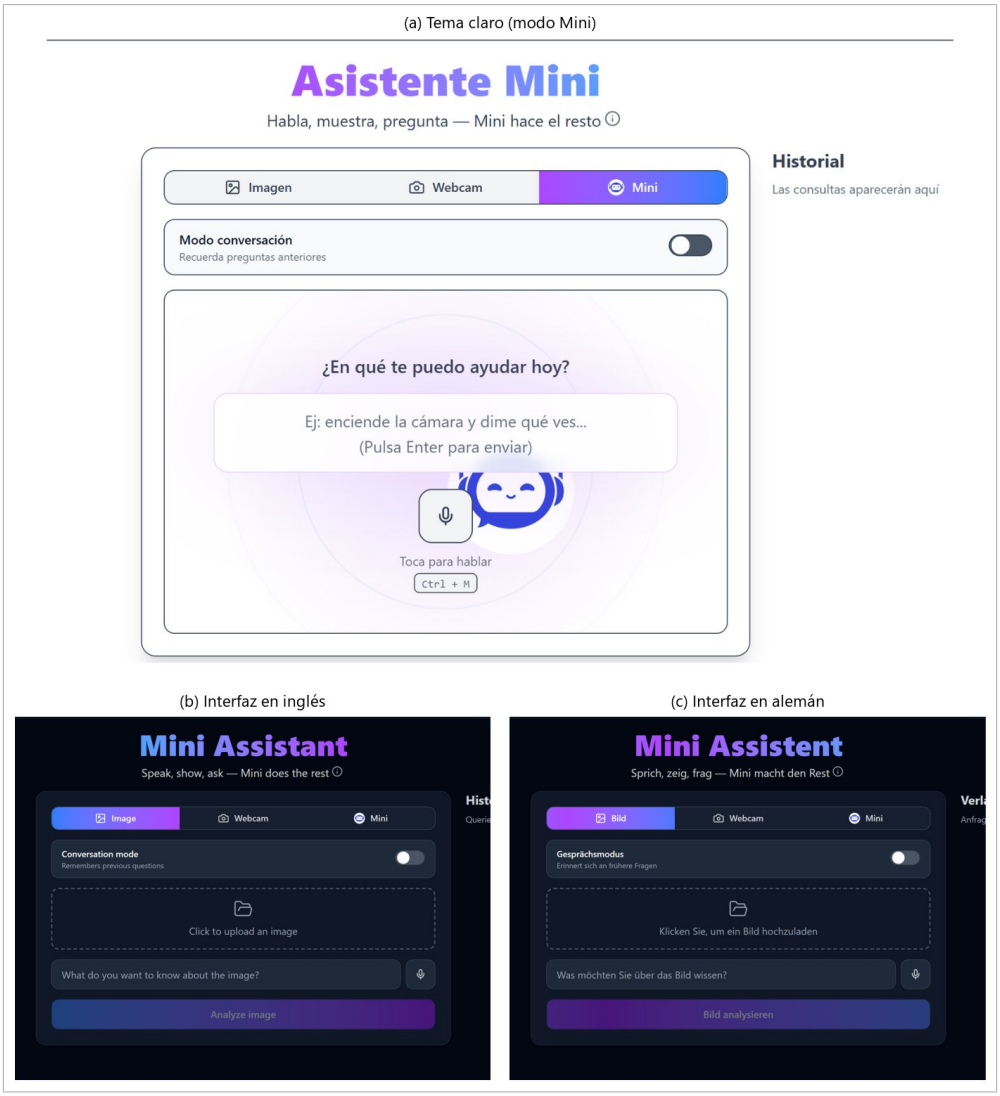


Figura 3.19: Tema claro e internacionalización: (a) modo Mini con el tema claro; (b) y (c) la interfaz en inglés y en alemán.

Listado 3.5: Sincronización de las animaciones de gradiente con un reloj global (utils/animations.ts).

```
export const syncGradient = (el: HTMLElement | null) => {
  if (el && !el.style.animationDelay) {
    el.style.animationDelay = `-${Date.now() % 4000}ms`   // misma fase para todos los elementos
  }
}
```

3.9. Seguridad, validación y robustez

Aunque el sistema es un prototipo de uso local, se adoptaron varias medidas de seguridad y robustez, y el análisis del código ha permitido identificar otras que quedan pendientes (tabla 3.6).

- **Inyección de *prompts*.** Los mensajes de sistema del backend indican que el texto delimitado por `<user_query>` es contenido a analizar y nunca instrucciones, aunque contenga frases como «ignora lo anterior». En `/api/chat` la pregunta se envía efectivamente entre esas etiquetas. Sin embargo, la revisión realizada para esta memoria ha detectado que en `/api/analyze` y `/api/analyze-frame` la pregunta se inserta sin las etiquetas, y que el agente la incluye directamente en su tarea, por lo que la defensa solo se aplica completa en el modo conversación. Tampoco se protege contra la inyección indirecta a través del texto que aparezca en las imágenes.

- **Validación de entradas.** Las preguntas de más de 1000 caracteres se rechazan en el backend con un error HTTP 400, y el cuadro de texto del navegador impone el mismo límite (`maxLength`) y muestra un mensaje traducido si el servidor lo rechaza. El servicio del agente no aplica esa validación. Los fotogramas se validan tras decodificarlos y se descartan si están vacíos o corruptos.

- **Errores de la API.** El error de límite de peticiones se traduce a un mensaje comprensible en los seis idiomas; el resto de excepciones se devuelve como texto, lo que es útil para depurar pero puede exponer detalles internos y debería sustituirse por un mensaje genérico con registro en el servidor.

- **Secretos.** La clave de OpenAI solo existe en `.env`, que está excluido de Git y de las imágenes Docker (`.dockerignore`) y se inyecta en tiempo de ejecución con `env_file`. Las imágenes publicadas en Docker Hub no contienen, por tanto, ninguna clave.

- **Ficheros temporales.** Todos los ficheros temporales (audio, MP3, imágenes del agente) se borran en bloques `finally` o en tareas de fondo.

- **CORS y red.** Los servicios aceptan peticiones de cualquier origen (`allow_origins=["*"]`), una configuración cómoda para el desarrollo que el propio código señala como pendiente de restringir al origen del frontend en un despliegue real, junto con HTTPS y autenticación.

3.10. Contenerización y despliegue

El sistema se ejecuta con un único comando, `docker compose up`, gracias a tres `Dockerfile` y un fichero `docker-compose.yml` (figura 3.20). Las imágenes de Python parten de `python:3.12-slim` —una versión anterior a la 3.13 del entorno de desarrollo, elección conservadora que facilita disponer de paquetes precompilados para todas las dependencias— e instalan las bibliotecas de sistema que necesita OpenCV. Después

Aspecto	Implementado	Pendiente
Inyección de *prompts*	Etiquetas y aviso en el mensaje de sistema (`/api/chat`).	Aplicarlo en todos los *endpoints* y en el agente; inyección indirecta.
Validación de entradas	Longitud máxima en backend y frontend; validación de fotogramas.	Validación en el servicio del agente; tamaño máximo de ficheros.
Integridad del historial	—	Firmar el historial o guardarlo en el servidor.
Secretos	`.env` fuera de Git y de las imágenes.	Gestor de secretos en despliegues reales.
Concurrencia	Agente nuevo por petición.	Estado de las herramientas por petición.
Transporte y acceso	—	HTTPS, CORS restringido, autenticación, límites por usuario.

Tabla 3.6: Resumen de medidas de seguridad implementadas y pendientes.

instalan PyTorch desde el índice de paquetes *para CPU* de PyTorch, antes que el resto de dependencias: sin este paso, `pip` descargaría la versión con CUDA, varios gigabytes más pesada e inútil en un contenedor sin GPU. El frontend usa `node:24-alpine` y ejecuta el servidor de desarrollo de Vite con `-host` para que sea accesible desde fuera del contenedor.

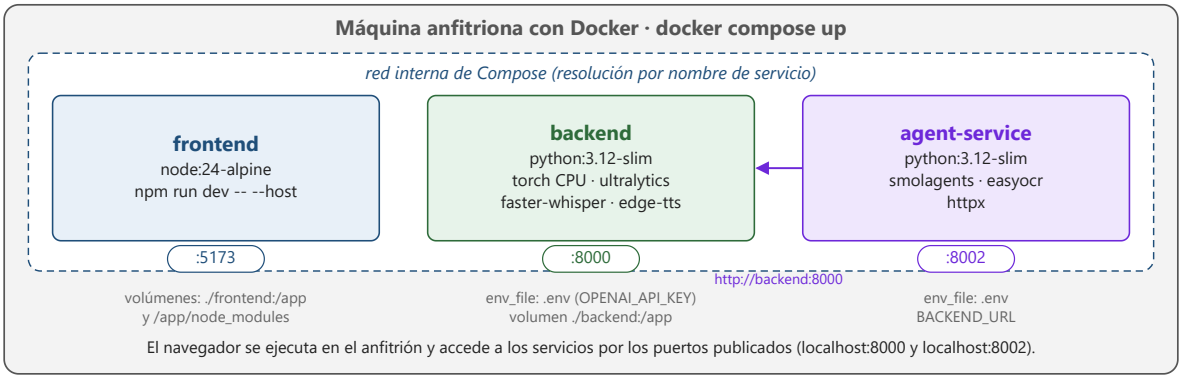


Figura 3.20: Despliegue con Docker Compose: tres contenedores en una red interna en la que el agente accede al backend por su nombre de servicio, mientras que el navegador, que se ejecuta en el anfitrión, usa los puertos publicados.

En Compose, cada servicio publica su puerto y recibe la clave desde `.env`. El agente localiza al backend mediante la variable `BACKEND_URL=http://backend:8000`, aprovechando que Compose resuelve el nombre de cada servicio dentro de su red interna. Hay aquí una sutileza que conviene entender: el *navegador* no está en esa red, sino en la máquina anfitriona, así que el frontend debe apuntar a `http://localhost:8000` y `http://localhost:8002` (variables `VITE_API_URL` y `VITE_AGENT_URL`), no a los nombres de servicio. Los volúmenes montan el código de cada servicio para poder desarrollar con recarga automática, y un volumen anónimo en `/app/node_modules` impide que la carpeta del anfitrión oculte las dependencias instaladas en la imagen. Los pesos de los modelos (YOLO, Whisper y

EasyOCR) no se incluyen en las imágenes: se descargan en el primer uso, lo que alarga el primer arranque.

La contenerización también tuvo sus problemas: el paquete `libgl1-mesa-glx`, que recomiendan muchos tutoriales para OpenCV, ya no existe en Debian *trixie* (la base de las imágenes *slim* actuales) y hubo que sustituirlo por `libgl1`; el contenedor del agente se caía al importar OpenCV porque le faltaban bibliotecas del sistema (`libxcb1`, entre otras); y una configuración errónea de `env_file` impedía que la clave llegase a los servicios. Al cerrar la fase 2, las imágenes se publicaron en Docker Hub, de modo que cualquiera puede desplegar el sistema sin instalar Python ni Node.js, sustituyendo en el fichero de Compose las directivas `build` por las imágenes publicadas. Con ello se etiquetó la versión v1.2.0. El anexo A resume el procedimiento de instalación y despliegue.

> **PARA QUIEN CONTINÚE ESTE TRABAJO**
>
> El frontend se despliega con el servidor de desarrollo de Vite, cómodo pero no apropiado para producción. Una mejora sencilla sería una imagen multietapa que compile la aplicación (`npm run build`) y sirva los ficheros estáticos con Nginx, que además podría actuar como *proxy* inverso hacia los dos servicios y eliminar la necesidad de CORS.

Capítulo 4

Experimentación y casos de uso

Este capítulo evalúa el sistema de forma cuantitativa y crítica. Para cada componente se muestra primero su funcionamiento en casos canónicos y después su rendimiento sobre baterías de ejemplos, con especial atención a los casos que revelan sus limitaciones. Al final se analiza el coste de la orquestación mediante el agente, se repasan los casos de uso del sistema completo y se contrastan los resultados con los criterios de éxito fijados en la propuesta.

4.1. Entorno y metodología experimental

Todos los experimentos se ejecutaron en el mismo equipo en el que se desarrolló el proyecto: un procesador Intel Core i5-9400F (6 núcleos a 2,9 GHz), 16 GB de memoria y Windows 11. El equipo dispone de una GPU NVIDIA GTX 1660, pero *no se ha utilizado*: los contenedores del sistema ejecutan PyTorch para CPU, y el objetivo era medir la configuración real de despliegue. Las versiones de software fueron Python 3.13.5, PyTorch 2.14 (CPU), Ultralytics 8.4.33, faster-whisper 1.2.1 (CTranslate2 4.8), EasyOCR 1.7.2, edge-tts 7.2.8, OpenCV 5.0 y Chrome 153.

Cada experimento reproduce exactamente la ruta de código del backend: las mismas llamadas, parámetros y umbrales (YOLO con confianza 0,5; Whisper *small* en `int8` con idioma forzado y VAD; los mismos rangos HSV; la misma lógica de búsqueda de la herramienta de OCR; las mismas voces de síntesis). Las latencias se miden tras unas ejecuciones de calentamiento y se resumen con la mediana y el percentil 95, más robustos que la media ante valores atípicos. La tabla 4.1 resume las baterías utilizadas.

> **ALCANCE DE LA EVALUACIÓN**
>
> Las baterías automáticas no incluyen llamadas a GPT-4o mini. Ejecutarlas habría requerido usar la clave privada de la API en un entorno de pruebas, con un coste por ejecución, y

Componente	Datos	Métricas
YOLOv8n	Foto del repositorio; 500 imágenes aleatorias de COCO val2017 (3651 objetos); 100 de ellas en 3 estilos no fotográficos; logotipo de Mini.	Detecciones, precisión, recall, mAP, AP por clase, latencia por etapa.
Análisis de color	463 colores con nombre de la encuesta de xkcd; 9 colores puros oscurecidos en 6 niveles; 2 imágenes reales.	Exactitud, matriz de confusión, robustez a la iluminación, latencia.
Voz a texto	10 consultas típicas × 6 idiomas × 2 voces = 120 grabaciones sintéticas (≈ 3,1 s), sin ruido y con ruido blanco a 10 y 5 dB.	WER por modelo, condición e idioma; latencia.
Texto a voz	Las 120 consultas anteriores; respuestas en español de 8 a 128 palabras.	Tiempo de síntesis.
OCR	380 imágenes sintéticas (20 palabras, 5 tipografías, 3 tamaños, 3 rotaciones, fondo fotográfico); sopas de letras; foto real.	Tasa de acierto de la búsqueda, localización, latencia.
Agente	*Prompts* reales del sistema tokenizados; precios públicos de la API.	Llamadas al LLM y tokens por consulta; coste estimado.

Tabla 4.1: Baterías de experimentos realizadas para evaluar cada componente.

sus respuestas no son deterministas, lo que exige un protocolo de evaluación propio (varias repeticiones y una valoración, humana o automática, de cada respuesta). Por ello, la parte del sistema que depende del LLM se evalúa en este capítulo de tres formas complementarias: analizando el coste y el número de llamadas a partir de los *prompts* reales, describiendo el recorrido de cada caso de uso según el diseño, y comentando los comportamientos observados y documentados durante el desarrollo. La medición sistemática de la latencia extremo a extremo con el modelo remoto queda como trabajo futuro.

4.2. Detección de objetos con YOLOv8n

4.2.1. Caso canónico: la fotografía del repositorio

La figura 4.1 muestra las detecciones de YOLOv8n sobre la fotografía de prueba incluida en el repositorio (un salón de 1536×2048 píxeles), dibujadas con el mismo estilo que la aplicación. Con el umbral de confianza del backend (0,5) se detectan dos objetos: el televisor (86 %) y la planta (55 %). Ambos son correctos. Con el umbral por defecto de Ultralytics (0,25) aparecen tres detecciones más, las tres erróneas: una caja azul del mueble clasificada como «televisor» (31 %), la mesa cubierta con una tela azul clasificada como «cama» (27 %) y la figura decorativa de lo alto de la estantería clasificada como «planta» (26 %).

El caso es representativo del comportamiento del detector en escenas domésticas: lo que

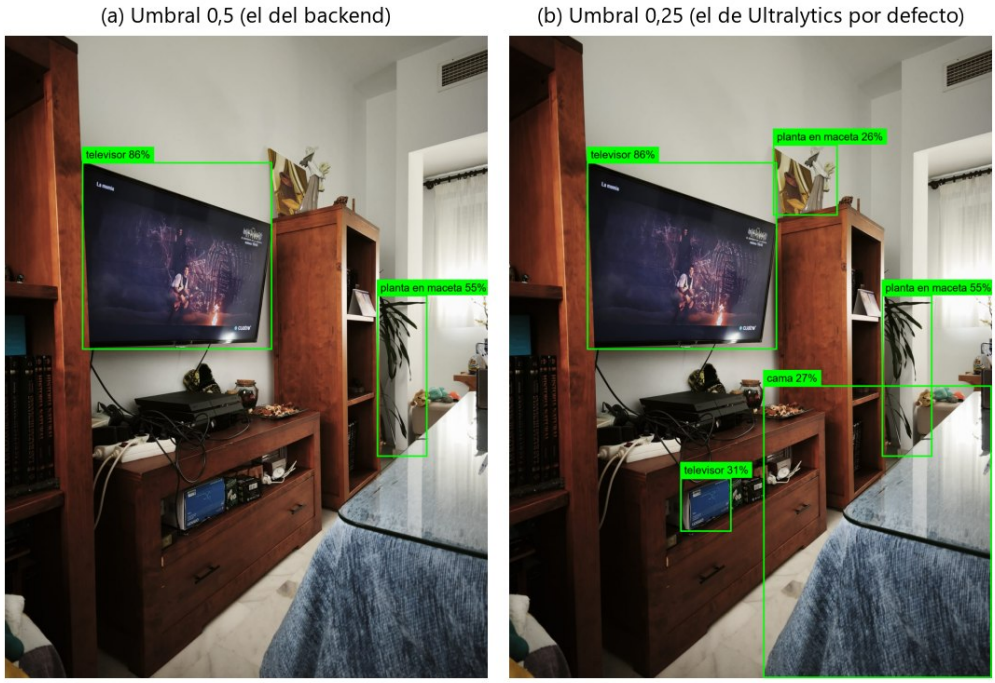


Figura 4.1: Detecciones de YOLOv8n sobre la fotografía de prueba del repositorio con el umbral del backend (a) y con el umbral por defecto de Ultralytics (b). Las tres detecciones adicionales de (b) son falsos positivos.

detecta con confianza alta suele ser correcto, pero deja sin detectar muchos objetos visibles —los libros de la estantería, la mesa, los peluches, la consola—, ya sea porque aparecen pequeños, parcialmente ocultos o con un aspecto poco típico. Bajar el umbral no recupera esos objetos y sí introduce errores. Para el sistema, esto significa que las detecciones son una fuente fiable pero muy incompleta, y justifica que el LLM reciba también la imagen.

4.2.2. Precisión en un conjunto independiente

Para medir la precisión del detector de forma objetiva se seleccionaron al azar (con semilla fija) 500 imágenes del conjunto de validación de COCO 2017, que contienen 3651 objetos anotados y *no* se usaron para entrenar el modelo. Sobre ellas, YOLOv8n obtiene una $\text{mAP}_{50\text{–}95}$ de 0,391 y una mAP_{50} de 0,546, coherentes con el 0,373 que Ultralytics publica para el conjunto de validación completo (la diferencia es la esperable al usar una muestra). Como ejercicio de contraste, la validación sobre COCO128 —el pequeño conjunto de demostración de Ultralytics— da una $\text{mAP}_{50\text{–}95}$ de 0,448, sensiblemente mayor. La explicación es instructiva: COCO128 está formado por las primeras 128 imágenes del conjunto de *entrenamiento*, que el modelo ya ha visto. Evaluar sobre datos de entrenamiento sobreestima el rendimiento, y es un error fácil de cometer.

La mAP resume el comportamiento del detector para todos los umbrales, pero la aplicación usa uno fijo. Por eso se midieron además la precisión y el recall a varios umbrales, emparejando cada detección con un objeto real de la misma clase con IoU $\geq 0{,}5$ (figura 4.2

y tabla 4.2). Con el umbral de 0,5 del backend, el 87 % de las detecciones son correctas, pero solo se encuentra el 37 % de los objetos, y apenas el 16 % de los objetos pequeños (los que ocupan menos del 1 % de la imagen).

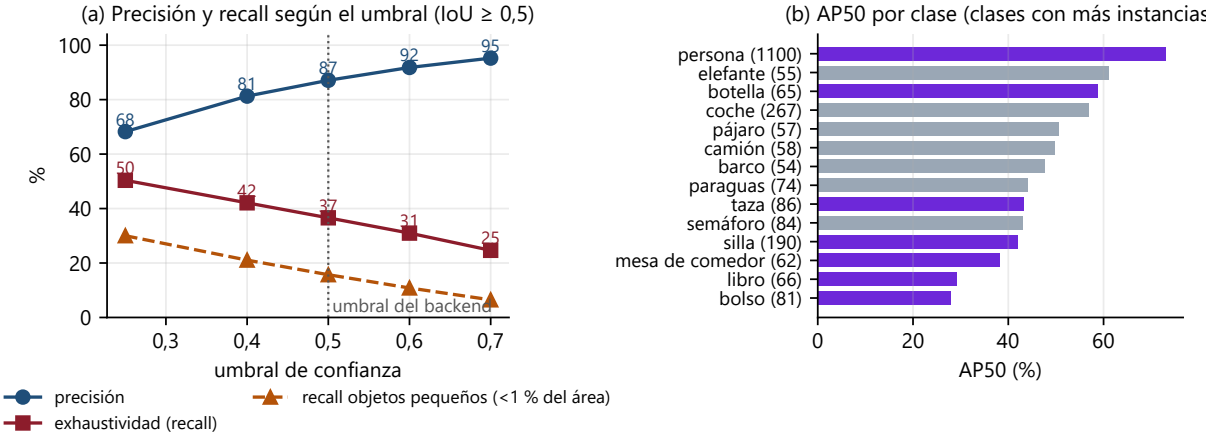


Figura 4.2: Rendimiento de YOLOv8n sobre 500 imágenes de COCO val2017: (a) precisión y recall en función del umbral de confianza; (b) AP50 de las clases con más instancias en la muestra (entre paréntesis), destacando en morado las más habituales en interiores.

Umbral	Detecciones correctas	Falsos positivos	Precisión	Recall	Recall objetos pequeños
0,25	1839	856	68,2 %	50,4 %	30,0 %
0,40	1537	354	81,3 %	42,1 %	21,1 %
0,50	**1336**	**198**	**87,1 %**	**36,6 %**	**15,7 %**
0,60	1131	101	91,8 %	31,0 %	10,8 %
0,70	900	45	95,2 %	24,7 %	6,5 %

Tabla 4.2: Precisión y recall de YOLOv8n según el umbral de confianza (500 imágenes de COCO val2017, IoU $\geq$ 0,5).

¿Es 0,5 un buen umbral para este sistema? El análisis sugiere que sí, por una razón que va más allá de la detección: las detecciones se entregan al LLM como hechos, y un falso positivo —una «cama» inexistente— puede contaminar su respuesta, mientras que un falso negativo lo compensa en parte el propio modelo al ver la imagen. En un sistema híbrido como este interesa, por tanto, un detector preciso más que exhaustivo. El AP por clase (figura 4.2b) matiza el resultado: las personas se detectan bien (AP50 del 73 %), pero clases domésticas frecuentes como sillas (42 %), mesas (38 %), libros (29 %) o bolsos (28 %) quedan muy por debajo, lo que coincide con lo observado en la fotografía del repositorio.

4.2.3. Latencia

La figura 4.3 desglosa el tiempo de cada etapa en CPU, medido con 50 repeticiones tras cinco de calentamiento. Para un fotograma de webcam de 640×480, la mediana total es de

48 ms, de los que 44 corresponden a la inferencia; decodificar el JPEG y preparar el tensor apenas cuesta 2–3 ms. Para una foto de 1536×2048 la inferencia cuesta lo mismo —la red trabaja siempre a 640 píxeles de lado mayor—, pero la decodificación del JPEG grande añade 28 ms. En términos de fotogramas por segundo, YOLOv8n podría procesar unos 20 fotogramas por segundo en esta CPU, muy por encima del objetivo de 5 que fijaba la propuesta para la fase 1. La detección no es, por tanto, el cuello de botella del sistema: lo son las llamadas al LLM y sus límites de uso (apartado 3.4.2).

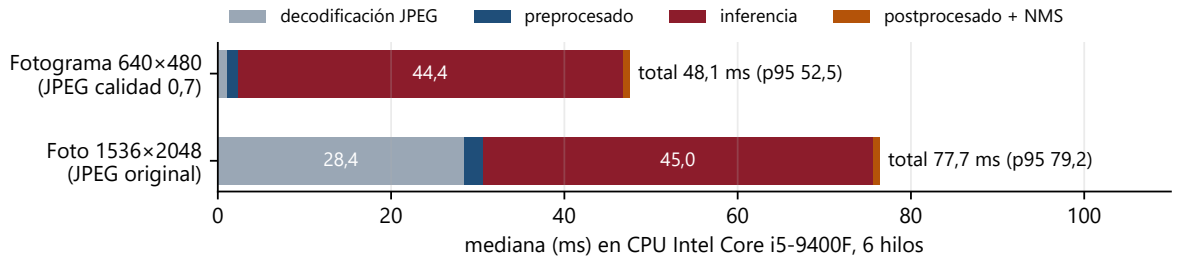


Figura 4.3: Latencia de YOLOv8n por etapas en CPU (mediana de 50 ejecuciones) para un fotograma de webcam y para la fotografía del repositorio.

Una observación de implementación: estos 48 ms se ejecutan dentro de la función `async` del *endpoint*, y durante ese tiempo el bucle de eventos de FastAPI queda bloqueado (apartado 2.9). Con un único usuario es irrelevante; con varios usuarios simultáneos convendría ejecutar la inferencia en un hilo auxiliar, como ya se hace con Whisper.

4.2.4. Robustez ante el cambio de dominio

En la sesión 5 del desarrollo se observó que YOLO no detectaba nada en logotipos ni ilustraciones. Para cuantificar el fenómeno se tomaron 100 imágenes de la muestra de COCO (760 objetos anotados) y se transformaron con tres filtros que las alejan del dominio fotográfico: un efecto de pintura (`cv2.stylization`), un boceto a color (`cv2.pencilSketch`) y un efecto de dibujo animado (suavizado bilateral, cuantización a 8 colores y contornos). La figura 4.4 muestra el resultado. El recall cae del 34,9 % en las fotos originales al 12,4 % con el efecto de pintura, al 9,5 % con el boceto y al 2,6 % con el dibujo animado, en el que además la precisión se desploma hasta el 52,6 %. El ejemplo de la figura es elocuente: los siete elefantes detectados en la foto desaparecen en las versiones estilizadas, y en el boceto uno de ellos se confunde con un «banco». Sobre el logotipo de Mini —una ilustración— no hay ninguna detección con umbral 0,5 ni 0,25; solo al bajarlo a 0,1 aparece una «cometa» con un 12 % de confianza.

El experimento confirma cuantitativamente la observación del desarrollo y la decisión de enviar la imagen al LLM: un detector entrenado solo con fotografías es frágil fuera de ese dominio, mientras que los modelos multimodales, entrenados con datos mucho más diversos, describen sin dificultad ilustraciones, capturas de pantalla o dibujos.

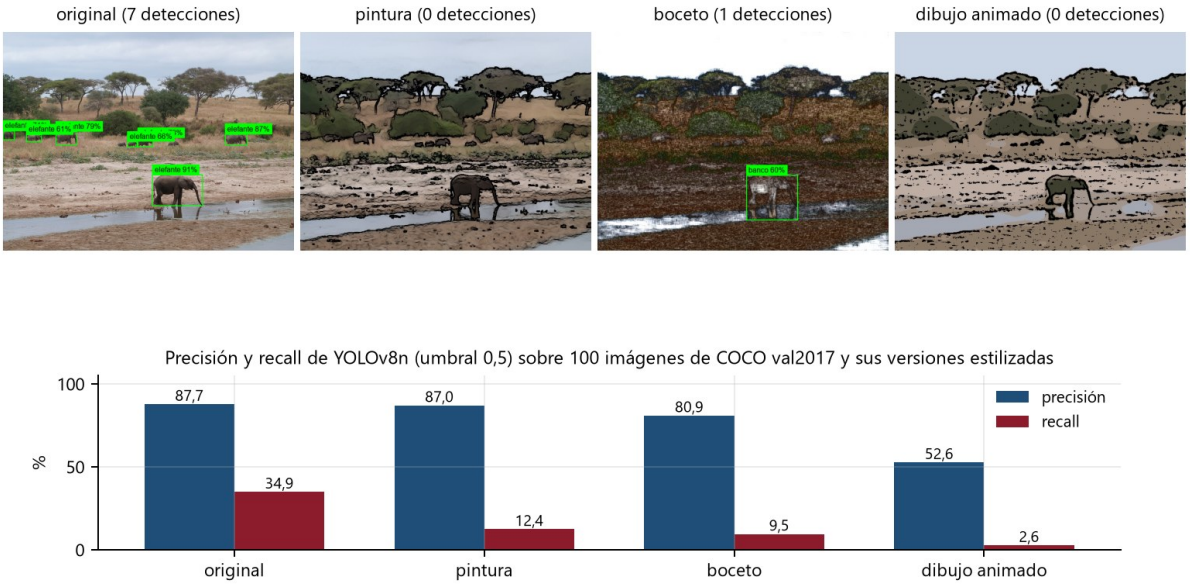


Figura 4.4: Robustez de YOLOv8n ante el cambio de dominio. Arriba, una imagen de COCO val2017 y sus tres versiones estilizadas con las detecciones obtenidas (umbral 0,5). Abajo, precisión y recall agregados sobre 100 imágenes.

4.3. Análisis de color

4.3.1. Exactitud sobre colores con nombre

Evaluar un clasificador de colores exige una referencia de cómo nombran los colores las personas. Se utilizó la encuesta de colores de xkcd [64], en la que participantes de más de doscientas mil sesiones nombraron colores mostrados en pantalla, y que proporciona casi 950 colores con el nombre más votado. Se seleccionaron los 463 cuyo nombre contiene exactamente uno de los términos básicos de color (por ejemplo, *dark blue* o *forest green*, pero no *blue green*), y se clasificó un parche uniforme de cada uno con el mismo algoritmo del backend. La figura 4.5 muestra la matriz de confusión.

Sobre los 431 colores cuyas categorías existen en el sistema, la exactitud es del 80,7 %. El análisis por categorías es más revelador que la cifra global: rojo (96 %), morado (95 %), azul (90 %), naranja (89 %) y verde (88 %) se reconocen bien; el amarillo (72 %) se confunde con el naranja y el verde en sus variantes oscuras o verdosas; el gris solo se acierta en 5 de 15 casos, porque muchos grises con nombre tienen un tinte que los lleva a la franja de saturación sin categoría o a un color cromático; y el rosa, con solo un 31 % de aciertos, es el caso más problemático: la mitad de los rosas se clasifican como rojos, porque el sistema los define solo por un estrecho rango de tono (161–169) cuando, como se vio en la figura 2.10c, un rosa es sobre todo un rojo poco saturado. Por último, **ninguno de los 32 marrones** puede clasificarse correctamente, porque el marrón no existe en la paleta: 26 se clasifican como naranja y 3 como rojo. Es una omisión relevante, porque el marrón es uno de los once términos básicos de color [9] y el color de buena parte del mobiliario doméstico.

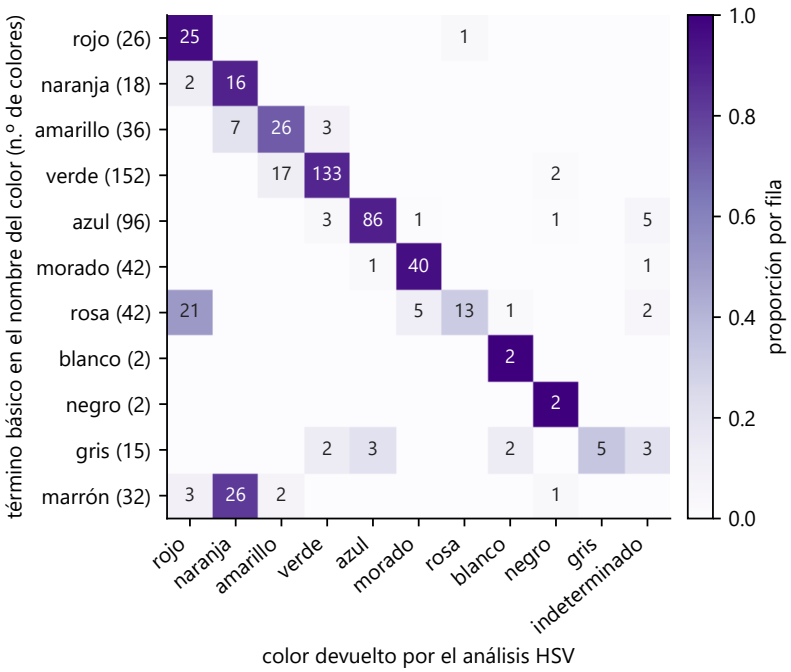


Figura 4.5: Matriz de confusión del análisis de color HSV sobre 463 colores de la encuesta de xkcd. Las filas son el término básico presente en el nombre del color y las columnas, el color devuelto por el sistema.

4.3.2. Robustez ante la iluminación y colores en escenas reales

La figura 4.6 muestra qué ocurre al oscurecer nueve colores puros, multiplicando su brillo por factores entre 0,15 y 1. Con el brillo reducido a la cuarta parte, el verde, el morado y el gris ya se clasifican como negro, y el blanco se clasifica como gris hasta que recupera el 80 % de su brillo. En otras palabras, el mismo objeto cambia de color según la luz de la habitación, algo que el ojo humano compensa pero el análisis por umbrales no.

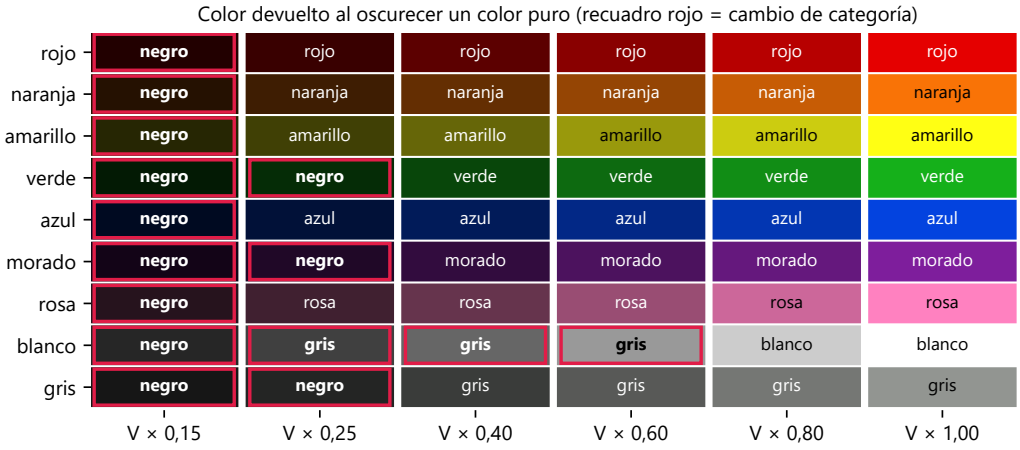


Figura 4.6: Efecto de la iluminación: color devuelto por el sistema al reducir el brillo de nueve colores puros. Los recuadros rojos marcan los cambios de categoría.

En las imágenes reales, el análisis se comporta como cabía esperar de estas limitaciones.

En el fotograma de la webcam (figura 3.9), el color dominante devuelto es el «rojo» (24,5 %), que corresponde a la madera de los muebles; en la fotografía completa del salón, el dominante es el «negro» (26,3 %), seguido del gris (25,1 %), el rojo (18,7 %), el azul de la tela de la mesa (9,9 %) y el blanco (7,6 %). Los porcentajes informados suman el 87,6 %: el resto son píxeles de colores por debajo del 5 % o de la franja de saturación sin categoría. El análisis es, eso sí, muy rápido: la mediana del *endpoint* completo, incluida la decodificación del fotograma, es de 2,9 ms.

La conclusión práctica es que el análisis HSV global responde bien a «¿qué colores predominan en la imagen?», pero no a la pregunta más habitual, «¿de qué color es *este objeto*?», que requiere restringir el análisis a la caja del objeto y, preferiblemente, usar un espacio perceptual como CIELAB y una paleta con los once términos básicos. En el sistema actual, esas preguntas las responde GPT-4o mini mirando la imagen.

4.4. Reconocimiento de voz

Para evaluar el reconocimiento de voz se escribieron diez consultas típicas del asistente («¿Qué objetos ves en esta imagen?», «Enciende la cámara, por favor», «Busca la palabra salida en el cartel», «¿Qué ha cambiado respecto a lo que veías antes?»...) y se tradujeron a los seis idiomas de la interfaz. Cada una se sintetizó con dos voces neuronales por idioma —la que usa la aplicación y otra del sexo opuesto—, lo que da 120 grabaciones de unos 3,1 segundos. Se transcribieron con los modelos *tiny*, *base* y *small* en la configuración del backend (CPU, `int8`, idioma forzado y VAD) en tres condiciones: sin ruido y con ruido blanco añadido con una relación señal/ruido de 10 dB y de 5 dB. El WER se calcula sobre el texto normalizado (minúsculas, sin signos de puntuación). La figura 4.7 y la tabla 4.3 resumen los resultados.

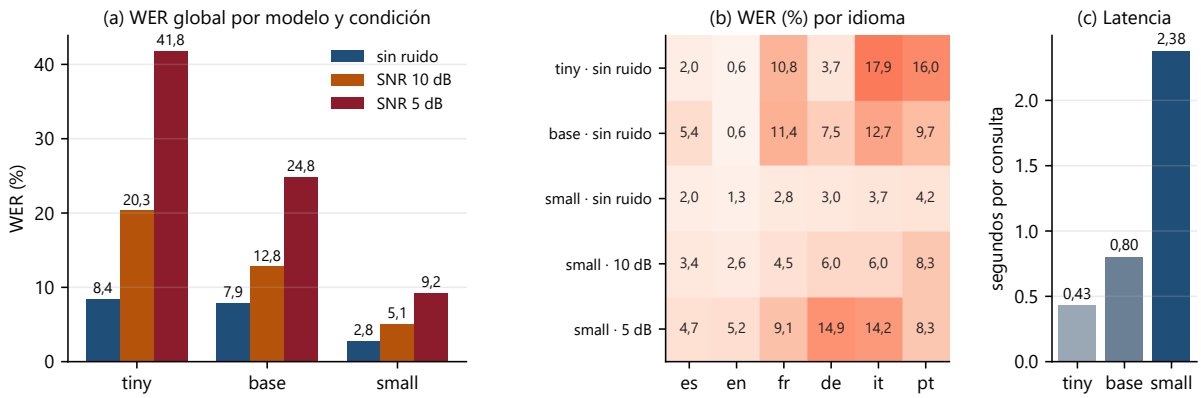


Figura 4.7: Reconocimiento de voz con faster-whisper: (a) WER global por modelo y condición; (b) WER por idioma; (c) latencia media por consulta en CPU.

Los resultados respaldan la decisión tomada en la sesión 11 de sustituir *base* por *small*: el

Modelo	WER sin ruido	WER a 10 dB	WER a 5 dB	Latencia media	Frases perfectas (sin ruido)
tiny	8,4 %	20,3 %	41,8 %	0,43 s	59 %
base	7,9 %	12,8 %	24,8 %	0,80 s	62 %
small (usado)	**2,8 %**	**5,1 %**	**9,2 %**	**2,38 s**	**84 %**

Tabla 4.3: WER global (120 consultas, 890 palabras por condición) y latencia media por consulta de faster-whisper en CPU.

error se divide por 2,8 en condiciones limpias y por 2,7 con ruido fuerte, y la degradación con el ruido es mucho más suave (del 2,8 % al 9,2 %, frente al 7,9 %–24,8 % de *base* y el 8,4 %–41,8 % de *tiny*). El precio es la latencia: 2,4 segundos por consulta en esta CPU, frente a 0,8 de *base*. Como el usuario revisa la transcripción antes de enviarla, se consideró preferible esperar algo más a tener que corregir el texto. Por idiomas, el español y el inglés son los mejor reconocidos, y el alemán y el italiano los más sensibles al ruido, aunque con solo 134–176 palabras por idioma y condición estas diferencias deben interpretarse con cautela: un par de errores más o menos cambian el WER de un idioma en más de un punto.

El análisis cualitativo de los errores del modelo *small* sin ruido es tranquilizador. Buena parte son variantes que no alteran el significado para el LLM que recibe el texto: ortografía británica («colour», «analyse»), elisiones italianas («cos'è» por «cosa è») o palabras unidas («zuvorhin» por «zu vorhin»). Otros son ambigüedades fonéticas genuinas, como «Repite en voz alta» transcrito como «Repiten voz alta», que suena igual. Los errores que sí cambian una palabra («panon» por «panneau») son pocos. Conviene subrayar, en todo caso, la principal limitación del experimento: la voz sintética es clara, sin acento marcado ni reverberación, y el ruido blanco es solo una aproximación al ruido real. Con usuarios reales y micrófonos domésticos cabe esperar errores mayores, y una evaluación con locutores reales es uno de los trabajos futuros propuestos.

4.5. Síntesis de voz

El tiempo de síntesis se midió en las mismas condiciones que el *endpoint*: desde que se invoca Edge-TTS hasta que el MP3 completo está guardado. Para las 120 consultas cortas (unos 3 segundos de audio), la mediana es de 0,8 segundos, con extremos de 0,5 y 1,7 segundos; el portugués y el italiano son algo más lentos que el resto (figura 4.8a). La longitud del texto sí influye (figura 4.8b): la primera síntesis de una respuesta de 32 palabras tardó 1,8 segundos, y las de 64 y 128 palabras, unos 3 segundos, para 22 y 44 segundos de audio respectivamente. Al repetir exactamente el mismo texto, la síntesis bajó a unos 0,7 segundos con independencia de la longitud, lo que sugiere una caché en el

servicio; como las respuestas del LLM son siempre distintas, la cifra relevante es la de la primera síntesis.

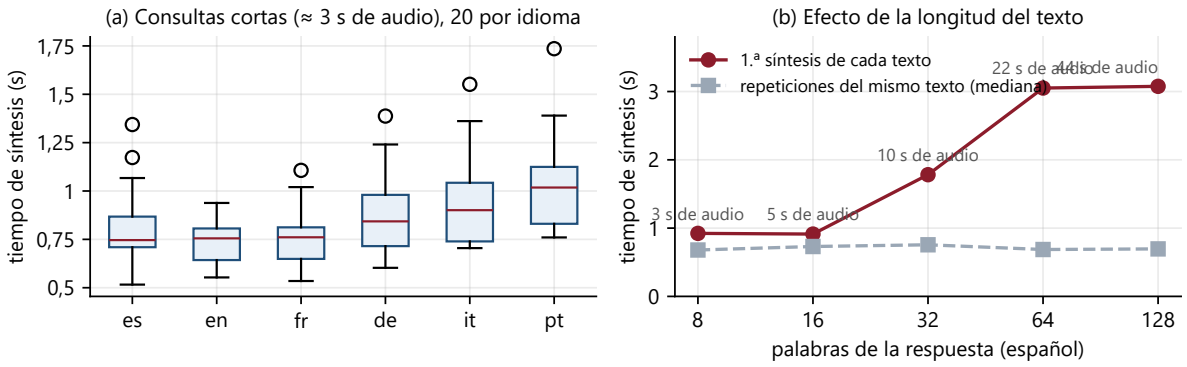


Figura 4.8: Tiempo de síntesis con Edge-TTS: (a) consultas cortas por idioma; (b) efecto de la longitud de la respuesta en español.

Para una respuesta típica del asistente (de 40 a 80 palabras), el usuario espera, por tanto, entre 2 y 3 segundos desde que pulsa el altavoz hasta que empieza a oír la respuesta. La causa es que el backend espera a tener el fichero completo antes de devolverlo. Edge-TTS entrega el audio por fragmentos a medida que lo genera, así que transmitirlos al navegador en *streaming* permitiría empezar a reproducir en menos de un segundo, una mejora sencilla de alto impacto en la sensación de fluidez.

4.6. Búsqueda de texto con OCR

4.6.1. Carteles y etiquetas

Para evaluar la herramienta `buscar_texto_en_imagen` se generaron imágenes de 640×480 con una palabra objetivo —veinte palabras típicas de carteles, como «SALIDA», «Farmacia» o «Información»— y un texto distractor, variando la tipografía (Arial, Times, Calibri, Consolas y Segoe Script, esta última de estilo manuscrito), el tamaño (16, 32 y 64 píxeles), la rotación (15°, 30° y 45°) y el fondo (liso o fotográfico). En cada imagen se ejecutó la misma lógica de búsqueda de la herramienta. Los resultados son muy buenos (figura 4.9d): se encuentra el 98,3 % de las palabras en las 300 combinaciones de tipografía y tamaño, el 100 % sobre fondo fotográfico y con rotaciones de hasta 30°, y el 95 % con 45°. Los pocos fallos se concentran en la tipografía Times a 16 píxeles, cuyos trazos finos se degradan a baja resolución. En todos los aciertos, la caja devuelta se superpone correctamente con la palabra.

La latencia media es de 1,1 segundos por imagen de 640×480 en CPU. Hay que sumar, en la primera búsqueda, la carga diferida del lector, que tarda unos 4,7 segundos (2,2 si los ficheros ya están en la caché del sistema operativo), y, cuando la imagen es grande, un

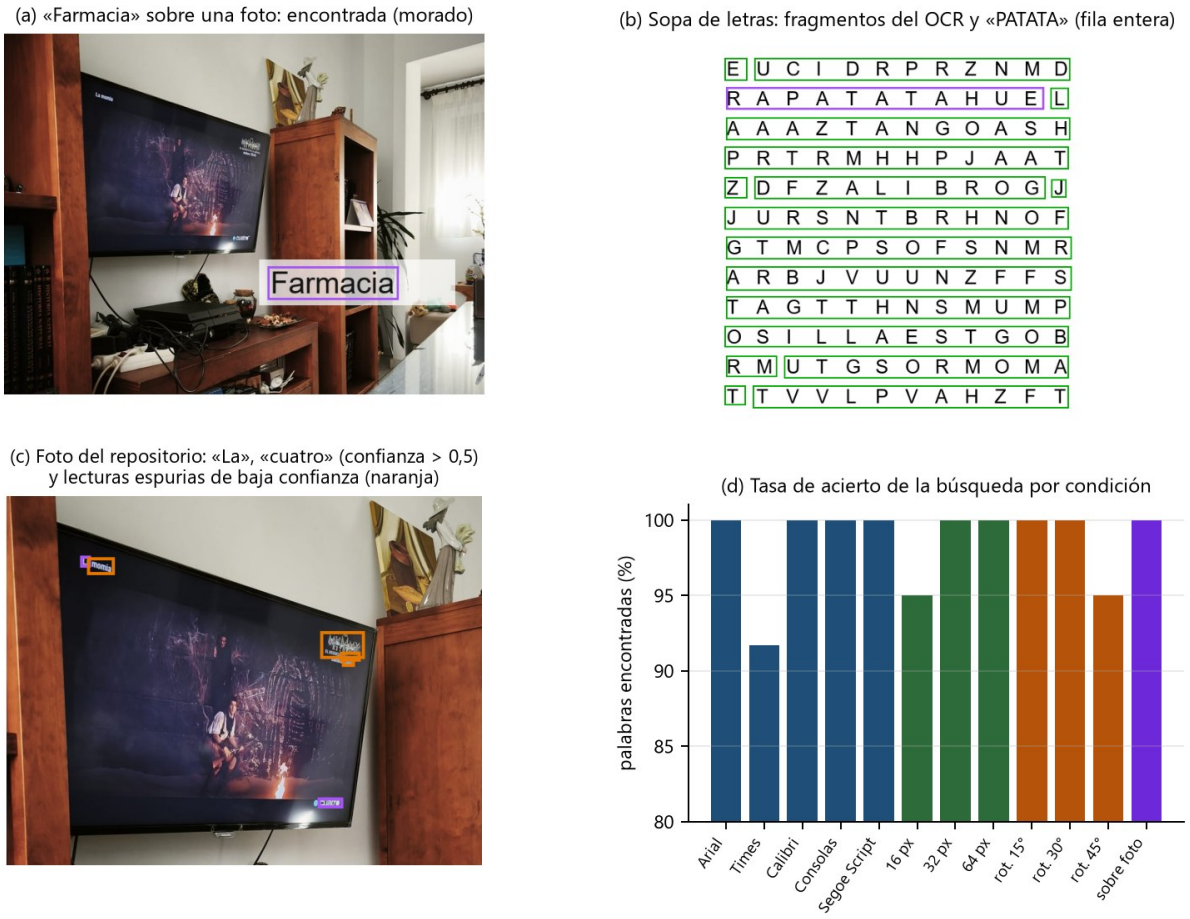


Figura 4.9: Evaluación de la búsqueda de texto: (a) palabra encontrada sobre una foto; (b) sopa de letras con los fragmentos que devuelve el OCR (verde) y el único acierto, cuya caja abarca la fila completa (morado); (c) lecturas sobre la fotografía del repositorio; (d) tasa de acierto por tipografía, tamaño, rotación y fondo.

coste muy superior: sobre la fotografía de 1536×2048 del repositorio el OCR tarda 12,8 segundos, porque la herramienta procesa la imagen a su resolución original. Redimensionar la imagen antes del OCR, como ya se hace antes de enviar los fotogramas al LLM, reduciría drásticamente este tiempo.

4.6.2. El caso de la sopa de letras

En la sesión 15 se detectó que el OCR fallaba en textos muy espaciados, como las sopas de letras. El experimento reproduce ese caso con sopas de 12×12 letras que esconden seis palabras (tres horizontales, dos verticales y una diagonal), con tres espaciados distintos. El resultado es contundente: con el espaciado más estrecho no se encuentra ninguna palabra, y con los otros dos solo se encuentra «PATATA». Tres causas se combinan. Primero, la **fragmentación**: CRAFT agrupa las letras en fragmentos que no coinciden con las filas, y una palabra partida entre dos fragmentos no se encuentra, aunque se ignoren los espacios. Segundo, las **confusiones de caracteres sin contexto**: sin palabras reconocibles, el

reconocedor no puede apoyarse en el contexto y confunde la O con el cero, la I con una barra y la S con el símbolo del dólar, de modo que «LIBRO» se lee «L/BR0» y «SILLA», «S/LLA». Tercero, la **geometría**: el OCR lee líneas horizontales, así que las palabras verticales y diagonales nunca aparecen como subcadenas. Además, cuando «PATATA» se encuentra, la caja devuelta abarca la fila entera (figura 4.9b), por lo que la flecha señala el centro de la fila y no la palabra.

Es un buen ejemplo de un límite estructural y no de un error de programación: el OCR clásico reconoce texto *escrito para ser leído*, en líneas y con palabras, y no resuelve un problema de búsqueda espacial. Un VLM razona mejor sobre la cuadrícula, pero no da coordenadas fiables. Una solución robusta combinaría ambos: localizar cada letra con el OCR a nivel de carácter y buscar las palabras en la matriz resultante en todas las direcciones con un algoritmo clásico.

4.6.3. Texto en una fotografía real

Sobre la fotografía del repositorio (figura 4.9c), el OCR encuentra con confianza alta «La» (86 %), del título de la película que se ve en el televisor, y «cuatro» (64 %), el logotipo de la cadena. «momia» se lee con solo un 27 % de confianza, y aparecen cuatro lecturas espurias («mx», «1174g»...) con confianzas inferiores al 20 %. La herramienta acepta la primera coincidencia sin mirar la confianza, así que una mejora inmediata sería ignorar las lecturas por debajo de un umbral.

4.7. El agente y el sistema completo

4.7.1. Llamadas al modelo y coste por consulta

La flexibilidad del agente tiene un precio que puede calcularse con exactitud a partir de los *prompts* reales. Tokenizando con el tokenizador de GPT-4o lo que el sistema envía en cada llamada, y aplicando la ecuación 2.12 a las imágenes, se obtiene la figura 4.10 y la tabla 4.4. Las cifras de coste usan los precios públicos de GPT-4o mini (0,15 dólares por millón de tokens de entrada y 0,60 por millón de salida) y suponen unos 120 tokens de salida por llamada.

De estas cifras se extraen tres conclusiones. La primera es que **las imágenes dominan el coste**: un fotograma supone 14 167 tokens, mientras que el *prompt* de sistema de smolagents y los esquemas de las herramientas, que se repiten en cada paso del agente, suman unos 2 300. El agente encarece una consulta visual en torno a un 40 %, no varias veces. La segunda es que el coste absoluto es bajo: incluso las consultas más caras cuestan menos de 4 dólares por cada mil. La tercera, menos evidente, es que el modo Imagen

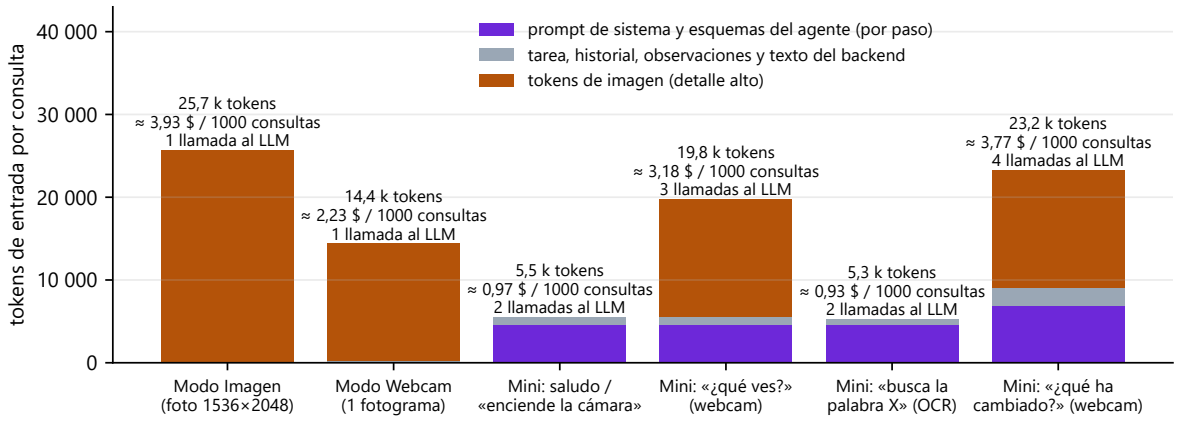


Figura 4.10: Estimación de los tokens de entrada, el número de llamadas al LLM y el coste por cada 1000 consultas según el tipo de consulta, calculados a partir de los *prompts* reales del sistema (imágenes en modo de detalle alto).

Consulta	**Llamadas al LLM**	**Tokens de entrada**	**Coste por 1000 consultas**
Modo Imagen, foto de 1536×2048	1	25 705	3,93 $
Modo Webcam, un fotograma	1	14 371	2,23 $
Mini: saludo o «enciende la cámara»	2	5 529	0,97 $
Mini: «¿qué ves?» con la webcam	3	19 786	3,18 $
Mini: «busca la palabra X» (OCR)	2	5 253	0,93 $
Mini: «¿qué ha cambiado?» con la webcam	4	23 219	3,77 $

Tabla 4.4: Llamadas secuenciales al LLM, tokens de entrada y coste estimado por tipo de consulta.

resulta el más caro, porque `/api/analyze` envía la foto a su resolución original en lugar de reducirla a 640×480 como hacen los demás *endpoints*: redimensionarla ahorraría un 44 % de los tokens sin pérdida apreciable para el LLM, que de todos modos trabaja con la imagen reescalada.

Donde el agente sí penaliza de forma significativa es en la **latencia**. Una consulta en los modos clásicos implica una sola llamada al modelo; la misma consulta en el modo Mini implica tres llamadas *secuenciales* (la decisión del agente, el análisis del backend con la imagen y la respuesta final), y una comparación con el historial, cuatro. Cada llamada añade el tiempo de ida y vuelta a la API y el de generación. La tabla 4.5 reúne los tiempos medidos de los componentes locales: todos son del orden de milisegundos salvo la voz y el OCR, lo que confirma que el tiempo de respuesta percibido está dominado por las llamadas al LLM remoto y por su número.

Etapa	Tiempo	Observaciones
Captura y codificación de un fotograma en el navegador	5 ms	≈ 50 KB en base64
Decodificación + YOLOv8n (fotograma 640×480)	48 ms	≈ 20 fotogramas/s posibles
Análisis de color HSV (incluida la decodificación)	3 ms	
Transcripción de una consulta de ≈ 3 s (Whisper *small*)	2,4 s	*base*: 0,8 s
Síntesis de una respuesta corta / de 64 palabras	0,8 s / 3 s	servicio remoto
Búsqueda OCR (640×480 / foto de 1536×2048)	1,1 s / 12,8 s	+ 4,7 s la primera vez
Llamadas a GPT-4o mini	—	1 (clásico) a 4 (Mini)

Tabla 4.5: Latencias medidas de los componentes locales del sistema (medianas en la CPU de pruebas).

4.7.2. Casos de uso del agente

La tabla 4.6 recorre los casos de uso representativos del modo Mini, indicando para cada uno la plantilla de tarea que se aplica, el recorrido de herramientas que prescribe el diseño, la acción resultante y las observaciones relevantes del desarrollo o del análisis de este capítulo.

Consulta y contexto	Recorrido previsto	Acción	Observaciones
«Hola, ¿qué tal?» (sin imagen)	`responder_directamente` → `final_answer`	—	Dos llamadas al LLM para un saludo: el coste fijo del agente es proporcionalmente alto en las consultas triviales.
«Enciende la cámara»	`activar_camara` → `final_answer`	`ACTIVATE_WEBCAM`	La interfaz cambia de modo y enciende la cámara sin intervención del usuario.
«Quiero subir una foto»	`adjuntar_documento` → `final_answer`	`OPEN_FILE_PICKER`	Riesgo de bloqueo del selector si la respuesta supera la ventana de activación del navegador (apartado 3.7.6).
«¿Qué ves?» (webcam)	`analiza_frame_webcam` → `final_answer`	—	Tres llamadas secuenciales; el agente reformula la respuesta del backend.
«¿Dónde está el televisor?» (webcam)	`analiza_frame_webcam` → `final_answer` con `[POINT_TO]`	`POINT_TO`	Se señalan todas las detecciones, no solo el televisor (figura 3.16).
«¿Qué colores predominan?» (imagen)	`analiza_color_en_imagen` → `final_answer`	—	Respuesta cuantitativa, sujeta a las limitaciones de la paleta (sin marrón).
«Busca la palabra Farmacia» (imagen o webcam)	`buscar_texto_en_imagen` → `final_answer` con `[POINT_TO]`	`POINT_TO`	Localización exacta en carteles; falla en sopas de letras (apartado 4.6).
«¿Qué ha cambiado?» (webcam, modo conversación)	`analiza_frame_webcam` → `responder_directamente` → `final_answer`	—	Cuatro llamadas; la comparación se hace con el historial textual (el agente no conserva imágenes).

Consulta y contexto	Recorrido previsto	Acción	Observaciones
«¿Qué te pregunté antes?» (modo conversación)	`responder_directamente` con el historial	—	La plantilla delimita explícitamente el historial y define cuál es el último mensaje del usuario para no confundirlo con la pregunta actual.
«Repite tu última respuesta»	`responder_directamente("[SPEAK_LAST]")`	`SPEAK_LAST`	La plantilla informa al modelo de que sí puede «hablar», porque la acción la ejecuta el navegador, para evitar que se niegue.

Tabla 4.6: Casos de uso del agente Mini: recorrido previsto por el diseño y observaciones.

4.7.3. Casos reales documentados durante el desarrollo

Las dos capturas del desarrollo incluidas en el capítulo 3 son ejecuciones reales del sistema completo y merecen un análisis crítico. En la de la fase 0 (figura 3.5), ante la petición implícita de describir un salón, YOLO aporta dos detecciones correctas y de alta confianza (silla, 90 %; televisor, 84 %), y GPT-4o mini produce una descripción rica y correcta que incluye elementos fuera del vocabulario de COCO, como la lámpara de pie, las molduras de la pared, las cortinas o la escultura. Es el comportamiento ideal del sistema híbrido: el detector ancla lo que puede y el modelo de lenguaje completa. En la captura de la fase 2 (figura 3.11), el usuario saluda con «hola» en el modo conversación con la webcam activa; YOLO detecta una persona (73 %) y una «cama» con una confianza de 0,53, apenas por encima del umbral, que no puede verificarse en la miniatura. El modelo responde correctamente al saludo sin describir la escena, lo que muestra que distingue una interacción social de una petición de análisis. El caso ilustra, de paso, que las detecciones próximas al umbral son las más sospechosas, como confirma la tabla 4.2.

4.8. Discusión: criterios de éxito de la propuesta

La propuesta del TFG fijaba unos criterios de éxito para cada fase [95]. La tabla 4.7 los contrasta con los resultados de este capítulo.

En conjunto, los componentes de percepción funcionan dentro de lo esperable para modelos ligeros en CPU, y la evaluación ha permitido cuantificar limitaciones que durante el desarrollo solo se intuían —la fragilidad del detector fuera del dominio fotográfico, la ausencia del marrón, la fragmentación del OCR— y descubrir otras nuevas, como el envío de fotos a resolución completa en el modo Imagen o el señalamiento indiscriminado de la acción `POINT_TO`. Estos resultados alimentan directamente las conclusiones y las líneas de trabajo futuro del capítulo siguiente.

Fase	Criterio	Resultado	Evidencia
0	3 o más tipos de preguntas respondidas correctamente	Cumplido	Descripción, recuento, localización, color y lectura de texto (capítulo 3 y casos reales).
0	Precisión de detección superior al 60 %	Cumplido	Precisión del 87,1 % con el umbral del sistema; mAP_{50} de 0,546 en COCO val2017.
0	Demo funcional y código documentado	Cumplido	Versión v1.0.0, README, guía de contribución, imágenes Docker.
1	Vídeo procesado a más de 5 FPS	Parcial	YOLO alcanza unos 20 FPS en CPU, pero el sistema analiza un fotograma cada 10 s por los límites de tokens de la API.
1	Latencia inferior a 5 s por consulta	No medido de extremo a extremo	Componentes locales por debajo de 50 ms en visión; la latencia depende de 1–4 llamadas al LLM remoto.
1	Overlays actualizándose correctamente	Cumplido	Cajas y flechas alineadas con el vídeo; se refrescan con cada análisis.
2	STT con WER inferior al 20 %	Cumplido (voz sintética)	2,8 % sin ruido y 9,2 % a 5 dB con Whisper *small*.
2	Conversación fluida por voz	Cumplido con matices	Voz en ambos sentidos y memoria; el micrófono es de pulsar para hablar y la síntesis se activa por botón.
2	Dos servicios funcionando de forma independiente	Cumplido	Backend de percepción y servicio del agente en contenedores separados.

Tabla 4.7: Cumplimiento de los criterios de éxito fijados en la propuesta para cada fase.

Capítulo 5

Conclusiones

Este último capítulo valora el trabajo realizado: qué objetivos se han cumplido y en qué medida, qué ha funcionado bien, qué no ha funcionado o se ha quedado corto y cómo podría mejorarse, qué ampliaciones son posibles y por qué no se han abordado, y, por último, una valoración personal de la experiencia.

5.1. Grado de cumplimiento de los objetivos

La tabla 5.1 resume el grado de cumplimiento de los objetivos específicos del apartado 1.3. El objetivo general —un sistema que permita diálogos naturales por voz sobre contenido visual, orquestado por agentes LLM— se ha alcanzado: Mini conversa por voz y texto en seis idiomas sobre imágenes y vídeo en directo, decide por sí mismo qué herramientas usar y actúa sobre la interfaz. Las fases 0, 1 y 2 de la estrategia incremental están completas y publicadas como la versión 1.2.0 del proyecto, con imágenes Docker listas para desplegar. Los objetivos que quedan parcialmente cubiertos lo están, en su mayoría, porque corresponden a la fase 3.

5.2. Lo que ha funcionado

El enfoque híbrido. La decisión más importante del trabajo, combinar un modelo multimodal general con modelos especializados, se ha revelado acertada y los experimentos la respaldan. El detector aporta lo que el LLM no da de forma fiable —recuentos y coordenadas precisas, con un 87 % de precisión al umbral elegido— y el LLM cubre lo que el detector no ve: objetos fuera de sus 80 clases, ilustraciones (donde el recall de YOLO cae hasta el 2,6 %), texto y matices de la escena. Los casos reales de la fase 0, en los que YOLO detecta dos objetos y el LLM describe una decena, lo ilustran mejor que cualquier cifra.

Objetivo	Grado	Comentario
O1. Captura y procesamiento de vídeo en tiempo real desde múltiples fuentes	Parcial	Webcam del navegador con análisis periódico y superposiciones en directo. No se han integrado dispositivos de realidad virtual ni cámaras IP.
O2. Agente LLM que interpreta consultas y orquesta modelos de visión	Cumplido	Agente Mini (smolagents) con siete herramientas, plantillas de tarea por contexto, memoria y protocolo de acciones.
O3. Integración de modelos preentrenados para tareas específicas	Parcial	Detección de objetos (YOLOv8), color (HSV), texto (EasyOCR) y visión general (GPT-4o mini). Sin segmentación ni estimación de pose.
O4. Entrada y salida de voz	Cumplido	faster-whisper *small* con VAD (WER del 2,8 % en voz sintética) y Edge-TTS con una voz nativa por idioma.
O5. Realimentación visual sobre el vídeo	Cumplido con matices	Cajas con etiquetas y flechas de señalización sobre imagen y vídeo. La flecha señala objetos, pero no da indicaciones de dirección para la navegación.
O6. Interfaz web interactiva	Cumplido	Tres modos, modo conversación, seis idiomas, temas claro y oscuro, control de volumen, atajos de teclado y guía de uso integrada.
O7. Evaluación de latencia, precisión y experiencia de usuario	Parcial	Evaluación cuantitativa de precisión y latencia de todos los componentes locales y análisis de coste del agente. Sin medición extremo a extremo con el LLM ni estudio con usuarios.

Tabla 5.1: Grado de cumplimiento de los objetivos específicos del trabajo.

Un agente que actúa, no solo responde. El protocolo de acciones mediante etiquetas es una solución deliberadamente sencilla —no requiere más que buscar una cadena en la respuesta— que convierte al asistente en algo más que un chat: Mini enciende la cámara, abre el selector de archivos, lee en voz alta y señala objetos o palabras. Y el patrón de herramientas ha demostrado ser muy extensible: el OCR se añadió en una sola sesión, reutilizando incluso el dibujo de las detecciones gracias a un formato común.

La voz. El canal de voz funciona en ambos sentidos y en seis idiomas con una calidad notable para modelos que se ejecutan en CPU o de forma gratuita: el cambio de Whisper *base* a *small*, validado experimentalmente, divide el error por casi tres, y la decisión de que la transcripción sea editable antes de enviarse evita que un error de reconocimiento desencadene una consulta equivocada.

La ingeniería del software. La metodología incremental por fases ha permitido tener siempre un sistema funcionando, con dos versiones publicadas; el diario de sesiones ha sido una herramienta valiosa tanto para planificar como para escribir esta memoria; la estrategia de ramas y la convención de mensajes han mantenido el repositorio ordenado; y la contenerización permite desplegar el sistema completo con un solo comando y sin

instalar nada más que Docker. La internacionalización de extremo a extremo —el idioma elegido condiciona la interfaz, la respuesta del LLM, la traducción de clases y colores, la voz y el reconocimiento— es otro de los aspectos más cuidados del sistema.

5.3. Lo que no ha funcionado o se ha quedado corto

La evaluación crítica de los capítulos 3 y 4 ha puesto de manifiesto varias limitaciones. Para cada una se indica cómo podría resolverse.

- **El «tiempo real» es de un fotograma cada diez segundos.** El detector podría analizar unos 20 fotogramas por segundo, pero cada consulta al LLM con imagen cuesta más de 14 000 tokens, y los límites de la API obligaron a espaciar los análisis. Soluciones: fijar explícitamente el modo de detalle bajo de la API (2833 tokens por imagen en lugar de 14 167), ejecutar YOLO de forma continua y llamar al LLM solo cuando la escena cambie de forma significativa (detectable comparando fotogramas o sus detecciones) o cuando el usuario pregunte, y sustituir el *polling* por WebSockets para que el servidor envíe los resultados en cuanto estén disponibles.

- **El agente añade latencia.** Una consulta visual en el modo Mini requiere tres llamadas secuenciales al modelo y una comparación, cuatro. Soluciones: resolver las intenciones triviales (saludos, encender la cámara) sin pasar por el agente, con reglas o un clasificador ligero; permitir que el agente combine varias herramientas en un solo paso (el `CodeAgent` de smolagents lo hace); transmitir las respuestas en *streaming*; y medir sistemáticamente la latencia extremo a extremo para optimizar con datos.

- **La acción `POINT_TO` no sabe a qué señalar.** Se dibuja una flecha sobre todas las detecciones de la imagen. Solución: que la etiqueta lleve un argumento con la clase o el texto buscado y filtrar las detecciones en consecuencia.

- **El análisis de color es global e incompleto.** No distingue el color de un objeto del de la escena, no tiene la categoría marrón y es sensible a la iluminación (80,7 % de exactitud global y solo un 31 % en los rosas). Solución: analizar el color dentro de la caja del objeto, trabajar en CIELAB con agrupamiento de píxeles y completar la paleta con los once términos básicos.

- **El OCR falla fuera del texto «normal».** Funciona muy bien en carteles y etiquetas (98 % de acierto), pero no en sopas de letras, acepta lecturas de confianza muy baja y tarda casi 13 segundos en fotos grandes. Soluciones: redimensionar antes del OCR, filtrar por confianza y, para búsquedas espaciales, trabajar a nivel de carácter.

- **Estado compartido y concurrencia.** Las herramientas del agente comparten el fotograma y las detecciones mediante variables globales, lo que no es seguro con varios usuarios simultáneos, y las inferencias de YOLO bloquean brevemente el bucle de eventos. Soluciones: contexto por petición (`contextvars`) y ejecución de la inferencia en hilos auxiliares o en un servicio de inferencia dedicado.

- **Seguridad.** La defensa frente a la inyección de *prompts* solo se aplica completa en el modo conversación, el historial viaja sin protección en el cliente y no hay autenticación ni HTTPS. Para un despliegue real habría que completar las medidas de la tabla 3.6.

- **Ausencia de pruebas automáticas.** La fase de pruebas planificada al cerrar la fase 2 no llegó a realizarse. Es la carencia de ingeniería más evidente del proyecto: faltan pruebas unitarias de los *endpoints* (sustituyendo el LLM por dobles de prueba), pruebas extremo a extremo de la interfaz —las propias capturas de esta memoria se obtuvieron automatizando la aplicación con Playwright y una cámara simulada, lo que demuestra que son viables— y pruebas de carga.

- **Detalles de eficiencia.** El modo Imagen envía las fotos a resolución completa al LLM (un 44 % más de tokens de los necesarios), la síntesis de voz espera al fichero completo en lugar de reproducirse en *streaming*, y el frontend se despliega con el servidor de desarrollo de Vite. Son mejoras sencillas de alto impacto.

5.4. Ampliaciones y trabajo futuro

Más allá de las mejoras anteriores, que corrigen limitaciones del sistema actual, existen ampliaciones que extenderían sus capacidades. La mayoría estaban previstas en la fase 3 de la propuesta y no se han abordado por una razón común: cada una añade un modelo, un servicio o un protocolo nuevos, y su integración rigurosa —con su evaluación— requería un tiempo del que no se disponía una vez completadas las fases 0 a 2 con el nivel de calidad buscado. Siguiendo la regla de oro de la propuesta, se prefirió entregar la fase 2 completa y pulida antes que una fase 3 a medias.

- **Detección de vocabulario abierto.** Detectores como YOLO-World [17] o Grounding DINO [55] localizan objetos descritos en lenguaje natural («mi taza verde») en lugar de limitarse a 80 clases. Integrarlos permitiría que el agente buscase exactamente lo que pide el usuario. Alternativamente, modelos multimodales entrenados para devolver coordenadas, como Qwen2.5-VL [6], podrían unificar la descripción y la localización.

- **Segmentación y búsqueda visual.** SAM [45] permitiría recortar con precisión el objeto señalado —y, por ejemplo, calcular su color real—, y CLIP [82] buscar en la imagen por descripción.

- **Seguimiento y profundidad.** Un seguidor como ByteTrack [127] mantendría las cajas alineadas con los objetos entre análisis, y un estimador de profundidad monocular como MiDaS [84] permitiría responder a «¿está cerca?» o dar indicaciones de dirección («gira a la izquierda»), lo que completaría el objetivo O5 y acercaría el sistema a la navegación asistida. La estimación de pose con MediaPipe [57] añadiría preguntas sobre lo que hacen las personas.

- **Arquitectura completa de la fase 3.** Separar un servicio de audio y otro de visión, añadir Redis como caché de resultados y almacén de sesiones, y usar WebSockets para la comunicación en tiempo real.

- **Modelos locales y privacidad.** Sustituir los servicios externos por modelos locales —un VLM abierto servido con Ollama o vLLM, Whisper en GPU y una síntesis de voz local como Piper— eliminaría la dependencia de Internet y el envío de imágenes y textos a terceros, algo especialmente relevante para usuarios con discapacidad visual que usarían el sistema en su intimidad. Requiere, eso sí, una GPU más potente que la disponible.

- **Voz en tiempo real.** Las API de voz a voz, como la Realtime API de OpenAI [73], permiten conversaciones con interrupciones y latencias de cientos de milisegundos, sustituyendo la canalización STT $\rightarrow$ LLM $\rightarrow$ TTS.

- **Otras fuentes de vídeo.** Cámaras IP mediante RTSP y gafas de realidad virtual o aumentada, como proponía el objetivo O1, en las que la interacción por voz sobre lo que se ve es especialmente natural.

- **Memoria a largo plazo.** Una base de datos vectorial permitiría recordar interacciones de días anteriores («¿dónde dejé ayer las llaves?»).

- **Evaluación con usuarios.** Un estudio de usabilidad con el cuestionario SUS [13], idealmente con personas con discapacidad visual, y una evaluación sistemática del agente sobre una batería de consultas con varias repeticiones, midiendo la tasa de acierto en la elección de herramientas y la latencia extremo a extremo.

5.5. Valoración personal

Cuando empecé este trabajo, la propuesta me pareció enorme: microservicios, agentes, visión, voz, tiempo real. La estrategia por fases que planteó mi tutor fue lo que lo hizo

abordable, y creo que es la lección más valiosa que me llevo: empezar por algo pequeño que funcione de verdad y hacerlo crecer. Cada fase me obligó a entender una capa nueva y, casi siempre, el problema difícil no era el que esperaba. Pensaba que lo complicado sería integrar YOLO o Whisper y resultó que era entender por qué la API rechazaba mis peticiones con solo veinte por minuto, o por qué el primer fotograma de la webcam no servía.

También he aprendido que trabajar con modelos de lenguaje es una forma distinta de programar. Muchos de los errores del agente no se arreglaban cambiando código, sino explicándole mejor al modelo lo que tenía que hacer, y eso exige una mezcla curiosa de precisión técnica y redacción. Al mismo tiempo, escribir esta memoria me ha obligado a mirar el sistema con ojos críticos y a medir lo que antes solo intuía: descubrir que el detector encuentra solo un tercio de los objetos, o que el análisis de color no conoce el marrón, ha sido tan formativo como construirlo.

Me quedo con la satisfacción de haber construido un sistema completo, de la cámara del navegador al contenedor Docker, que cualquiera puede desplegar y usar, y con muchas ganas de seguir explorando la dirección que apunta: asistentes que no solo conversan, sino que ven, escuchan y actúan en el mundo de quien los usa.

Bibliografía

[1] Alayrac, J.B., et al.: Flamingo: A visual language model for few-shot learning. In: Advances in NeurIPS, pp. 23716–23736 (2022)

[2] Anderson, P., et al.: Bottom-up and top-down attention for image captioning and visual question answering. In: Proc. IEEE CVPR, pp. 6077–6086 (2018)

[3] Antol, S., et al.: VQA: Visual question answering. In: Proc. IEEE ICCV, pp. 2425–2433 (2015)

[4] Baek, Y., Lee, B., Han, D., Yun, S., Lee, H.: Character region awareness for text detection. In: Proc. IEEE/CVF CVPR, pp. 9365–9374 (2019)

[5] Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate. In: Proc. Int. Conf. Learning Representations (ICLR) (2015)

[6] Bai, S., et al.: Qwen2.5-VL technical report. arXiv:2502.13923 (2025)

[7] Be My Eyes: Introducing Be My AI (formerly Virtual Volunteer) for people who are blind or have low vision, powered by OpenAI's GPT-4 (2023). URL https://www.bemyeyes.com/blog/introducing-be-my-ai

[8] Bengio, Y., Ducharme, R., Vincent, P., Jauvin, C.: A neural probabilistic language model. Journal of Machine Learning Research **3**, 1137–1155 (2003)

[9] Berlin, B., Kay, P.: Basic Color Terms: Their Universality and Evolution. University of California Press, Berkeley, CA, EE. UU. (1969)

[10] Bierman, G., Abadi, M., Torgersen, M.: Understanding TypeScript. In: Proc. European Conf. Object-Oriented Programming (ECOOP), pp. 257–281 (2014)

[11] Bochkovskiy, A., Wang, C.Y., Liao, H.Y.M.: YOLOv4: Optimal speed and accuracy of object detection. arXiv:2004.10934 (2020)

[12] Bradski, G.: The OpenCV library. Dr. Dobb's Journal of Software Tools (2000)

[13] Brooke, J.: SUS: A “quick and dirty” usability scale. In: P.W. Jordan, et al. (eds.) Usability Evaluation in Industry, pp. 189–194. Taylor & Francis, Londres, Reino Unido (1996)

[14] Brown, T.B., et al.: Language models are few-shot learners. In: Advances in NeurIPS, pp. 1877–1901 (2020)

[15] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-end object detection with transformers. In: Proc. ECCV, pp. 213–229 (2020)

[16] Chase, H.: LangChain. Software (2022). URL https://github.com/langchain-ai/langchain

[17] Cheng, T., Song, L., Ge, Y., Liu, W., Wang, X., Shan, Y.: YOLO-World: Real-time open-vocabulary object detection. In: Proc. IEEE/CVF CVPR, pp. 16901–16911 (2024)

[18] Conventional Commits: Conventional Commits 1.0.0. Especificación (2019). URL https://www.conventionalcommits.org/es/v1.0.0/

[19] Coqui AI: Coqui TTS: a deep learning toolkit for text-to-speech. Software (2021–2024). URL https://github.com/coqui-ai/TTS

[20] Dalal, N., Triggs, B.: Histograms of oriented gradients for human detection. In: Proc. IEEE CVPR, pp. 886–893 (2005)

[21] Davis, S., Mermelstein, P.: Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. IEEE Trans. Acoustics, Speech, and Signal Processing **28**(4), 357–366 (1980)

[22] Django Software Foundation: ASGI (Asynchronous Server Gateway Interface) specification (2019). URL https://asgi.readthedocs.io

[23] Dosovitskiy, A., et al.: An image is worth 16x16 words: Transformers for image recognition at scale. In: Proc. ICLR (2021)

[24] Driessen, V.: A successful Git branching model (2010). URL https://nvie.com/posts/a-successful-git-branching-model/

[25] Egido Carnerero, J.: multimodal-vision-tfg. Repositorio de código del proyecto, release v1.2.0 (2026). URL https://github.com/josegicar/multimodal-vision-tfg

[26] Everingham, M., Van Gool, L., Williams, C.K.I., Winn, J., Zisserman, A.: The PASCAL Visual Object Classes (VOC) challenge. International Journal of Computer Vision **88**, 303–338 (2010)

[27] Feng, C., Zhong, Y., Gao, Y., Scott, M.R., Huang, W.: TOOD: Task-aligned one-stage object detection. In: Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), pp. 3490–3499 (2021)

[28] Fette, I., Melnikov, A.: The WebSocket protocol. RFC 6455, IETF (2011)

[29] Fielding, R., Nottingham, M., Reschke, J.: HTTP semantics. RFC 9110, IETF (2022)

[30] Fielding, R.T.: Architectural styles and the design of network-based software architectures. Tesis doctoral, University of California, Irvine (2000)

[31] Girshick, R., Donahue, J., Darrell, T., Malik, J.: Rich feature hierarchies for accurate object detection and semantic segmentation. In: Proc. IEEE CVPR, pp. 580–587 (2014)

[32] Gonzalez, R.C., Woods, R.E.: Digital Image Processing, 4th edn. Pearson, Nueva York, NY, EE. UU. (2018)

[33] Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, Cambridge, MA, EE. UU. (2016)

[34] Graves, A., Fernández, S., Gomez, F., Schmidhuber, J.: Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In: Proc. Int. Conf. Machine Learning (ICML), pp. 369–376 (2006)

[35] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: Proc. ACM Workshop on Artificial Intelligence and Security (AISec), pp. 79–90 (2023)

[36] Gupta, T., Kembhavi, A.: Visual programming: Compositional visual reasoning without training. In: Proc. IEEE/CVF CVPR, pp. 14953–14962 (2023)

[37] Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997)

[38] Jacob, B., et al.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: Proc. IEEE CVPR, pp. 2704–2713 (2018)

[39] Jaided AI: EasyOCR: Ready-to-use OCR with 80+ supported languages. Software (2020–2025). URL https://github.com/JaidedAI/EasyOCR

[40] Ji, Z., et al.: Survey of hallucination in natural language generation. ACM Computing Surveys **55**(12), 1–38 (2023)

[41] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. Software, v8.0.0 y posteriores (2023). URL https://github.com/ultralytics/ultralytics

[42] Josefsson, S.: The Base16, Base32, and Base64 data encodings. RFC 4648, IETF (2006)

[43] Jurafsky, D., Martin, J.H.: Speech and Language Processing. 3.ª ed. (borrador en línea) (2025). URL https://web.stanford.edu/~jurafsky/slp3/

[44] Kim, J., Kong, J., Son, J.: Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In: Proc. ICML, pp. 5530–5540 (2021)

[45] Kirillov, A., et al.: Segment Anything. In: Proc. IEEE/CVF ICCV, pp. 4015–4026 (2023)

[46] Krizhevsky, A., Sutskever, I., Hinton, G.E.: ImageNet classification with deep convolutional neural networks. In: Advances in Neural Information Processing Systems (NeurIPS), pp. 1097–1105 (2012)

[47] LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE **86**(11), 2278–2324 (1998)

[48] Levenshtein, V.I.: Binary codes capable of correcting deletions, insertions, and reversals. Soviet Physics Doklady **10**(8), 707–710 (1966)

[49] Li, J., Li, D., Savarese, S., Hoi, S.: BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In: Proc. ICML, pp. 19730–19742 (2023)

[50] Li, X., et al.: Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In: Advances in NeurIPS, pp. 21002–21012 (2020)

[51] Lin, T.Y., Dollár, P., Girshick, R., He, K., Hariharan, B., Belongie, S.: Feature pyramid networks for object detection. In: Proc. IEEE CVPR, pp. 2117–2125 (2017)

[52] Lin, T.Y., et al.: Microsoft COCO: Common objects in context. In: Proc. ECCV, pp. 740–755 (2014)

[53] Liu, H., Li, C., Wu, Q., Lee, Y.J.: Visual instruction tuning. In: Advances in NeurIPS (2023)

[54] Liu, S., Qi, L., Qin, H., Shi, J., Jia, J.: Path aggregation network for instance segmentation. In: Proc. IEEE CVPR, pp. 8759–8768 (2018)

[55] Liu, S., et al.: Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In: Proc. ECCV (2024)

[56] Liu, W., et al.: SSD: Single shot multibox detector. In: Proc. European Conf. Computer Vision (ECCV), pp. 21–37 (2016)

[57] Lugaresi, C., et al.: MediaPipe: A framework for building perception pipelines. arXiv:1906.08172 (2019)

[58] Masinter, L.: The "dataÜRL scheme. RFC 2397, IETF (1998)

[59] Masinter, L.: Returning values from forms: multipart/form-data. RFC 7578, IETF (2015)

[60] Merkel, D.: Docker: Lightweight Linux containers for consistent development and deployment. Linux Journal **2014**(239), 2 (2014)

[61] Meta Platforms: React: the library for web and native user interfaces. Documentación (2025). URL https://react.dev

[62] Microsoft: Seeing AI: aplicación de asistencia visual (2017). URL https://www.seeingai.com

[63] Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space. arXiv:1301.3781 (2013)

[64] Munroe, R.: Color survey results. xkcd blog (2010). URL https://blog.xkcd.com/2010/05/03/color-survey-results/

[65] Neubeck, A., Van Gool, L.: Efficient non-maximum suppression. In: Proc. Int. Conf. Pattern Recognition (ICPR), pp. 850–855 (2006)

[66] Newman, S.: Building Microservices: Designing Fine-Grained Systems, 2nd edn. O'Reilly Media, Sebastopol, CA, EE. UU. (2021)

[67] OpenAI: Images and vision. Documentación de la API (cálculo de tokens de imagen). URL https://platform.openai.com/docs/guides/images-vision. Consultada en 2026

[68] OpenAI: Rate limits. Documentación de la API. URL https://platform.openai.com/docs/guides/rate-limits. Consultada en 2026

[69] OpenAI: Function calling and other API updates. Anuncio técnico, 13 de junio (2023). URL https://openai.com/index/function-calling-and-other-api-updates/

[70] OpenAI: GPT-4V(ision) system card. Informe técnico, OpenAI (2023)

[71] OpenAI: GPT-4o mini: advancing cost-efficient intelligence. Anuncio técnico, 18 de julio (2024). URL https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/

[72] OpenAI: GPT-4o system card. arXiv:2410.21276 (2024)

[73] OpenAI: Introducing the Realtime API. Anuncio técnico, 1 de octubre (2024). URL https://openai.com/index/introducing-the-realtime-api/

[74] OpenNMT: CTranslate2: Fast inference engine for Transformer models. Software (2019–2025). URL https://github.com/OpenNMT/CTranslate2

[75] Organización Mundial de la Salud: World Report on Vision. OMS, Ginebra, Suiza (2019)

[76] Ouyang, L., et al.: Training language models to follow instructions with human feedback. In: Advances in NeurIPS, pp. 27730–27744 (2022)

[77] OWASP Foundation: OWASP Top 10 for Large Language Model Applications (2025). URL https://genai.owasp.org/llm-top-10/

[78] Perez, F., Ribeiro, I.: Ignore previous prompt: Attack techniques for language models. In: NeurIPS ML Safety Workshop (2022)

[79] Rabiner, L.R.: A tutorial on hidden Markov models and selected applications in speech recognition. Proceedings of the IEEE **77**(2), 257–286 (1989)

[80] Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C., Sutskever, I.: Robust speech recognition via large-scale weak supervision. In: Proc. ICML, pp. 28492–28518 (2023)

[81] Radford, A., Narasimhan, K., Salimans, T., Sutskever, I.: Improving language understanding by generative pre-training. Informe técnico, OpenAI (2018)

[82] Radford, A., et al.: Learning transferable visual models from natural language supervision. In: Proc. ICML, pp. 8748–8763 (2021)

[83] Ramírez, S.: FastAPI. Software y documentación (2018–2026). URL https://fastapi.tiangolo.com

[84] Ranftl, R., Lasinger, K., Hafner, D., Schindler, K., Koltun, V.: Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. IEEE Trans. Pattern Analysis and Machine Intelligence **44**(3), 1623–1637 (2022)

[85] rany2: edge-tts: Use Microsoft Edge's online text-to-speech service from Python. Software (2021–2025). URL https://github.com/rany2/edge-tts

[86] Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You only look once: Unified, real-time object detection. In: Proc. IEEE CVPR, pp. 779–788 (2016)

[87] Redmon, J., Farhadi, A.: YOLO9000: Better, faster, stronger. In: Proc. IEEE CVPR, pp. 7263–7271 (2017)

[88] Redmon, J., Farhadi, A.: YOLOv3: An incremental improvement. arXiv:1804.02767 (2018)

[89] Ren, S., He, K., Girshick, R., Sun, J.: Faster R-CNN: Towards real-time object detection with region proposal networks. In: Advances in NeurIPS, pp. 91–99 (2015)

[90] Ren, Y., et al.: FastSpeech 2: Fast and high-quality end-to-end text to speech. In: Proc. ICLR (2021)

[91] Ries, E.: The Lean Startup. Crown Business, Nueva York, NY, EE. UU. (2011)

[92] Roucher, A., Villanova del Moral, A., Wolf, T., von Werra, L., Kaunismäki, E.: smolagents: a smol library to build great agentic systems. Software, Hugging Face (2025). URL https://github.com/huggingface/smolagents

[93] Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. Nature **323**, 533–536 (1986)

[94] Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach, 4th edn. Pearson, Hoboken, NJ, EE. UU. (2021)

[95] Sancho Caparrini, F.: Propuesta de Trabajo Fin de Grado: Sistema multimodal de interacción visual conversacional mediante agentes LLM y visión por computador en tiempo real. Documento interno, Dpto. de Ciencias de la Computación e Inteligencia Artificial, Universidad de Sevilla (2026)

[96] Schick, T., et al.: Toolformer: Language models can teach themselves to use tools. In: Advances in NeurIPS (2023)

[97] Sennrich, R., Haddow, B., Birch, A.: Neural machine translation of rare words with subword units. In: Proc. Annual Meeting of the ACL, pp. 1715–1725 (2016)

[98] Shen, J., et al.: Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions. In: Proc. IEEE ICASSP, pp. 4779–4783 (2018)

[99] Shen, Y., et al.: HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In: Advances in NeurIPS (2023)

[100] Shi, B., Bai, X., Yao, C.: An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. IEEE Trans. Pattern Analysis and Machine Intelligence **39**(11), 2298–2304 (2017)

[101] Silero Team: Silero VAD: pre-trained enterprise-grade voice activity detector. Software (2021–2025). URL https://github.com/snakers4/silero-vad

[102] Smith, A.R.: Color gamut transform pairs. ACM SIGGRAPH Computer Graphics **12**(3), 12–19 (1978)

[103] Smith, R.: An overview of the Tesseract OCR engine. In: Proc. Int. Conf. Document Analysis and Recognition (ICDAR), pp. 629–633 (2007)

[104] Stevens, S.S., Volkmann, J., Newman, E.B.: A scale for the measurement of the psychological magnitude pitch. Journal of the Acoustical Society of America **8**(3), 185–190 (1937)

[105] Surís, D., Menon, S., Vondrick, C.: ViperGPT: Visual inference via Python execution for reasoning. In: Proc. IEEE/CVF ICCV, pp. 11888–11898 (2023)

[106] SYSTRAN: faster-whisper: Faster Whisper transcription with CTranslate2. Software (2023–2025). URL https://github.com/SYSTRAN/faster-whisper

[107] Szeliski, R.: Computer Vision: Algorithms and Applications, 2nd edn. Springer, Cham, Suiza (2022)

[108] Tailwind Labs: Tailwind CSS v4. Documentación (2025). URL https://tailwindcss.com

[109] Terven, J., Córdova-Esparza, D.M., Romero-González, J.A.: A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS. Machine Learning and Knowledge Extraction **5**(4), 1680–1716 (2023)

[110] van den Oord, A., et al.: WaveNet: A generative model for raw audio. arXiv:1609.03499 (2016)

[111] Vaswani, A., et al.: Attention is all you need. In: Advances in NeurIPS, pp. 5998–6008 (2017)

[112] Vinyals, O., Toshev, A., Bengio, S., Erhan, D.: Show and tell: A neural image caption generator. In: Proc. IEEE CVPR, pp. 3156–3164 (2015)

[113] Viola, P., Jones, M.: Rapid object detection using a boosted cascade of simple features. In: Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), pp. I–511–I–518 (2001)

[114] W3C: Media Capture and Streams. W3C Candidate Recommendation (2025). URL https://www.w3.org/TR/mediacapture-streams/

[115] W3C: MediaStream Recording. W3C Working Draft (2025). URL https://www.w3.org/TR/mediastream-recording/

[116] Wang, C.Y., Liao, H.Y.M., Wu, Y.H., Chen, P.Y., Hsieh, J.W., Yeh, I.H.: CSPNet: A new backbone that can enhance learning capability of CNN. In: Proc. IEEE/CVF CVPR Workshops, pp. 390–391 (2020)

[117] Wang, L., et al.: A survey on large language model based autonomous agents. Frontiers of Computer Science **18**(6), 186345 (2024)

[118] Wang, X., et al.: Executable code actions elicit better LLM agents. In: Proc. ICML (2024)

[119] Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: Advances in NeurIPS, pp. 24824–24837 (2022)

[120] WHATWG: Fetch Living Standard — CORS protocol (2025). URL https://fetch.spec.whatwg.org/#http-cors-protocol

[121] WHATWG: HTML Living Standard — The canvas element (2025). URL https://html.spec.whatwg.org/multipage/canvas.html

[122] Wu, C., et al.: Visual ChatGPT: Talking, drawing and editing with visual foundation models. arXiv:2303.04671 (2023)

[123] Xu, K., et al.: Show, attend and tell: Neural image caption generation with visual attention. In: Proc. ICML, pp. 2048–2057 (2015)

[124] Yang, Z., et al.: MM-REACT: Prompting ChatGPT for multimodal reasoning and action. arXiv:2303.11381 (2023)

[125] Yao, S., et al.: ReAct: Synergizing reasoning and acting in language models. In: Proc. ICLR (2023)

[126] You, E., et al.: Vite: next generation frontend tooling. Software (2020–2026). URL https://vite.dev

[127] Zhang, Y., et al.: ByteTrack: Multi-object tracking by associating every detection box. In: Proc. ECCV, pp. 1–21 (2022)

[128] Zheng, Z., Wang, P., Liu, W., Li, J., Ye, R., Ren, D.: Distance-IoU loss: Faster and better learning for bounding box regression. In: Proc. AAAI Conf. Artificial Intelligence, pp. 12993–13000 (2020)

Anexo A

Instalación, despliegue y uso

A.1. Requisitos

- Una clave de la API de OpenAI con saldo disponible (el modelo utilizado es `gpt-4o-mini`).
- Para el despliegue recomendado: Docker Desktop (o Docker Engine con el complemento Compose).
- Para el desarrollo local: Python 3.12 o 3.13 y Node.js 18 o superior.
- Un navegador moderno (Chrome, Edge o Firefox) con permiso para usar la cámara y el micrófono. Los navegadores solo conceden ese permiso en contextos seguros: `localhost` o HTTPS.
- Conexión a Internet para la API de OpenAI, la síntesis de voz y la descarga de los modelos en el primer uso.

A.2. Configuración

Toda la configuración se realiza con un fichero `.env` en la raíz del proyecto, que se crea copiando la plantilla `.env.example` y que nunca debe subirse al repositorio. La tabla A.1 recoge las variables que usa la versión actual.

La plantilla incluye otras variables (`HOST`, `PORT`, `DEBUG`, `YOLO_MODEL_PATH`, `CAMERA_INDEX`, `DETECTION_THRESHOLD`) reservadas para versiones futuras: la versión 1.2.0 no las lee y usa valores fijos en el código (modelo `yolov8n.pt` y umbral 0,5).

Variable	Servicio	Descripción
`OPENAI_API_KEY`	backend, agent-service	Clave de la API de OpenAI (obligatoria).
`BACKEND_URL`	agent-service	URL del backend para las herramientas del agente. En Compose, `http://backend:8000`; en local, `http://localhost:8000`.
`VITE_API_URL`	frontend	URL del backend vista desde el navegador (por defecto `http://127.0.0.1:8000`).
`VITE_AGENT_URL`	frontend	URL del servicio del agente vista desde el navegador (por defecto `http://localhost:8002`).

Tabla A.1: Variables de entorno utilizadas por el sistema.

A.3. Despliegue con Docker

Desde la raíz del proyecto, con el fichero `.env` configurado:

Listado A.1: Despliegue completo con Docker Compose.

```
cp .env.example .env          # y editar OPENAI_API_KEY
docker compose up --build     # construye y arranca los tres servicios
```

Una vez arrancado, la aplicación está en `http://localhost:5173`, y la documentación interactiva de las API en `http://localhost:8000/docs` (backend) y `http://localhost:8002/docs` (agente). Para usar las imágenes publicadas en Docker Hub en lugar de construirlas, basta con sustituir en `docker-compose.yml` cada directiva `build: ./servicio` por la imagen correspondiente (`image: josegicar/tfg-<servicio>:latest`).

A.4. Ejecución local para desarrollo

Cada servicio se ejecuta en su propio terminal. Se recomienda un entorno virtual de Python:

Listado A.2: Ejecución local de los tres servicios.

```
python -m venv venv && source venv/bin/activate    # en Windows: venv\Scripts\activate
pip install -r backend/requirements.txt -r agent-service/requirements.txt

cd backend && uvicorn main:app --reload                    # puerto 8000
cd agent-service && uvicorn main:app --port 8002 --reload # puerto 8002
cd frontend && npm install && npm run dev                   # puerto 5173
```

A.5. Primer arranque y problemas frecuentes

- **Descarga de modelos.** En el primer arranque, el backend descarga YOLOv8n (unos 6 MB) y Whisper *small* (unos 0,5 GB); la primera búsqueda de texto descarga los modelos de EasyOCR (unos 100 MB). Las primeras respuestas son, por tanto, más lentas.
- **Cámara o micrófono bloqueados.** Hay que conceder el permiso en la barra de direcciones del navegador y acceder por `localhost` (o HTTPS si se accede desde otro equipo).
- **«Límite de peticiones alcanzado».** Es el error 429 de la API de OpenAI: basta con esperar unos segundos. Con la webcam, no conviene reducir el intervalo de análisis por debajo de 10 segundos.
- **Puertos ocupados.** Los servicios usan los puertos 5173, 8000 y 8002, que deben estar libres.

A.6. Guía rápida de uso

- **Modo Imagen.** Se sube una imagen, se escribe o dicta una pregunta y se pulsa «Analizar imagen». La respuesta aparece con las detecciones dibujadas sobre la imagen.
- **Modo Webcam.** Se activa la cámara y se escribe una pregunta: el sistema la responde sobre un fotograma cada 10 segundos. Con el modo conversación activo, el análisis se lanza con el botón «Enviar» y el asistente recuerda los turnos anteriores.
- **Modo Mini.** Se habla o escribe libremente: «enciende la cámara», «¿qué ves?», «¿dónde está el televisor?», «busca la palabra Farmacia», «quiero subir una foto» o «repite tu respuesta». Mini decide qué hacer y cambia de modo si es necesario.
- **Voz.** El micrófono (o `Ctrl` + `M`) inicia y detiene la grabación; la transcripción se añade al cuadro de texto para revisarla antes de enviarla con `Enter`. El icono de altavoz de cada respuesta la lee en voz alta.
- **Preferencias.** La barra superior permite ajustar el volumen, cambiar entre tema claro y oscuro y elegir entre seis idiomas; la guía de uso completa se abre con el icono de información junto al subtítulo.

Anexo B

Referencia de la API

Todos los *endpoints* aceptan peticiones `POST` con cuerpo `multipart/form-data`. El campo `language` (`es`, `en`, `fr`, `de`, `it` o `pt`; por defecto `es`) está presente en todos ellos. Las detecciones siguen siempre el formato `{"class": str, "confidence": float, "bbox": [x1, y1, x2, y2]}`, con las coordenadas en píxeles de la imagen recibida.

Endpoint	Campos de entrada	Respuesta
`GET /`	—	`{"status": "ok", "message": ...}`
`/api/analyze`	`image` (fichero), `query` ($\leq$ 1000 caracteres)	`answer`, `detections`. Error 400 si la consulta es demasiado larga.
`/api/analyze-frame`	`frame_base64` (*data URL* JPEG), `query`	`answer`, `detections`; `error` si el fotograma no es válido.
`/api/chat`	`image` o `frame_base64`, `query`, `history` (JSON, por defecto `[]`)	`answer`, `detections`, `history` (JSON actualizado, $\leq$ 12 mensajes, una sola imagen).
`/api/analyze-color`	`frame_base64`	`colors` (lista de `{name, percentage}` $> 5\,\%$), `dominant`.
`/api/stt/transcribe`	`audio` (fichero, p. ej. WebM/Opus)	`text`, `language`; `error` si falla la transcripción.
`/api/tts/synthesize`	`text`	Fichero `audio/mpeg` (MP3 de 24 kHz).

Tabla B.1: Endpoints del backend (puerto 8000).

Endpoint	Campos de entrada	Respuesta
`/api/mini`	`query`; opcionales: `image`, `frame_base64`, `history` (JSON)	`answer` (sin etiquetas), `action` (`POINT_TO`, `ACTIVATE_WEBCAM`, `OPEN_FILE_PICKER`, `SPEAK_LAST` o `null`), `detections`, `history`.

Tabla B.2: Endpoint del servicio del agente (puerto 8002).

En caso de alcanzar el límite de peticiones de OpenAI, los *endpoints* del backend devuelven una respuesta normal cuyo campo `answer` contiene un mensaje explicativo en el idioma solicitado y una lista de detecciones vacía.

Anexo C

Diario de desarrollo

La tabla C.1 resume el diario de sesiones mantenido durante el proyecto (`docs/diary.md`), con la duración anotada de cada sesión, sus logros principales y los problemas encontrados.

Sesión	**Fecha**	**Duración**	**Logros**	**Problemas**
1	31/03/2026	0 h 52	Primera aplicación FastAPI y entorno virtual.	—
2	02/04/2026	1 h 18	Backend MVP: YOLO detectando y OpenAI respondiendo.	Clave de la API y saldo mínimo.
3	02/06/2026	1 h 17	Repositorio limpio, `.gitignore`, primera versión del frontend.	Pesos de YOLO subidos por error.
4	12/07/2026	0 h 40	Rediseño con Tailwind, icono y cajas con `OverlayCanvas`.	Configuración de Tailwind v4.
5	13/07/2026	2 h 12	GPT-4o viendo la imagen, historial con miniaturas, 6 idiomas.	YOLO no detecta en ilustraciones.
T	18/07/2026	0 h 26	Estudio de smolagents.	—
6	03/08/2026	2 h 43	Modo webcam, `/api/analyze-frame`, cajas en directo, *polling*.	Límite de peticiones, vídeo no listo, historial duplicado.
7	05/08/2026	3 h 28	Webcam estable, iconos, gestión del límite, versión v0.2.0.	Límite de peticiones.
8	10/08/2026	3 h 02	Análisis de color HSV, inicio de la memoria y de Docker.	La memoria no era fluida.
9	15/08/2026	4 h 32	Memoria con `/api/chat`, modo conversación, Docker Compose.	Paquete obsoleto en Debian, `env_file`, pasado frente a presente.
10	29/08/2026	2 h 25	Whisper STT, micrófono, política de ramas.	Dependencia ausente en Docker, permisos del micrófono.

Sesión	Fecha	Duración	Logros	Problemas
11	30/08/2026	6 h 10	Edge-TTS en 6 idiomas, nombre «Asistente Mini», `config.ts`.	Coqui incompatible con Python 3.13; Whisper *base* impreciso.
12	31/08/2026	2 h 37	Sincronización de animaciones, inicio de smolagents.	Validación de `BaseTool` (resuelto con `@tool`).
13	01/09/2026	7 h 32	Agente Mini completo, interfaz del modo Mini, guía de uso.	Diseño de tareas, fuentes visuales opuestas, scroll del modal.
14	10/09/2026	4 h 15	Temas claro y oscuro.	—
15	25/09/2026	5 h 20	EasyOCR, volumen global, Docker Hub, versión v1.2.0, atajo `Ctrl` + `M`.	Dependencias de OpenCV en Docker; fragmentación del OCR.

Tabla C.1: Resumen del diario de sesiones de desarrollo.